



Octopus

A cycle-accurate memory-system simulator
for predictable multicores

Reference manual

Fanos Research Lab · McMaster University

Revision dc1f53dd · October 1, 2026

<https://github.com/FanosResearch/OctopusSimulator>

About this manual

Octopus is a cycle-accurate simulator of the memory system of a multicore: private caches, interconnects, a shared last-level cache, coherence and main memory, every layer a configurable component and the coherence protocol an editable table. It was built so that mechanisms for *predictable* multicores — arbiters, partitioned caches, real-time coherence protocols, predictable memory controllers — could be built, observed and compared on one substrate rather than each in its own bespoke simulator.

This manual is the reference for the simulator as it ships on the `esweek-tutorial` branch. It is written for three readers: someone running experiments, who needs Chapters 2, 10 and 11 and the appendices; someone reading or extending the code, who needs Chapter 3 to Chapter 8 and Chapter 13; and someone following the hands-on tutorial, who has Chapter 15 and the figures the exercises point at.

How to read it. Chapters are self-contained but ordered: the system (Chapter 3), its central component (Chapter 4), the shipped LLC in detail (Chapter 5), the coherence engine (Chapter 6), one request end to end (Chapter 7), then the interconnects, memory, configuration and monitoring, then tasks and the demo, extension, integration, the tutorial and evaluation. Every chapter opens with a box saying what it holds and closes with a *Where in the code* box naming the files. Two other boxes recur:

Trap

Something that bites in practice — a rule that is easy to break, with the symptom and the fix.

Design note

A design fact that is not obvious from the code or the configuration — what the model does and does not do, and why.

Figures use one palette throughout: **amber** is waiting, **purple** a bus transfer, **teal** being served, **red** damage or a fault. Code, keys and file names are set in monospace; configuration keys are indexed at the back.

Where the sources live. The manual is built from `docs/manual/` by `build.sh`; the SVG figures it embeds are the ones in `docs/imgs/`, the tables of configuration keys and FSMs are generated from `configuration/` and `Protocols_FSM/` at build time, and the Markdown documents under `docs/` remain the per-topic references this manual consolidates. The revision stamp on the title page is the commit it was built from.

Contents

About this manual	ii
1 Introduction	1
1.1 The problem	1
1.2 Five ways to evaluate an idea	1
1.3 What Octopus is	2
1.4 Built on Octopus	2
1.5 What it is not	3
2 Getting started	4
2.1 Prerequisites and build	4
2.2 Zero-install: a Codespace	4
2.3 Benchmarks	4
2.4 Is the environment healthy?	5
2.5 A first run	5
2.6 What comes out	5
3 System organization	7
3.1 Design principles	7
3.2 The clock model	8
3.3 Component map	8
3.4 The system as shipped	8
4 The cache controller	12
4.1 One class, every level	12
4.2 From message to actions	13
4.3 The data handler	14
4.4 Admission and back-pressure	14
4.5 The LLC's extra duties	15
4.6 Message encoding	15
4.7 Delivery order at the bus interface	15
5 The last-level cache datapath	17
5.1 The datapath	17

5.2	A read hit	17
5.3	A write hit	20
5.4	A read miss into a free way	20
5.5	A miss that evicts	22
6	The coherence engine: CSV finite-state machines	24
6.1	Coherence as data	24
6.2	The table	24
6.3	Stable and transient states	26
6.4	Snooping and directory	26
6.5	A worked example: MI without C++	27
7	A request, end to end	29
7.1	The journey	29
7.2	The stamps	29
7.3	The columns	29
7.4	Checking a column	33
8	Interconnects and arbitration	34
8.1	Topology, controller, interfaces	34
8.2	The bus family	34
8.3	Arbiters	35
8.4	Mesh and NoC	36
9	Main memory	38
9.1	Two models, one interface	38
9.2	Clocks at the boundary	39
10	Configuration	40
10.1	One model for every component	40
10.2	Files and inheritance	40
10.3	Typed keys	41
10.4	Data and code	41
10.5	Presets, and the protocol axis	41
10.6	The sweep harness	42
11	Monitoring: Logger, Debugger, trace and visualizer	44
11.1	The Logger	44
11.2	The Debugger	46
11.3	The raw event trace	47
11.4	Triage: when a run wedges	48
11.5	The visualizer	48
12	Periodic tasks and the interference demo	49

12.1 Periodic tasks	49
12.2 Inclusion interference	50
12.3 Way reservation	51
12.4 The demo	51
13 Extending Octopus	53
13.1 The map	53
13.2 An arbiter	53
13.3 A protocol change	55
13.4 A third cache level	55
13.5 Planned	56
14 Integration modes: gem5 and the full-system stack	57
14.1 Only the endpoints change	57
14.2 Full system: gem5, Arm, Linux	58
14.3 Running a session on it	59
15 Hands-on tutorial	60
15.1 How the exercises are built	60
15.2 00 — Setup and first run	60
15.3 01 — Exploration, one axis at a time	61
15.4 02 — The configuration model	61
15.5 03 — Extending Octopus	61
15.6 04 — gem5 and the full-system stack	62
16 Evaluation	63
16.1 What is measured	63
16.2 The figures to come	63
16.3 Reproducing	63
A Configuration key reference	65
B The shipped coherence FSMs	77
B.1 MSI_splitBus_snooping	77
B.2 MSI_LLC	79
B.3 MESI_splitBus_snooping	81
B.4 MESI_LLC	83
B.5 MOESI_splitBus_snooping	85
B.6 MOESI_LLC	87
B.7 MSI_directory	89
B.8 MSI_LLC_directory	91
C Command line, scripts and environment	93
C.1 The simulator	93

C.2	Scripts	93
C.3	Environment variables	93
D	File formats	95
D.1	LatencyReport_C<n>.csv	95
D.2	Summary.csv	95
D.3	JobReport_C<n>.csv	96
D.4	The task program, task_C<n>.task.csv	96
D.5	The sweep result, results/<axis>/<suite>.csv	96
D.6	The raw event trace	96

List of Figures

1.1	The evaluation paths open to predictable-architecture research, and what each trades away.	2
1.2	The case for a common, cycle-accurate simulation framework.	2
2.1	A run's outputs and what reads them.	6
3.1	The clock model. One <code>ClockManager</code> advances every <code>ClockedObj</code> by one tick; components exchange messages only through interfaces, which the next component drains on its own tick.	8
3.2	What a configured system can contain: clusters of cores with private caches on direct interconnects, further cache levels inside or across clusters, a shared last-level cache and main memory — all of it chosen by configuration, not hard-coded.	9
3.3	The class diagram. Cache-related classes in red, interconnect-related in blue, utilities in white.	10
3.4	The system <code>MultiCoreSystem.csv</code> builds. Four trace-driven cores with private MESI L1s on a <code>TripleBus</code> , one shared inclusive LLC, a point-to-point memory bus and a fixed-latency main memory. Every number is a configuration key (Chapter A).	10
4.1	Inside a cache controller. Messages from both interfaces are serialised into one processing queue; the protocol handler turns each one into actions by looking the (state, event) pair up in the FSM table that the FSM reader parsed at start-up; the data handler executes those actions against the array, the MSHRs and the write-back buffer, arbitrating the data port.	12
4.2	One message through the controller. The handler decodes the message into an event, looks up the transition for the line's current state, and hands the controller a list of protocol-agnostic actions; the controller does not know which protocol it is running.	13
5.1	The shipped LLC as a datapath: <code>CacheController_End2End</code> with <code>CacheDataHandler_COTS</code> , 32 KB, 2-way, 64 B lines, the <code>MESI_LLC</code> FSM and one timed data port. Bus interfaces on top, control on the left, the single array in the middle read by an untimed bits path and a timed data port on the right, MSHR and write-back buffer beside it, the memory bus below.	18
5.2	A read or write hit — a <code>GetS</code> or <code>GetM</code> that finds the line in a stable state — from the request's arrival on the bus to its data leaving on the response lane, in ten numbered steps.	19

5.3	The write hit on its own. The path is the read hit's; what is new is the branch marked 5b — the FSM's SetOwner written back into the line's bits over the dashed control wire, untimed — and the plain-data response.	20
5.4	A read miss in the simplest case: the set still has a free way, so nothing is evicted. The request's own path in teal; the array write of the fill, off the critical path, dashed.	21
5.5	A miss into a <i>full</i> set: the requested line in teal, the evicted victim in amber. The victim is chosen when the fill is written, and its exit is a coherence transaction of its own.	22
6.1	An excerpt of MESI_LLC.csv with the four things to know about the format. Chapter B prints every shipped table in full.	25
6.2	One GetM from core A while core B shares the line, under the shipped snooping MESI (left) and the shipped directory MSI (right).	26
7.1	The system as shipped with one LLC <i>miss</i> from core 0 routed through it. Each hop carries the colour of the LatencyReport column that measures it (amber: waiting; purple: a bus transfer; teal: being served) and the number of the Logger stamp that bounds it, ①–⑩ being the report's nine latency columns in order; the side panel lists the stamp pair behind every column.	30
7.2	The same system with an LLC <i>hit</i> from core 0. The route stops at the LLC's array port, where the data is emitted at the grant; the memory bus and DRAM are not on the path, so L2-DRAM Bus and DRAM are 0.	30
7.3	The same two routes on a time axis: an LLC hit and an LLC miss, with the nine LatencyReport columns bracketed between the Logger stamps that bound them. Ticks are in order, not to scale. Below the miss axis, the array write of the fill runs alongside the response and lands in no column.	31
8.1	An interconnect is a topology that creates a connection map and instantiates one controller and the interfaces; the interfaces are the queues that hold messages at each node; the controller moves messages between interfaces according to the map and the configured latencies.	34
8.2	The three shapes that ship between the L1s and the LLC.	35
8.3	Three requests ready at the same cycle on one request lane — two from core 0, one from core 1 — under the three shipped arbiters.	35
9.1	The LLC, the memory bus and the two memory models with their clocks.	38
10.1	Hierarchical propagation of parameters: from the command line through the top-level system module down to each sub-component, each level able to override what it passes on.	40
11.1	The Logger's view of one request: a chain of self-describing events.	44
11.2	Every component owns a DebugPrint; instances that share a target interleave into one file, distinct targets give per-component files, an empty target prints to stdout.	46
11.3	What print does, and the rule condition applies.	46
11.4	A DebugPrint's life. A disabled instance deregisters from the clock in its constructor, so observability costs nothing when off.	47

12.1 Two cores, one LLC set, and the line core 0 loses. Core 0 brings X in and hits on it in its L1; core 1 misses on Y in the same set; the fill evicts X; the invalidation on the service lane removes X from core 0's L1 too; core 0's next Load X is a full DRAM round trip it did nothing to cause.	50
12.2 The robot's path in the three acts: alone and protected coincide exactly; under contention one 181 266-cycle job left the robot steering for 35 periods on a stale waypoint, the long chord across the figure.	52
13.1 The shipped system with a pin on every extension point. Amber pins exist; the grey one is planned.	53
14.1 Three operating modes, one hierarchy.	57
14.2 Full-system integration: gem5 executes Arm cores under Linux and holds the functional memory values; Octopus supplies the cycle-accurate timing and coherence for every request; MCsim models the DDR4 controller. The ATP engine injects aggressor traffic.	58

Introduction

What you will find here. Why shared-memory interference makes timing an end-to-end problem; the five ways the field evaluates its answers and what each gives up; what Octopus is, in four capabilities; what ships; and what it deliberately does not model.

1.1 The problem

Modern embedded platforms are multicore, but their adoption in real-time and safety-critical domains is throttled by *shared-memory interference*: the latency of a memory request depends not only on the request itself but on the concurrent activity of every other core across the interconnect, the coherence fabric, the shared cache and the DRAM controller. Deriving safe *and* tight timing bounds is therefore an end-to-end problem.

The field's first instinct is additive: analyse each resource in isolation, bound the worst wait at each, and sum. Chapter 7 shows why that is unsafe in a real hierarchy: resources interact and reorder. A request that was first onto the request bus can return last because three younger requests hit open DRAM rows and took the response channel ahead of it; a core whose working set fits its private cache still goes to DRAM because another core's streaming evicted its lines from an inclusive shared cache. Measured on a four-core system with round-robin arbitration at every resource, the worst observed latency of the oldest request was $35\times$ the per-resource bound. Mechanisms proposed in response – predictable arbiters, partitioned caches, predictable memory controllers, real-time coherence protocols – are each usually prototyped in a bespoke simulator, which makes cross-comparison, reuse and whole-hierarchy studies difficult.

1.2 Five ways to evaluate an idea

Figure 1.1 is the choice every such paper makes. An analytical model is fast and precise about what it models, and blind to the interactions above. A custom simulator models exactly the idea and nothing comparable; reviewers cannot reproduce it and the next group cannot reuse it. The general-purpose architecture simulators are reproducible but were not built for worst-case questions or for swapping a coherence protocol. RTL is exact and a one-design-point effort. COTS hardware is real and closed. What is lost without a common substrate is concrete (Figure 1.2): no fair comparison, no composability of mechanisms, no reproducibility, a high barrier to entry, and slow exploration.

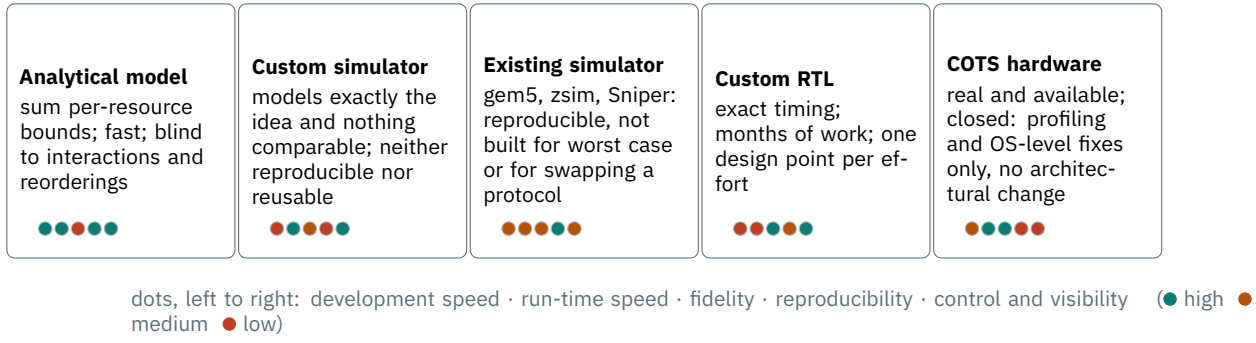


Figure 1.1. The evaluation paths open to predictable-architecture research, and what each trades away.

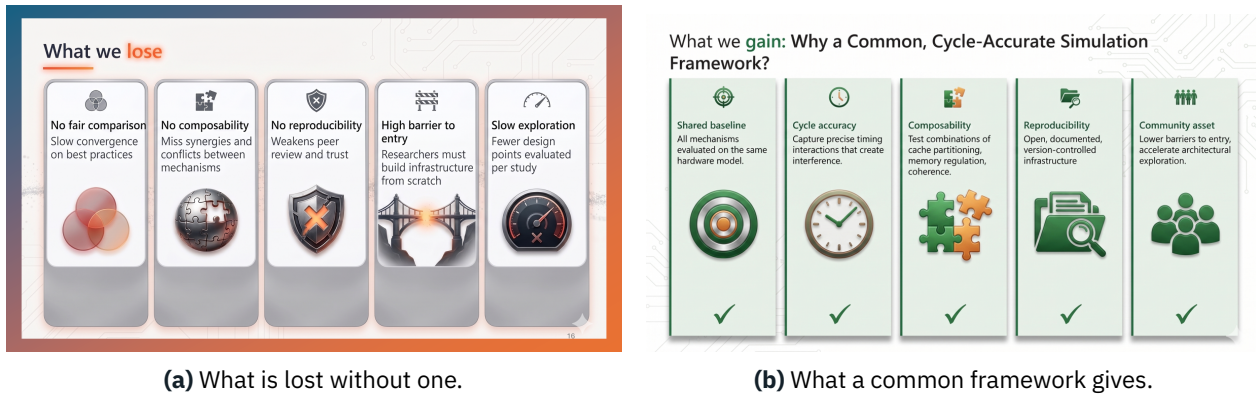


Figure 1.2. The case for a common, cycle-accurate simulation framework.

1.3 What Octopus is

Octopus is a reusable, whole-hierarchy, cycle-accurate substrate in which memory-system ideas can be built, observed, validated and verified under one roof. Unlike simulators that fix an architecture, it treats every layer as a configurable, swappable component and makes the hardest component to change — the coherence protocol — an editable spreadsheet rather than C++ (Chapter 6). Four capabilities follow, each tied to a mechanism in this manual:

Table 1.1. Four pillars, and where each is realised.

pillar	capability	chapter
Controllability	sweep any layer — coherence, arbiter, replacement, cache geometry, memory — with no recompile	10
Observability	per-request latency in nine stages with per-core worst cases; an address-filtered coherence trace; a raw trace of every message	7, 11
Timing validation	cycle-accurate worst-case latency, deterministic and platform-independent; analytical against measured	16
Verification	auditable CSV FSMs; undefined transitions halt the run; deterministic replay	6

1.4 Built on Octopus

Octopus has been the modelling and evaluation platform for a body of peer-reviewed work on real-time memory systems, developed since 2018: the simulator itself (IEEE Computer

Table 1.2. What ships, by axis. Every cell is a configuration choice or an editable CSV.

axis	ships with
coherence	snoop and directory MSI, MESI, MOESI as CSV FSMs with stable and transient states (Chapter 6); the predictable protocols listed below in research configurations
interconnect	point-to-point, unified, split and triple bus, mesh, single-switch NoC (Chapter 8)
arbitration	FCFS, round-robin, TDM on every shared lane and on the LLC port
cache hierarchy	any depth, private or shared, non-blocking with MSHRs and write-back buffers, allocate-on-fill; LRU, FIFO, random replacement; way partitioning
main memory	a fixed-latency model, or cycle-accurate DDR4 through MCsim with twenty schedulers (Chapter 9)
operating modes	standalone trace replay or periodic tasks; full system behind gem5 with Arm cores under Linux (Chapter 14)
monitoring	the Logger, the Debugger, the raw trace and the visualizer (Chapter 11)

Architecture Letters, 2024) and its gem5-integrated full-system extension (ECRTS Industrial Challenge, 2025); the predictable coherence protocols DISCO (ECRTS 2020, IEEE TC 2022), PISCOT (RTSS 2020, ACM TECS 2022), Duetto (DATE 2021), DUPECO (ECRTS 2022), PCC (ECRTS 2022) and CoHoRT (DATE 2025); the coordinated arbitration scheme GRROF (ECRTS 2023); and the earlier predictable protocols PMSI, PMESI, PENDULUM, PMSI* and PMESI*, re-implemented in Octopus so that they can be compared head to head under identical conditions. PUBLICATIONS.md keeps the list current.

1.5 What it is not

Octopus is a *timing* model of the memory system. The trace-driven core issues addresses with compute gaps between them and carries no data values; a periodic task is a fixed compute budget plus accesses. There is no instruction set, no branch, no data-dependent control flow — for those, run it behind gem5 (Chapter 14). There is no preemption and no multi-task scheduling on a core. Deadlines are observed and reported, never enforced. The DRAM model, when not MCsim, is a single fixed latency. These are the lines the demo (Chapter 12) is careful to draw between what is measured and what is drawn, and they are the lines to keep in mind when reading any number out of it.

Chapter 2

Getting started

What you will find here. From nothing to a first completed simulation and a first read of its output: the toolchain, the build, the zero-install Codespace, the benchmarks, the environment check with its fixed numbers, one run, and what comes out of it.

2.1 Prerequisites and build

A C++17 toolchain and CMake ($g++ \geq 9$ or clang; MinGW on Windows), Bash for the driver scripts, xz to inflate the compressed SPLASH-2 traces, and Python 3 with duckdb, numpy and matplotlib for the visualizer and the figures. The simulator is tested on Ubuntu, on Windows under MinGW with Git Bash, and under WSL.

```
sudo apt-get install build-essential cmake xz-utils          # Ubuntu
pip install duckdb numpy matplotlib
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release && cmake --build build -j$(
nproc)
```

The binary is `build/Octopus_Simulator` (.exe on Windows). A debug build is the same with `-DCMAKE_BUILD_TYPE=Debug`. The repository layout: `src/` and `header/` hold the simulator, `Protocols_FSM/` the coherence tables, `configuration/` the CSV parameters, `tools/` `octoviz/` the visualizer, `sweeps/` the experiment harness, `tutorial/` the exercises, `docs/` the documents this manual consolidates.

2.2 Zero-install: a Codespace

The `esweek-tutorial` branch carries a dev container. On GitHub, *Code* → *Codespaces* → *Create codespace on esweek-tutorial* gives a working simulator in a browser: the container (`ghcr.io/fanosresearch`) carries the toolchain and the Python packages, `.devcontainer/on-create.sh` fetches the EEMBC traces, `post-create.sh` builds MCsim and the simulator and runs the environment check, and port 8765 is forwarded so the visualizer opens by itself. The two scripts double as documentation of the dependencies.

2.3 Benchmarks

The traces are a separate repository, `FanosResearch/OctopusBMs`: EEMBC automotive traces run on every core to create a data-sharing stress workload, and SPLASH-2 application traces, including the raytrace and radiosity giants, stored as xz archives (about 650 MB, 10 GB

inflated). Nothing is needed by hand: every driver script calls `get_benchmarks.sh`, which clones the repository into BMs/ if absent and inflates the archives in place, idempotently. Run it yourself only to call the binary on a benchmark directly, and use the sparse form for an EEMBC-only environment:

```
./get_benchmarks.sh                # everything, ~710 MB
  download
BMS_SPARSE="eembc-traces" ./get_benchmarks.sh  # EEMBC only, ~61 MB, seconds
```

2.4 Is the environment healthy?

```
bash scripts/check_environment.sh
```

One [PASS] line per component, then a short reference simulation — the single-core DRAM exerciser under MCsim — whose result is fixed: **15364 requests, worst-case DRAM latency 359 cycles**. The simulator is deterministic and platform-independent, so those numbers are an equality check, not a smoke test: a mismatch means a stale build, an edited file under configuration/ (git checkout - configuration/ puts everything back), or a bug. The check also catches the one portability trap in the tree: a working copy checked out on Windows and copied to Linux carries CRLF line endings, which MCsim's .ini parser rejects with *Malformed Line N (missing equals)*.

2.5 A first run

a2time01 from EEMBC, four cores, on the configuration as shipped: snooping MESI, a TDM-arbitrated TripleBus, fixed-latency memory (Section 3.4).

```
W=$PWD/BMs/eembc-traces/a2time01-trace      # Git Bash on Windows: W=$(
  cygpath -m "$PWD/BMs/eembc-traces/a2time01-trace")
./build/Octopus_Simulator -s MultiCoreSystem -p "workload_path(s)=$W/"
```

It takes a few seconds. -s names the system — the class `MultiCoreSystem` and configuration/ `SystemConfigurations/MultiCoreSystem.csv` — and -p overrides any key in it; Chapter 10 is about little else. The suite drivers wrap the same call: `./run_octopus.sh -protocol snoop -suite eembc -bench a2time01-trace` runs one benchmark under the snoop-MSI preset and prints a completion verdict.

2.6 What comes out

Everything lands in the workload's `newLogger/` directory (Figure 2.1):

```
column -s, -t $W/newLogger/Summary.csv | less -S          # one row per core
: worst cases, finish cycle
head -3 $W/newLogger/LatencyReport_C0.csv | column -s, -t # one row per
request
```

`Summary.csv` gives, per core, the worst-case latency of every stage and the finish cycle. `LatencyReport_C<n>.csv` has one row per memory request with its latency split into stages: CPU, L1 stall, request bus, L2 stall, L2 access, response bus, L2-DRAM bus, DRAM, L1 access. A total tells you a core was slow; the stages tell you *which shared resource* made it slow, and that

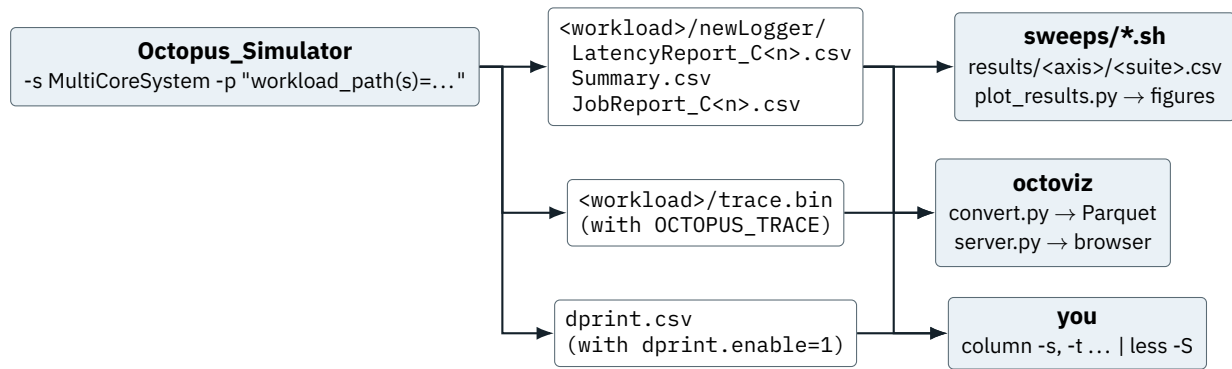


Figure 2.1. A run's outputs and what reads them.

decomposition is the reason the tool exists. Chapter 7 draws where each column is spent; keep it open while reading a first report.

To *see* the run rather than read it:

```
./octoviz.sh view BMs/eembc-traces/a2time01-trace
```

About eight seconds: it re-runs the benchmark with the raw event trace on, converts it and serves it, printing a URL in a codespace or opening the browser on a laptop. Three views to find, in this order: the request pipeline, the resource lanes under it, and the coherence transition table (Chapter 11).

The preset copy

```
./run_octopus.sh -protocol <snoop|directory> copies a preset over
MultiCoreSystem.csv. Any hand edit to that file is overwritten. Keep edits in -p
overrides, or expect to git checkout - configuration/.
```

Where in the code

- CMakeLists.txt – the build
- .devcontainer/devcontainer.json, .devcontainer/on-create.sh, .devcontainer/post-create.sh – the Codespace
- get_benchmarks.sh – fetches and inflates the benchmark traces
- scripts/check_environment.sh – the environment check
- run_octopus.sh – runs a suite under a protocol preset
- octoviz.sh – the visualizer entry point
- README.md, REPRODUCIBILITY.md – the same material online

System organization

What you will find here. The four principles the simulator is built on and the two base classes that carry them; the clock model, which is the reason every latency it reports is comparable; the map of components and the classes behind them; and the exact system that `MultiCoreSystem.csv` builds when you run the shipped configuration — the system every later chapter’s figures are drawn on.

3.1 Design principles

Octopus models a multicore memory hierarchy as a *network of clocked, configurable components* joined by bounded, back-pressured *interfaces*. A memory request is a first-class `Message` object that travels this network; a single global clock steps every component; and every hop is timestamped for the `Logger`. Four principles shape the code, and they are realised through two base classes that every component inherits (Table 3.1).

- **Modularity.** Each component is self-contained behind a narrow interface, so changing one does not ripple into others. A cache controller does not know whether it is an L1 or the LLC, nor which protocol it runs; a bus does not know what it carries.
- **Extensibility.** Inheritance and polymorphism: a new arbiter, replacement policy, topology or memory model is a subclass with a handful of virtual methods; a new coherence protocol is a CSV file. The effect shows in the code size of protocols derived from a common family (Chapter 6).
- **Configurability.** Every component derives from `Configurable`: it reads its defaults from its own CSV, a parent may override a child’s, and the command line overrides everything, so the whole design space is reachable without recompiling (Chapter 10).
- **Integrability.** The two endpoints — the request source and the memory sink — are swappable for external simulators: a trace-driven CPU or `gem5` on one side, a fixed-latency model or `MCsim`’s DDR controllers on the other, with the hierarchy between them unchanged (Chapter 14).

Table 3.1. The two base classes every component inherits.

base class	gives every component
<code>ClockedObj</code>	<code>m_clk_period</code> and <code>cycleProcess()</code> : the component can be advanced one tick by the <code>ClockManager</code>
<code>Configurable</code>	<code>name</code> , <code>parent_name</code> , <code>parameters</code> , <code>parseFile()</code> : hierarchical parameters read from CSV and overridable from the CLI

3.2 The clock model

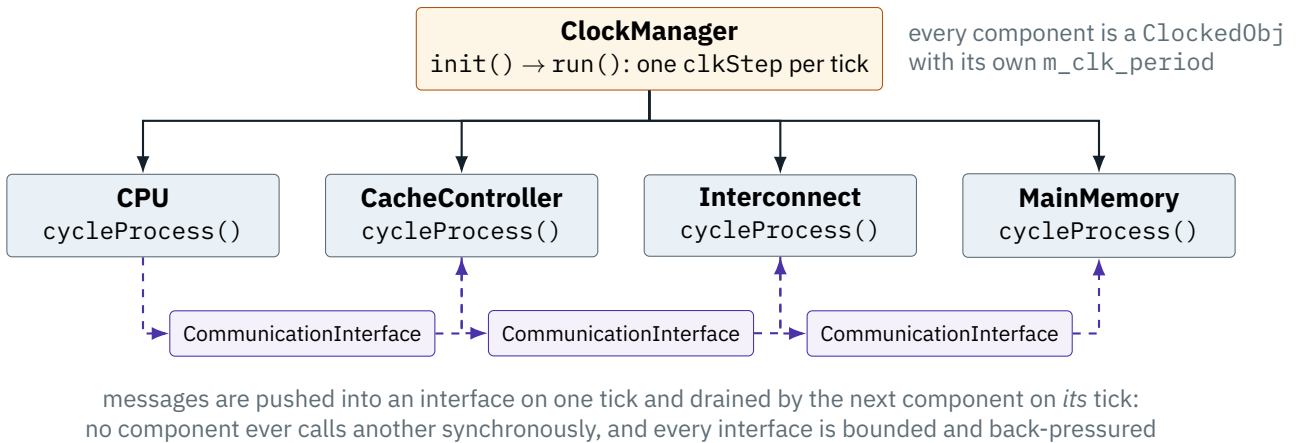


Figure 3.1. The clock model. One ClockManager advances every ClockedObj by one tick; components exchange messages only through interfaces, which the next component drains on its own tick.

The ClockManager owns the simulation loop. On each clkStep it advances every ClockedObj whose period has elapsed by calling its cycleProcess() (Figure 3.1). Components never call each other synchronously: they communicate only by placing Messages into interfaces, which the receiving component drains when it is next stepped. That decoupling is what makes the hierarchy modular and back-pressured – an interface that is full simply refuses the push, and the sender tries again on its next tick.

Because all components step under one manager, latencies are consistent system-wide: the Logger measures every stage of a request in the *originating core's* clock, whichever component stamps it (Chapter 11). Each component carries its own m_clk_period, so a memory model may legitimately tick at a different rate from the cores; in the shipped configuration every period is 100 and one tick is one cycle everywhere.

3.3 Component map

A system is clusters of CPU cores, each with private caches and a direct interconnect, joined through configurable interconnects to a shared LLC and main memory (Figure 3.2). Table 3.2 names the class behind each role; Figure 3.3 shows how they relate.

3.4 The system as shipped

Running `Octopus_Simulator -s MultiCoreSystem` instantiates the C++ system class `MultiCoreSystem` and configures it from `configuration/SystemConfigurations/MultiCoreSystem.csv`. Figure 3.4 is that system; it is the one every route, datapath and sequence figure in this manual is drawn on, and the one the tutorial runs first. The points that matter when reading a report:

- **Four cores** (num_cores), each a trace-driven CPU with an out-of-order window of eight outstanding requests, and each with a private L1 of `CacheControllerExclusive` running snoop MESI – 16 KB, direct-mapped, 64-byte lines, an untimed data port, 16 MSHRs, an 8-entry write-back buffer and a 16-deep processing queue.

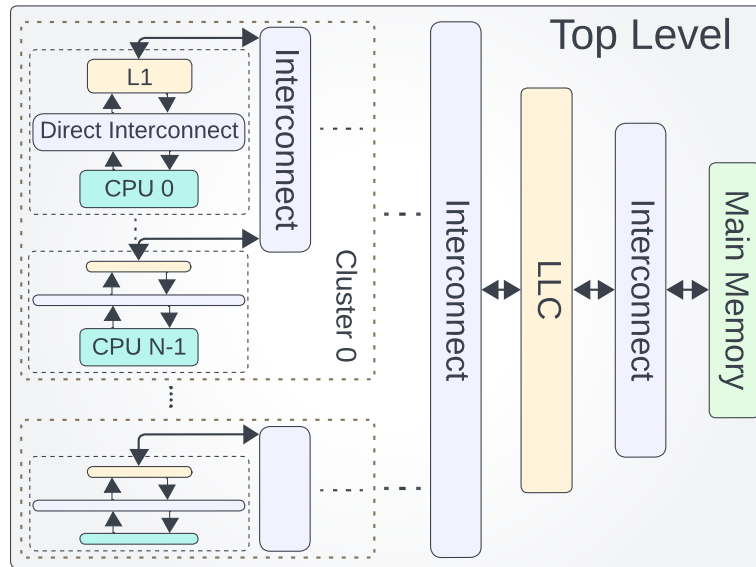


Figure 3.2. What a configured system can contain: clusters of cores with private caches on direct interconnects, further cache levels inside or across clusters, a shared last-level cache and main memory – all of it chosen by configuration, not hard-coded.

Table 3.2. Roles and the classes that play them.

role	class(es)	notes
clock	ClockManager, ClockedObj	drives cycleProcess() each tick
configuration	Configurable	hierarchical CSV + CLI parameters
core	CPU (trace), ExternalCPU (gem5)	issues Messages; the trace CPU has an out-of-order window of m_number_of_0o0_requests
transport	CommunicationInterface	pushMessage / peekMessage / popFrontMessage; bounded
message	Message	msg_id, addr, cycle, owner, source, data, kind
cache	BaseController → CacheController → CacheControllerExclusive, CacheController_End2End, CacheControllerDirectory	processing queue + handlers (Chapter 4)
coherence	CoherenceProtocolHandler → MSI / MESI / MOESI, their LLC and directory variants	driven by FSMReader (Chapter 6)
data	CacheDataHandler → array, MSHRs, write-back buffer, CacheDataHandler_COTS replacement, port arbiter	
interconnect	InterconnectTopology + bus, mesh, NoC (Chapter 8) InterconnectController + Arbiter	
memory	MainMemoryController (fixed latency), MCsimInterface	the sink (Chapter 9)
monitoring	Logger, DebugPrint	Chapter 11

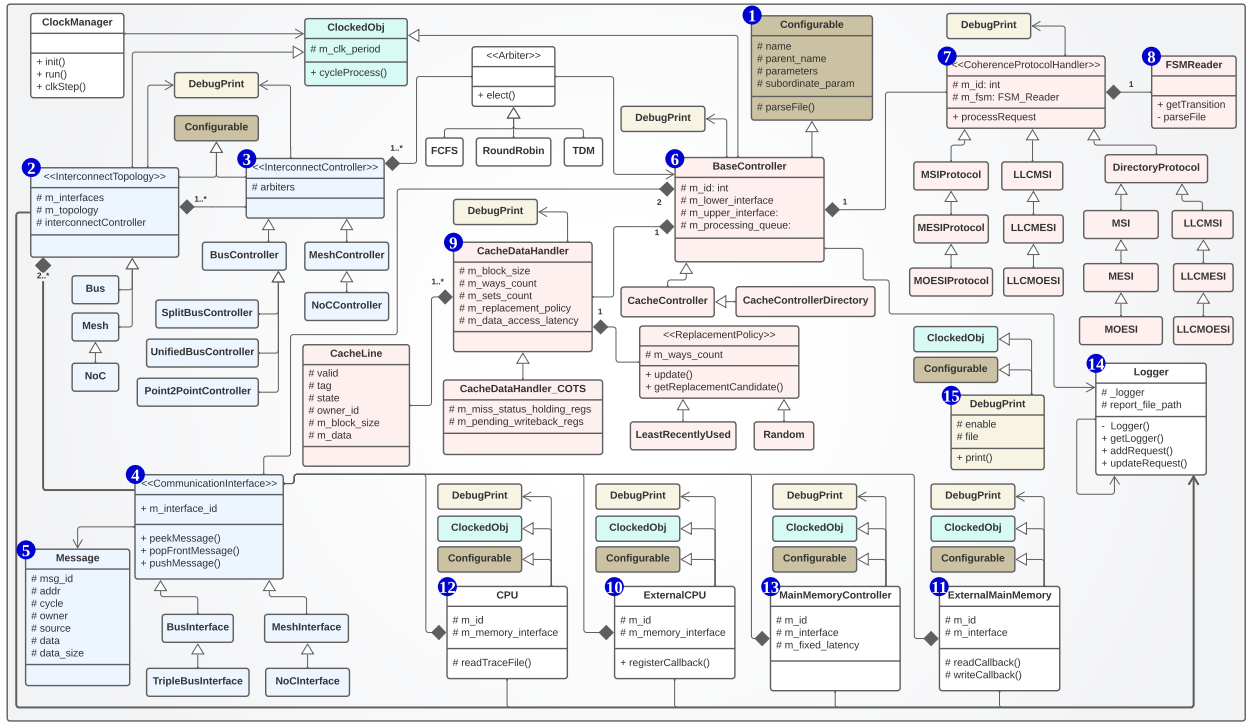


Figure 3.3. The class diagram. Cache-related classes in red, interconnect-related in blue, utilities in white.

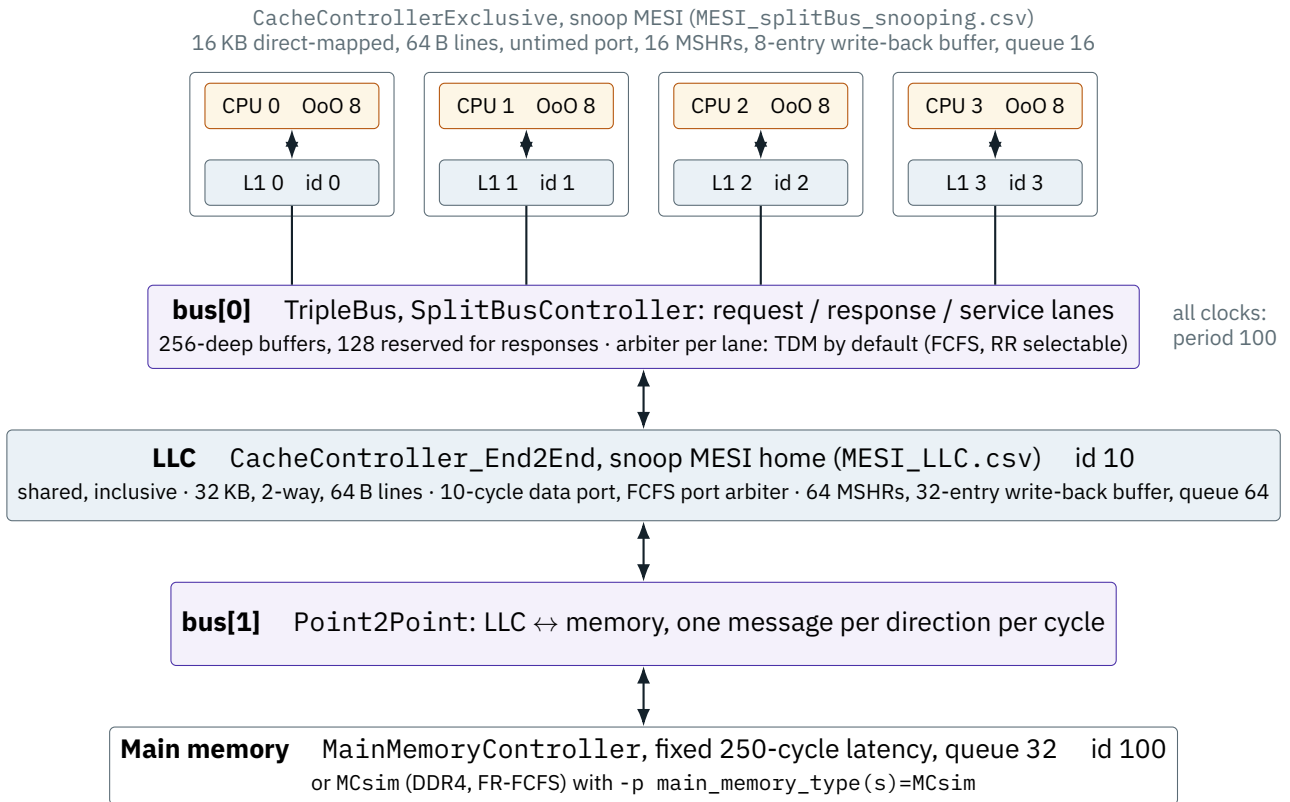


Figure 3.4. The system MultiCoreSystem.csv builds. Four trace-driven cores with private MESI L1s on a TripleBus, one shared inclusive LLC, a point-to-point memory bus and a fixed-latency main memory. Every number is a configuration key (Chapter A).

- **bus[0]** is a TripleBus under a SplitBusController: three lanes — request, response, service — each with its own arbiter, TDM by default (`SplitBusController.csv`); 256-deep buffers with 128 entries reserved for in-flight responses so that the request lane stalls before responses can overflow.
- **The LLC** (id 10) is a `CacheController_End2End`: shared, inclusive, 32 KB, 2-way, 64-byte lines, a 10-cycle data port with an FCFS port arbiter, 64 MSHRs, a 32-entry write-back buffer and a 64-deep queue. Its `MESI_LLC` FSM is the home of the protocol.
- **bus[1]** is point-to-point between the LLC and memory (id 100).
- **Main memory** is a `MainMemoryController` with a fixed 250-cycle latency; `-p "main_memory_type(s swaps in cycle-accurate DDR4 (Chapter 9).`

Two sibling presets build the same topology with a different protocol: `MultiCoreSystem_Snoop.csv` (snoop MSI) and `MultiCoreSystem_Directory.csv` (directory MSI, pinned to an FCFS bus); `MultiCoreSystem_Mesh.csv` replaces the bus with a mesh (Chapter 8). A protocol is five coupled keys, which is why these are presets and not single `-p` overrides (Chapter 10).

Where in the code

- `header/ClockManager.h`, `src/ClockManager.cpp` — the clock and `ClockedObj`
- `header/Configurable.h`, `src/Configurable.cpp` — parameters
- `header/Interconnect/CommunicationInterface.h` — Message and the interfaces
- `src/SystemConfigurations/MultiCoreSystem.cpp` — the system class that wires Figure 3.4
- `configuration/SystemConfigurations/MultiCoreSystem.csv` — its parameters
- `Octopus_Simulator.cpp`, `src/CacheSim.cpp` — main and the run

The cache controller

What you will find here. What every cache in Octopus is made of, whether it is a private L1 or the shared LLC: two interfaces, one processing queue, a protocol handler that reads a CSV state machine, and a data handler that owns the array, the MSHRs and the write-back buffer. How a message becomes a list of actions; what the controller admits and what it stalls; how the fields of a Message are decoded; and the one ordering rule at the bus interface that a snoop protocol's correctness rests on.

4.1 One class, every level

A cache controller is the same object at every level of the hierarchy. `BaseController` holds the machinery; `CacheController` adds the data array, the miss handling and the write-back path; `CacheControllerExclusive` (the shipped L1), `CacheController_End2End` (the shipped LLC) and `CacheControllerDirectory` (the directory home) specialise only what differs. What makes one instance an L1 and another the LLC is its position in the system and the FSM table it is given, not its class.

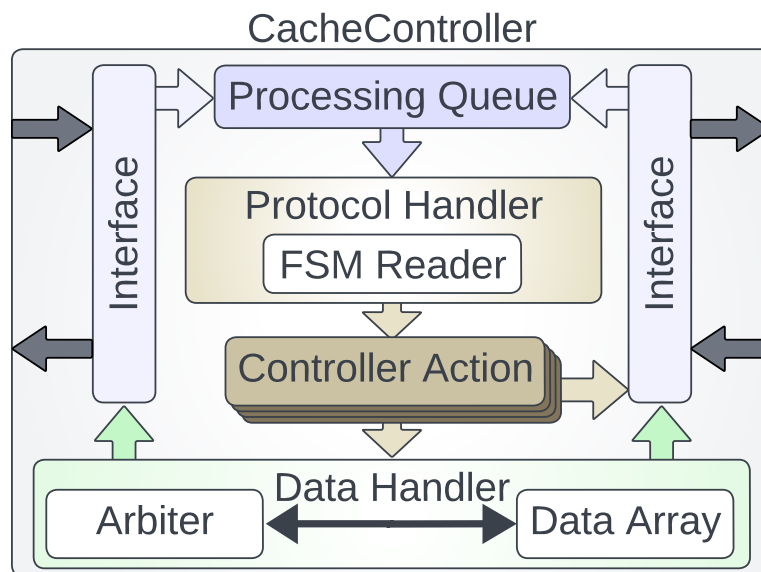


Figure 4.1. Inside a cache controller. Messages from both interfaces are serialised into one processing queue; the protocol handler turns each one into actions by looking the (state, event) pair up in the FSM table that the FSM reader parsed at start-up; the data handler executes those actions against the array, the MSHRs and the write-back buffer, arbitrating the data port.

Two interfaces. Every controller has a *lower* CommunicationInterface, facing the cores, and an *upper* one, facing the interconnect towards memory. Demand requests arrive from below; coherence requests, snooped transactions and data arrive from above; the controller's own requests leave through the upper interface. Interfaces are the only way in or out (Chapter 3): they are bounded queues, so a full interface pushes back on whoever is trying to fill it.

The processing queue. Messages from both interfaces are serialised into one queue ordered *first-ready, first-come-first-served* (FRFCFS_Buffer). A message is *ready* when the protocol handler judges that its line is in a state that can accept it; a message whose line is in a transient state waits without blocking the ones behind it that can proceed. The queue is bounded by `processing_queue_size`, and the bound applies to demand admission only: maintenance traffic the controller generates for itself, such as a write-back draining the buffer, is admitted with `force=true` so it can never be deadlocked by the demand it is making room for.

Per-line order. Among messages to the *same* line the queue keeps strict arrival order for demand requests and invalidations together; data messages are exempt, so a fill or a supply can always be processed and can never wait behind a stalled request. This per-line FCFS gate is what prevents an eviction's invalidation from leapfrogging an older GetS/GetM to the same line, and the Logger reports when a request was held by it (LLC Gate, Chapter 11).

4.2 From message to actions

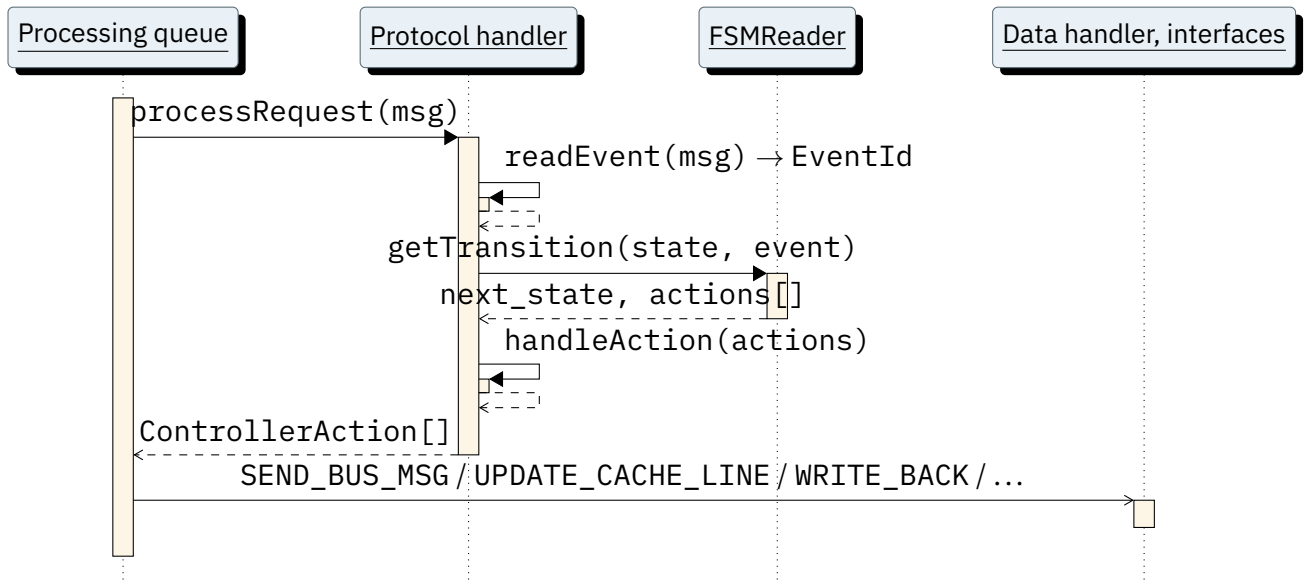


Figure 4.2. One message through the controller. The handler decodes the message into an event, looks up the transition for the line's current state, and hands the controller a list of protocol-agnostic actions; the controller does not know which protocol it is running.

The protocol handler (CoherenceProtocolHandler, specialised per family: MSIProtocol, MESIProtocol, MOESIProtocol, their LLC. . . and . . . Directory variants) does three things for every message (Figure 4.2):

1. **Decode the event.** `readEvent` maps the message to one column of the FSM table from its *source* and *contents*: a Load/Store from below; an Own_GetS/Own_GetM when the con-

troller sees its own request echoed back from the bus and `Other_GetS/Other_GetM` when it is another core's; `OwnData` when the message carries data for a line it is waiting on; `Invalidation` for an LLC back-invalidation; `Replacement` for the controller's own eviction.

2. **Look up the transition.** `FSMReader::getTransition(state, event)` returns the actions and next state the CSV table lists for that cell (Chapter 6).
3. **Translate to controller actions.** `handleAction` turns protocol actions such as `GetS`, `Data2Both` or `IssueInv` into `ControllerActions` the controller can execute without knowing the protocol: `SEND_BUS_MSG`, `UPDATE_CACHE_LINE`, `WRITE_BACK`, `HIT_Action`, `REMOVE_PENDING`, ...

A cell that reads `Fault/` means “this (state, event) must be unreachable”; the handler halts the run and reports it rather than mis-routing silently. That is what makes a protocol auditable: an incomplete table fails loudly, on the first trace that exercises the hole.

4.3 The data handler

`CacheDataHandler_COTS` owns everything that holds a line:

- **The array.** One array per controller, indexed by set, holding each line's tag, coherence bits and data together. Reading a line's *bits* (the tag compare, the state the FSM needs) is untimed and claims nothing. Reading or writing its *data* goes through a single port that is busy for `m_data_handler.m_data_access_latency` cycles per access; demand reads, eviction reads and fill or write-back writes contend for it through a round-robin arbiter (`m_data_access_arbiter`).
- **The MSHRs.** A miss allocates one entry and waits there, not in the array, until its data returns. `num_mshr` bounds the number of outstanding misses; `-1` removes the bound.
- **The write-back buffer.** A victim moves out of the array into the buffer, bits and data, the moment its way is needed, and leaves for memory later as a separate transaction. `m_data_handler.pwb_size` bounds it.
- **Replacement.** LRU, FIFO or random, chosen per controller; under way partitioning (Chapter 12) a core's fills install into, and evict from, its own ways only.

A line that currently sits in the MSHR or the write-back buffer is read without waiting for the port, because it is not in the array yet; it still restarts the port's busy timer, which is why such accesses cut in ahead of an access already in progress (the dotted bypass in Figure 5.1). Fill writes are not on the requester's critical path: the response is emitted from the fill message itself and the array write runs afterwards (Chapters 5 and 7).

4.4 Admission and back-pressure

A new demand miss is admitted only when the controller can finish it. `canAdmitRequest` stalls a demand request at the lower interface when the MSHRs are full or the write-back buffer has no headroom; `demandAdmissionBlocked` adds those two bounds to the processing-queue bound. Responses and the controller's own maintenance traffic are never subject to these checks, which is what keeps the system deadlock-free under finite buffers: a full queue of stalled requests can always drain, because the data they wait for is always admitted.

The default depths of every queue and buffer are in Chapter A.

4.5 The LLC's extra duties

The shipped LLC (`CacheController_End2End`) is *inclusive*: every line an L1 holds is also in the LLC. That gives it two duties an L1 does not have. When it evicts a line it must remove the L1 copies, which it does with the FSM action `IssueInv`: an INV broadcast on the TripleBus *service* lane that every L1 sees and that comes back to the LLC itself as `Own_Invalidation` to complete the eviction (Chapter 5). And it tracks, per line, which L1 currently owns it (`SetOwner/ClearOwner`), so that it can stay out of the way when an owner answers a request on the bus. The directory controller (`CacheControllerDirectory`) replaces snooping with a sharer list at the home and forwards requests point-to-point; Chapter 6 contrasts the two.

4.6 Message encoding

Every transaction is one Message object. The decoders in the protocol handlers read a small set of fields (Table 4.1); knowing them is what makes a Debugger trace or the raw event trace readable.

Table 4.1. The Message fields a decoder reads.

field	meaning
source	where the message entered the controller: LOWER_INTERCONNECT (from the cores' side), UPPER_INTERCONNECT (from the bus or memory side), SELF (a replacement the controller raised for itself)
data	NULL for a request or command; non-NULL for a data transfer
complementary_value	on a request, its type: 0 GetS, 1 GetM, 2 PutM, 10 INV (CPU→L1: 0 Load, 1 Store). On data it is <i>inherited</i> from the message the data was copied from: provenance, not a kind
owner	the requester on requests, supplies and fills; the evicting L1 on an eviction write-back; the LLC on an invalidation
to	destination interface ids; a response is delivered only to these, a broadcast request to everyone
msg_id	a demand request keeps one id from GetS/GetM through fill and response; an eviction write-back has its own id; a write-back forced by a back-invalidation carries the LLC eviction's id
kind	the producer's classification of the message (DEMAND, GETS, GETM, PUTM, INV, MEM_READ, EVICT, WB_DATA, WB_INV, SUPPLY, SUPPLY_DEFERRED, RESP, FILL, FILL_ROLLBACK, MEM_WRITE), set where it is created because the fields above cannot be read back into one

The bus `MessageType` passed to `pushMessage` (REQUEST, DATA_RESPONSE, SERVICE_REQUEST) selects the lane the message travels on; it is not stored in the message. Three things the same value means are worth remembering: `cv = 2` is a PutM on a request, an eviction write-back at the LLC and an exclusive grant at a MESI L1; `owner` is the requester for requests, supplies and fills but the LLC for invalidations; and `msg_id` joins a demand request across its life, while an invalidation-forced write-back is attributed to the LLC eviction, not to any core's request. The full producer-by-producer table is in `docs/MessageEncoding.md`.

4.7 Delivery order at the bus interface

A snoop protocol is correct only if every controller observes bus transactions in the *same* order. `BusInterface` therefore stamps each delivery into a controller's RX with one global

sequence number, and `TripleBusInterface` hands out a service-lane message (an LLC back-invalidation) only when it was delivered *before* the request at the head of the request RX. Data responses are deliberately not ordered against requests, so that a full queue of stalled requests can always drain.

Where in the code

- `header/CacheControllers/BaseController.h`,
`src/CacheControllers/BaseController.cpp` – the machinery every controller shares
- `header/CacheControllers/CacheController.h`,
`src/CacheControllers/CacheController.cpp` – the data array, miss handling, write-back path
- `header/CacheControllers/CacheControllerExclusive.h`,
`src/CacheControllers/CacheControllerExclusive.cpp` – the shipped L1
- `header/CacheControllers/CacheController_End2End.h`,
`src/CacheControllers/CacheController_End2End.cpp` – the shipped LLC
- `header/CacheControllers/CacheControllerDirectory.h`,
`src/CacheControllers/CacheControllerDirectory.cpp` – the directory home
- `src/CacheDataHandler_COTS.cpp` – array, MSHRs, write-back buffer, port arbiter
- `header/FRFCFS_Buffer.h` – the processing queue
- `src/Protocols/` – the handlers (`readEvent`, `handleAction`)
- `header/Interconnect/CommunicationInterface.h` – the message
- `src/Interconnect/BusInterface.cpp`,
`src/Interconnect/TripleBusInterface.cpp` – delivery order

The last-level cache datapath

What you will find here. The shipped LLC drawn as a datapath, and the same drawing with one request's path highlighted four times: a read hit, a write hit, a read miss into a free way, and a miss that evicts. These five figures are the concrete instance of everything Chapter 4 describes, wired up the way the four-core system actually configures it, and they are where the latencies the simulator reports come from.

5.1 The datapath

Figure 5.1 follows the model rather than a textbook cache. The address splits into tag, set and offset, and the set indexes *one* array: in Octopus a line carries its coherence bits and its data together, so there is no separate tag array. That array is read two ways, and the difference matters for every latency the simulator reports. Reading a line's bits — the tag compare, the state the FSM needs — is **untimed** and claims nothing. Reading or writing its **data** goes through a single port, which demand reads, eviction reads and fill or write-back writes contend for through the round-robin arbiter, and which is then busy for A_LLC cycles. A line that currently sits in the MSHR or the write-back buffer is read without waiting for that port, because it is not in the array yet — the dotted bypass — but it still restarts the busy timer, which is why such accesses cut in ahead of one already in progress.

Control is the dashed amber layer: the line state and the decoded event enter the coherence FSM, whose next state is written back into the line and whose actions become port claims, buffer allocations and messages. A miss allocates in the MSHR and leaves on the memory bus; a victim moves to the write-back buffer and frees its way at once; and because the cache is inclusive, the FSM's IssueInv on an eviction invalidates the copy in whichever L1 holds the line — the red path, and the reason a task whose working set fits in its own private cache can still be disturbed by other cores (Chapter 12).

5.2 A read hit

Steps 4–6 — tag compare, FSM, sequencer — take no simulated time. What a hit pays for is the wait in the processing queue (1–3), the wait for the port (7), and then the A_LLC-cycle access itself (8–9). The figure draws the case where the LLC serves the data: the line is valid in the array and no L1 owns it (I or S in MESI_LLC.csv). The third case, a hit on a line an L1 holds in EorM, does not use the port at all: the owner answers the GetS on the bus with Data2Both, and the LLC only saves that copy (S_d → SaveData → S); on a GetM the owner hands the line straight to the requester and the LLC merely rewrites the owner bits.

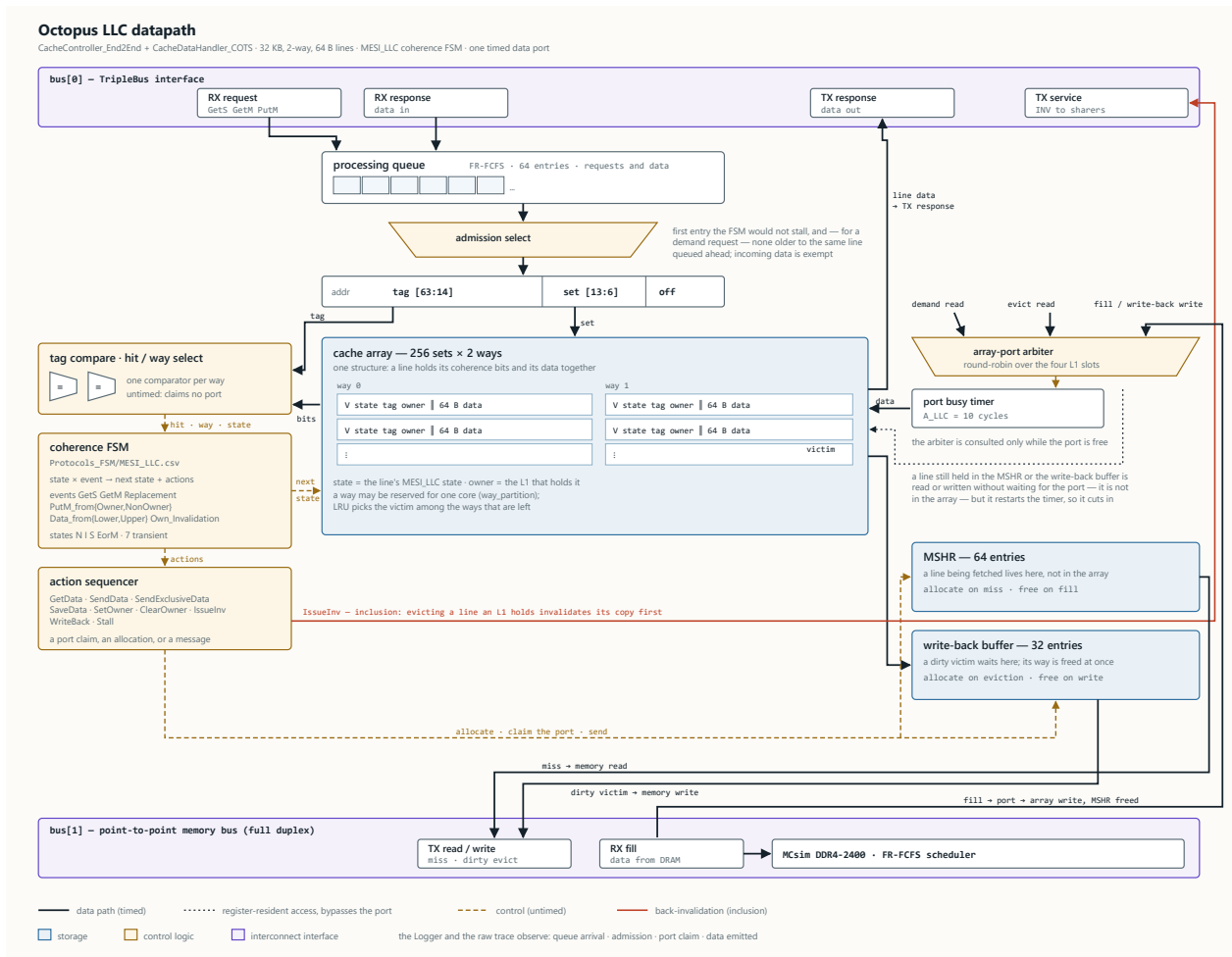


Figure 5.1. The shipped LLC as a datapath: CacheController_End2End with CacheDataHandler_COTS, 32 KB, 2-way, 64 B lines, the MESI_LLC FSM and one timed data port. Bus interfaces on top, control on the left, the single array in the middle read by an untimed bits path and a timed data port on the right, MSHR and write-back buffer beside it, the memory bus below.

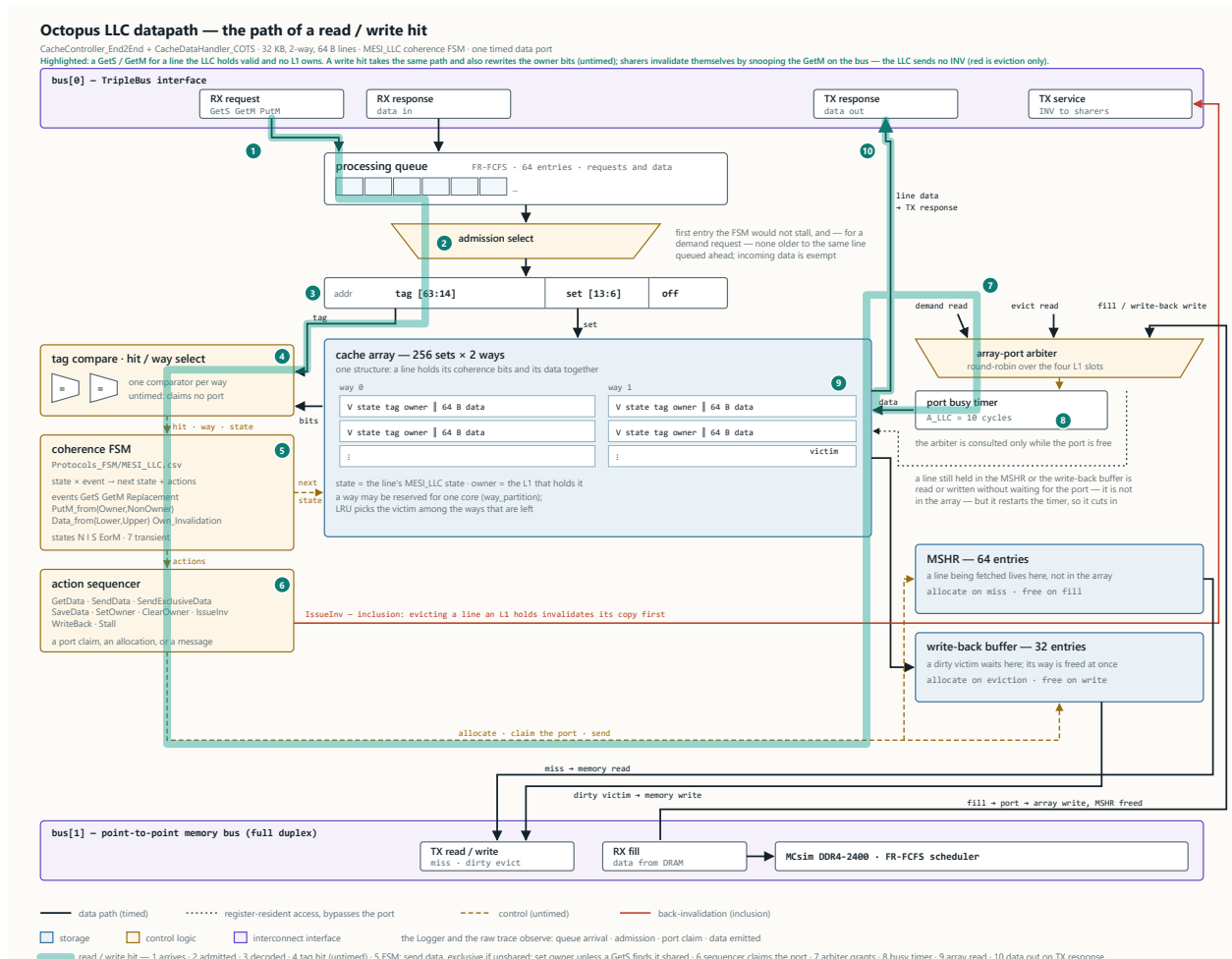


Figure 5.2. A read or write hit — a GetS or GetM that finds the line in a stable state — from the request's arrival on the bus to its data leaving on the response lane, in ten numbered steps.

5.3 A write hit

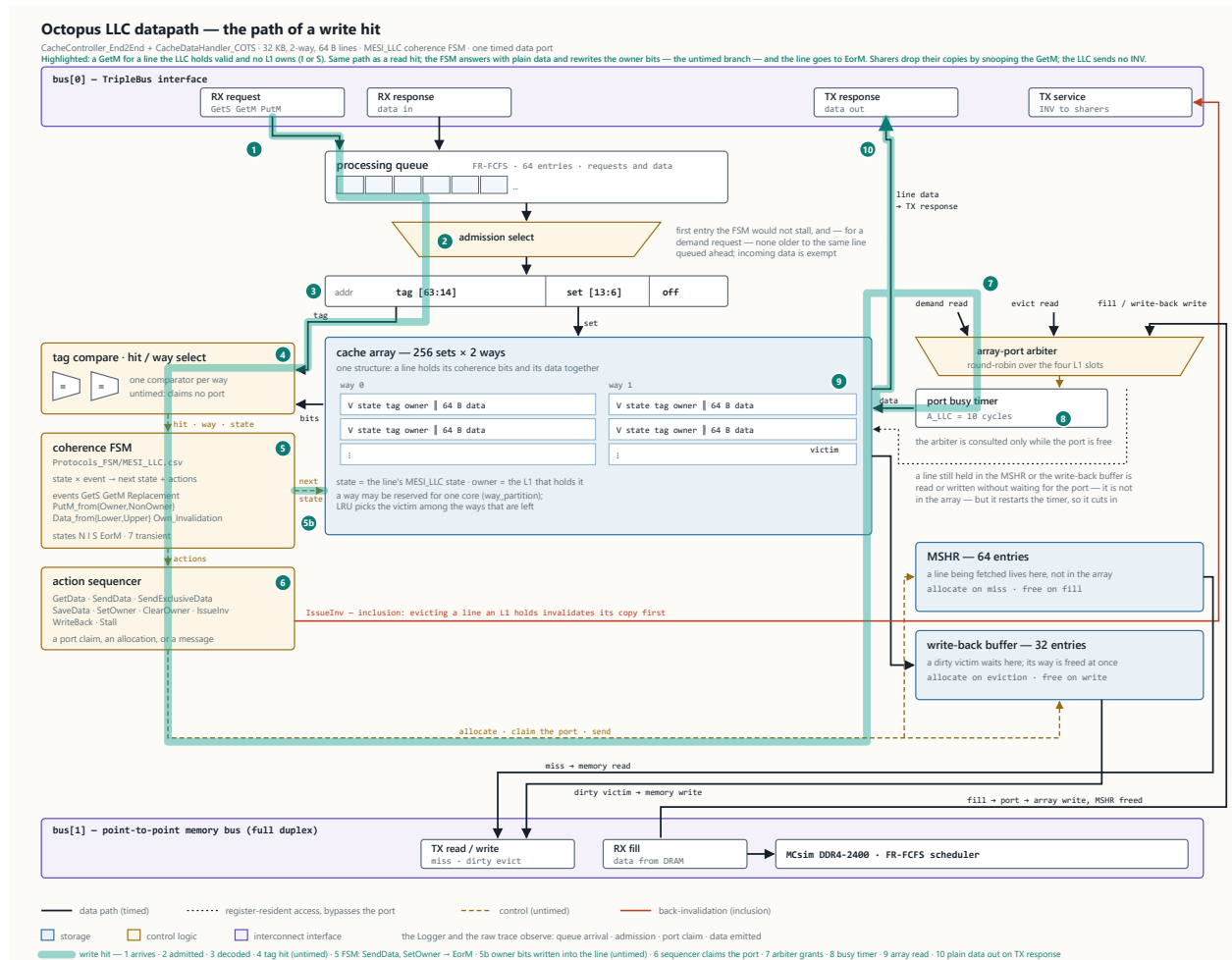
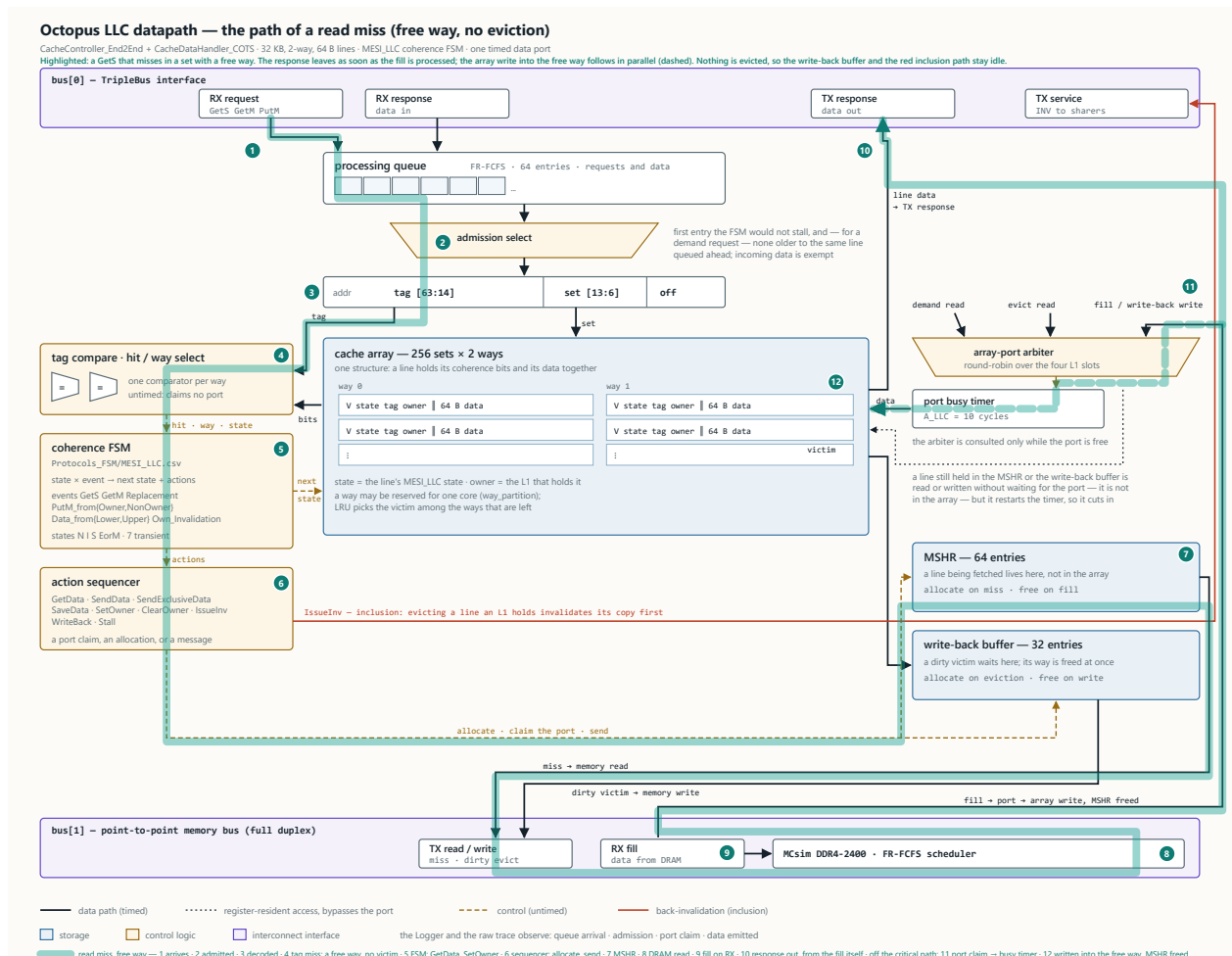


Figure 5.3. The write hit on its own. The path is the read hit's; what is new is the branch marked 5b — the FSM's SetOwner written back into the line's bits over the dashed control wire, untimed — and the plain-data response.

A write hit differs from a read hit in what the FSM does, not in the path: the GetM gets plain data where a GetS gets exclusive data, and the FSM also rewrites the owner bits. Nothing else changes — in particular the LLC sends **no** invalidation: on the snooping bus every sharer sees the GetM itself and drops its copy (S + Other_GetM → I), so IssueInv is reserved for evictions.

5.4 A read miss into a free way

The request takes the same first six steps and then diverges at the tag compare: the FSM's GetData allocates an MSHR entry, the read leaves on bus[1], and the line waits in the MSHR — not the array — until DRAM answers. When the fill is processed the response goes out on TX response at once, carrying the data from the fill message; the write into the free way — which contends for the port like any other array access and frees the MSHR — runs in parallel, off the requester's critical path. Because no victim is chosen, the write-back buffer and the red inclusion path are never touched. A write miss takes the identical route with a GetM in place of the GetS.



5.5 A miss that evicts

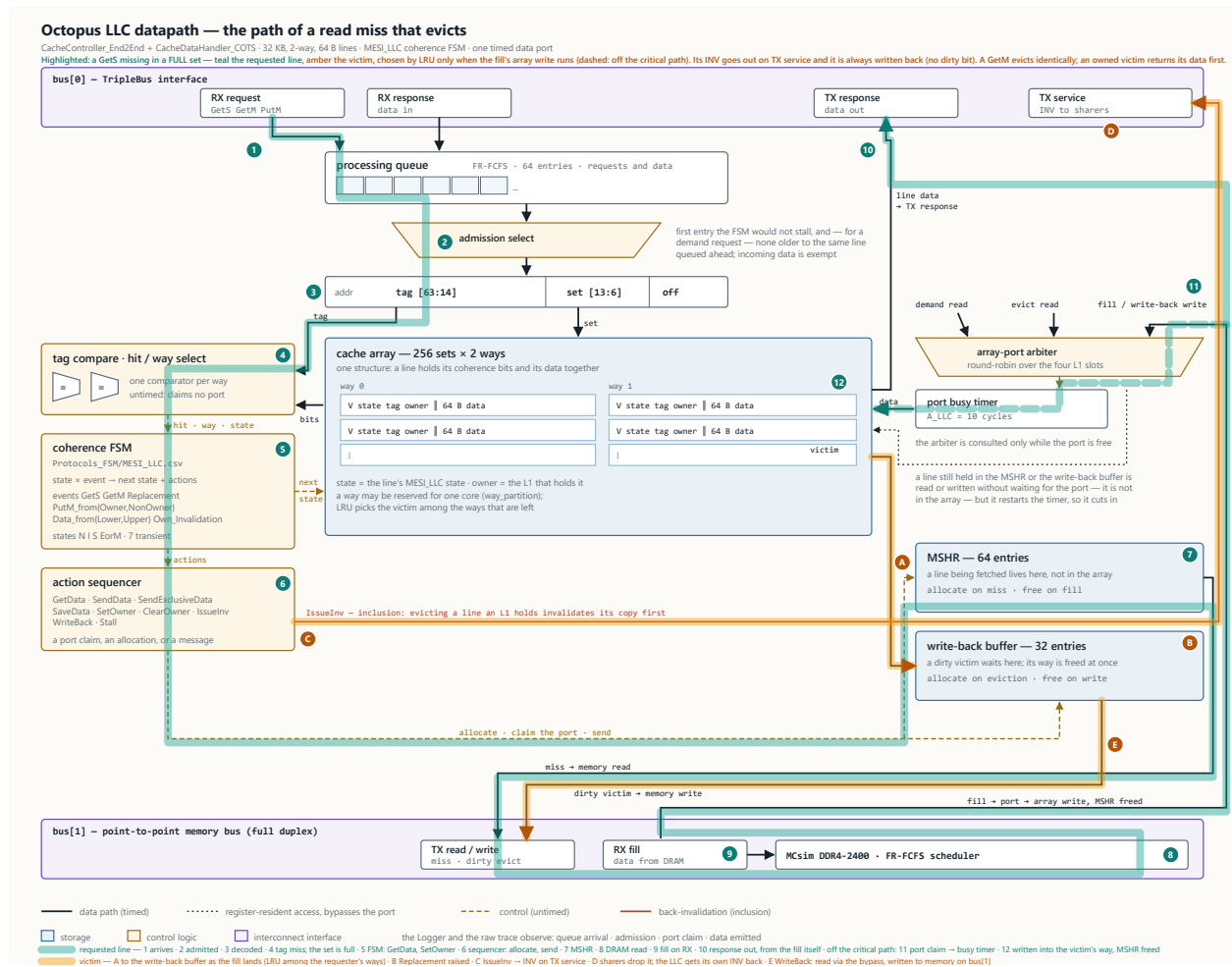


Figure 5.5. A miss into a *full* set: the requested line in teal, the evicted victim in amber. The victim is chosen when the fill is written, and its exit is a coherence transaction of its own.

Two things in Figure 5.5 are easy to get wrong from a textbook. First, **the victim is chosen late**: the miss allocates an MSHR and sends the read while the full set is left alone, and only when the fill's array write runs does the data handler pick the LRU line among the requester's allowed ways, move it — bits and data — into the write-back buffer, and put the fill in its way (moveLine2WB inside updateLineData). The requester's data leaves on TX response right then; everything the victim costs comes afterwards and lands on *other* traffic. Second, **the victim's exit is a coherence transaction**, not a buffer drain: a Replacement is raised for it and goes through the queue and the FSM like any request, IssueInv puts an INV on the service lane, every L1 that shares the line drops it, the LLC receives its own INV back, and only then does WriteBack read the line — from the buffer, over the dotted bypass — and send it to memory. There is no dirty bit in the model, so every victim is written back; the code marks the spot with a *ToDo*. A GetM miss evicts identically. A victim an L1 *owns* adds one leg: the owner answers the INV with its data, and that copy is what reaches memory.

Seen from the cores instead of from inside the LLC, that eviction is the whole story of inter-core interference through an inclusive LLC; Chapter 12 draws it as a sequence and Chapter 7 shows what it costs the victim's next request.

Where in the code

- `src/CacheControllers/CacheController_End2End.cpp` – the LLC controller
- `src/CacheDataHandler_COTS.cpp` – `updateLineData`, `moveLine2WB`, the port arbiter and the bypass
- `Protocols_FSM/MESI_LLC.csv` – the FSM (Chapter B)
- `src/Protocols/LLCMESIProtocol.cpp` – its handler
- `src/CacheControllers/CacheController.cpp` – `sendInvalidationMessage`, `checkReplacements`
- `configuration/SystemConfigurations/MultiCoreSystem.csv` – the `llc_controller.*` keys that size it

The coherence engine: CSV finite-state machines

What you will find here. Why a coherence protocol in Octopus is a spreadsheet and not a program; the exact format of that spreadsheet and the one rule about it that silently breaks protocols; the vocabulary of events a controller can see and actions it can perform; the families that ship and how a snooping protocol differs from a directory one; and a worked example, MI, added without a line of C++.

6.1 Coherence as data

A cache controller's coherence logic is a finite-state machine read at start-up from a CSV file. Rows are states, stable and transient; columns are coherence events; each cell lists the actions and the next state for that (state, event) pair. Adding or modifying a protocol means editing the table — no C++ change and no recompilation. The same engine, `FSMReader` plus a per-family protocol handler, drives MSI, MESI and MOESI in their snooping and directory forms, and the predictable protocols built on Octopus (Chapter 1).

The split pays off in code size. Because the handler only translates a small set of protocol actions into controller actions, a protocol that extends a family it already knows costs little: Table 6.1 contrasts the lines of code per protocol with `gem5's Ruby`.

Table 6.1. Lines of code per coherence protocol, Octopus versus `gem5's Ruby`. In Octopus the count is the CSV table plus the handler code specific to that protocol; MESI and MOESI extend MSI's handler.

	MSI	MESI	MOESI	PENDULUM
<code>gem5 (Ruby)</code>	800	1400	1300	2000
Octopus	300	70	120	500

6.2 The table

A file in `Protocols_FSM/` has four blocks (Figure 6.1):

1. **Events**, numbered from 0: the columns of the transition table. For the snooping L1 they are exactly `Load`, `Store`, `Replacement`, `Own_GetS`, `Own_GetM`, `Own_PutM`, `Other_GetS`, `Other_GetM`, `Other_PutM`, `OwnData`, `Invalidation` and `OwnData_Exclusive`; for the snooping LLC `GetS`, `GetM`, `Replacement`, `PutM_fromOwner`, `PutM_fromNonOwner`, `Data_fromLowerInterface`, `Data_fromUpperInterface` and `Own_Invalidation`.

a row is a state, stable (1) or transient (0); its *position* in this table must equal its StateNum, so a new state's row goes **last**

a column is an event, one of the EventNum list at the top of the file; the handler's readEvent maps each message to one of them

State	stable	GetS	GetM	Replacement	Own_Invalidation
N	1	GetData/SetOwner/NE_d	GetData/SetOwner/NM_d		SaveData/N
NE_d	0	Stall/	Stall/	Stall/	Fault/
I	1	SendExclusiveData/SetOwner/EorM	SendData/SetOwner/EorM	IssueInv/	WriteBack/N
EorM	1	ClearOwner/S_d	SetOwner/	IssueInv/	ClearOwner/MN_d

a cell is actions/next: do SendData, then SetOwner, then go to EorM. No trailing state (IssueInv/) means *stay*; an empty cell *ignores* the event

Fault/ marks a pair that must be unreachable; the run halts and names it, which is how an incomplete table fails loudly

Figure 6.1. An excerpt of MESI_LLC.csv with the four things to know about the format. Chapter B prints every shipped table in full.

2. **Actions**, numbered from 0: what a cell may ask for. The L1 offers Stall, Hit, GetS, GetM, PutM, Data2Req, Data2Both, SaveReq, Fault and removeSavedReq; the LLC offers Stall, GetData, SendData, SaveData, SetOwner, ClearOwner, IssueInv, WriteBack, Fault and SendExclusiveData. Each type of controller offers its list independently of the protocol, which is what lets one controller class run several protocols.
3. **States**, numbered from 0, stable and transient alike. Transient names follow the textbook convention: IS_ad is “going from I to S, awaiting the bus (a) and the data (d)”, IM_dI is “going to M, data pending, but invalidated meanwhile”.
4. **The transition table**: a header row State, stable, isDataValid, <events>, then one row per state. stable marks the stable states; isDataValid says whether the line's data may be read in that state. A cell is Action1/Action2/NextState. A cell whose last element is an action, not a state (IssueInv/), stays in the current state; an empty cell ignores the event; Fault/ declares the pair unreachable.

Row order is the state number

FSMReader indexes transition rows by *position*, not by name. When you add a state, its row must be appended **last**, so that its position matches the number you gave it in the state list. Put it anywhere else and the FSM silently runs another state's transitions — no error, just wrong results. This is the single most likely way a protocol edit goes wrong.

Reading a cell. In Figure 6.1, state I (the LLC has the line, no L1 owns it) on GetM reads SendData/SetOwner/EorM: send the data to the requester, record it as the owner, and move to EorM. The same state on Replacement reads IssueInv/: broadcast an invalidation and stay — the eviction completes later, when the LLC's own copy of that invalidation comes back as Own_Invalidation and the cell WriteBack/N runs. Chapter 5 draws those two paths.

Fault is an audit. A cell of Fault/ means “this (state, event) should be unreachable”. The handler halts the run and reports the pair rather than mis-routing silently. That is what makes a protocol auditable: an incomplete or wrong table fails loudly on the first trace that exercises the hole, and the Debugger's transition trace (Chapter 11) shows the exact sequence that reached it.

6.3 Stable and transient states

A snooping protocol on a split-transaction bus is mostly transient states. A request is issued (GetS), travels the request lane, is seen back by its own issuer (Own_GetS: the line is now ordered on the bus), and some time later the data arrives (OwnData); between those moments other cores' requests to the same line are seen (Other_GetS, Other_GetM) and may have to be remembered (SaveReq) and answered when the data finally comes (Data2Req, Data2Both). Each such interleaving is a row. The shipped MESI L1 has 4 stable and 20 transient states; its LLC 4 stable and 7 transient (Chapter B).

The LLC side uses states a textbook does not name. N is “not present”; EorM is “an L1 owns this line, I do not know in which of E or M”; MN_d is “evicting, waiting for the owner’s data”. They exist because the LLC is inclusive and tracks one owner per line, and they are where most of the subtle behaviour of Chapter 5 lives.

6.4 Snooping and directory

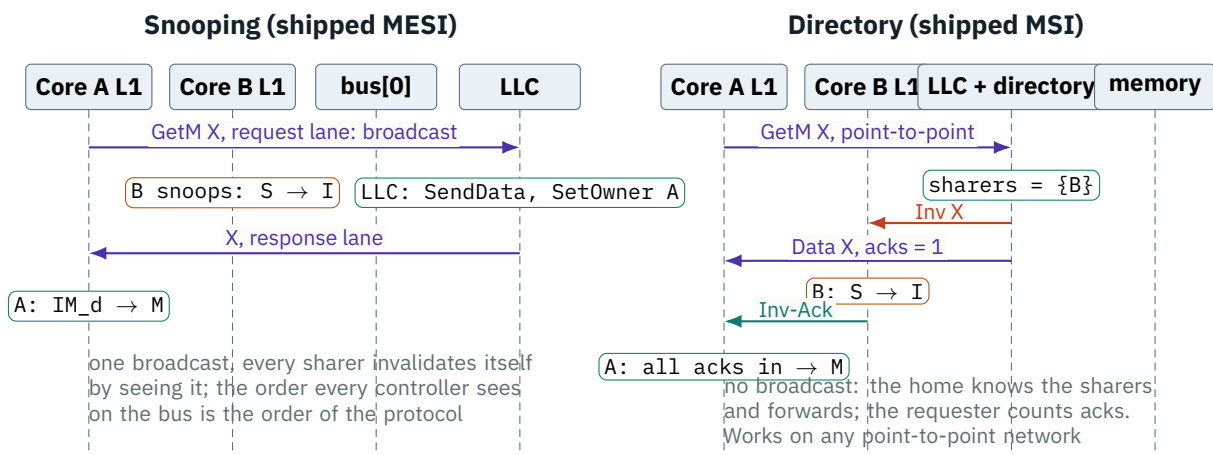


Figure 6.2. One GetM from core A while core B shares the line, under the shipped snooping MESI (left) and the shipped directory MSI (right).

The two families differ in who learns about a request (Figure 6.2). Under snooping every L1 sees every request on the request lane and acts on its own copy; the order of the bus is the order of the protocol, and the LLC’s job is to supply data when no L1 owns the line and to track the owner when one does. Under a directory the home — the LLC-side CacheControllerDirectory — keeps a sharer list per line, forwards each request only to the caches that need to act, and the requester counts the acknowledgements; nothing is broadcast, which is what lets a directory run on a point-to-point network such as the mesh (Chapter 8). The same controllers, queues and data handlers serve both; only the protocol handler and the tables differ.

A protocol is five keys, and the tables are a matched pair

Selecting a protocol sets the controller class, the L1 and LLC protocol_type and the L1 and LLC fsm_filename together (Chapter 10). A MESI L1 with an MSI LLC faults with “Wrong destination”; overriding some of the five from the command line faults or crashes. Switch by preset, not by hand.

Table 6.2. The protocol families that ship, and the preset that selects each. The L1 and LLC tables of a family are a matched pair.

family	L1 table	LLC table
snoop MSI	MSI_splitBus_snooping	MSI_LLC
snoop MESI	MESI_splitBus_snooping	MESI_LLC
snoop MOESI	MOESI_splitBus_snooping	MOESI_LLC
directory MSI	MSI_directory	MSI_LLC_directory
directory MESI, MOESI	MESI_directory, MOESI_directory	. . . _LLC_directory
predictable	PMSI, PMESI, PMSI_asterisk, PMESI_asterisk, Pendulum	their . . . _LLC tables

6.5 A worked example: MI without C++

MI is the simplest coherence protocol: a line is either Invalid or Modified, so every access, even a load, takes it exclusive. It is MSI with the S state and its transients removed and Load behaving like Store: 9 states instead of 22. Because MI uses only a subset of MSI's events and actions, it runs on the existing SNOOP_MSI handler and the existing MSI_LLC: the entire protocol is one CSV, `Protocols_FSM/MI_splitBus_snooping.csv`.

Listing 6.1. The two stable rows of `MI_splitBus_snooping.csv`.

State,	stable,	isDataValid,	Load,	Store,	... ,	OwnData,
Invalidation						
I,	1,	0,	GetM/IM_ad,	GetM/IM_ad,	... ,	Fault/,
M,	1,	1,	Hit/,	Hit/,	... ,	Fault/, Data2Req
/I						

Selecting it is one override on top of the MSI preset:

```
./build/Octopus_Simulator -s MultiCoreSystem \
-p "workload_path(s)=BMs/eembc-traces/mi_pingpong/" \
-p "cache_controller[*].fsm_filename(s)=MI_splitBus_snooping"
```

On the `mi_pingpong` toy workload, where cores 0 and 1 repeatedly read one line, MESI lets the two cores share it and finishes in 332 cycles; MI makes the line ping-pong exclusively between them and finishes in 482. The Logger shows the cost; the Debugger's address-filtered transition trace shows the cause — I -Load-> IM_ad under MI against I -Load-> IS_ad under MESI. One new file, one override, no compiler.

Where in the code

- `Protocols_FSM/` — the tables, one CSV per agent
- `header/FSMReader.h`, `src/FSMReader.cpp` — the parser and `getTransition`
- `src/Protocols/MSIProtocol.cpp`, `src/Protocols/MESIProtocol.cpp`,
`src/Protocols/MOESIProtocol.cpp` — the snooping L1 handlers
- `src/Protocols/LLCMSIProtocol.cpp`, `src/Protocols/LLCMESIProtocol.cpp` —
the snooping LLC handlers (MOESI's LLC runs on the MESI handler)
- `src/Protocols/MSIDirectory.cpp`, `src/Protocols/MESIDirectory.cpp`,
`src/Protocols/MOESIDirectory.cpp` — the directory L1 handlers
- `src/Protocols/LLCMSIDirectory.cpp`,
`src/Protocols/LLCMESIDirectory.cpp`,
`src/Protocols/LLCMOESIDirectory.cpp` — the directory home handlers
- `docs/AddingAProtocol.md` — the MI example in full
- `demo_protocol.sh` — runs it

Chapter 7

A request, end to end

What you will find here. One memory request followed from the core that issues it to the core that retires it, drawn on the system as shipped and on a time axis; the Logger stamp at every hop; and, from those stamps, the exact definition of every column of a `LatencyReport`. This is the chapter to have open the first time you read a report: a total tells you a core was slow, the columns tell you which shared resource made it slow, and this is what each column is.

7.1 The journey

A core issues a Message. Its L1 admits it, looks the line up and, on a miss, puts a GetS (read) or GetM (write) on the request lane of bus [0], where the arbiter elects it. Every other L1 snoops it; the LLC admits it into its queue. If the LLC has the line and no L1 owns it, the data is emitted when the array port is granted; if an L1 owns it, that L1 answers on the response lane instead; if the LLC does not have it, a read leaves on bus [1], DRAM serves it, the fill comes back to the front of the LLC's queue and the response is emitted from it. The response travels the response lane to the requesting L1, which fills and hands the data to the core.

Figures 7.1 and 7.2 are the same drawing with the two routes. Three things on them are worth saying in words. The other cores' L1s see every request on the request lane, and when one of them owns the line it answers on the response lane in the LLC's place, its stamps then standing in for the LLC's. The service lane carries the LLC's back-invalidations on an eviction and is on neither request's path. And a hit's L2 Access ends at the port *grant*: the data leaves then, and the read's A_LLC cycles on the port are paid by whoever waits for the port next — their L2 Access, or a fill's write.

7.2 The stamps

At each hop the component stamps a Logger checkpoint: an ENTER when it takes custody of the request, a SERVICE when it begins the actual work, an EXIT when it hands the request on. Every stamp names the component's *role* (CPU, L1, REQ_BUS, RESP_BUS, LLC, MEM_BUS, DRAM) and is taken in the originating core's clock, so a request's timeline is one ordered list of self-describing events and the columns are differences of *named* stamps, never inferences from position (Chapter 11 has the mechanism). Table 7.1 lists who stamps what.

7.3 The columns

Three things in Table 7.2 are easy to misread from the names alone.

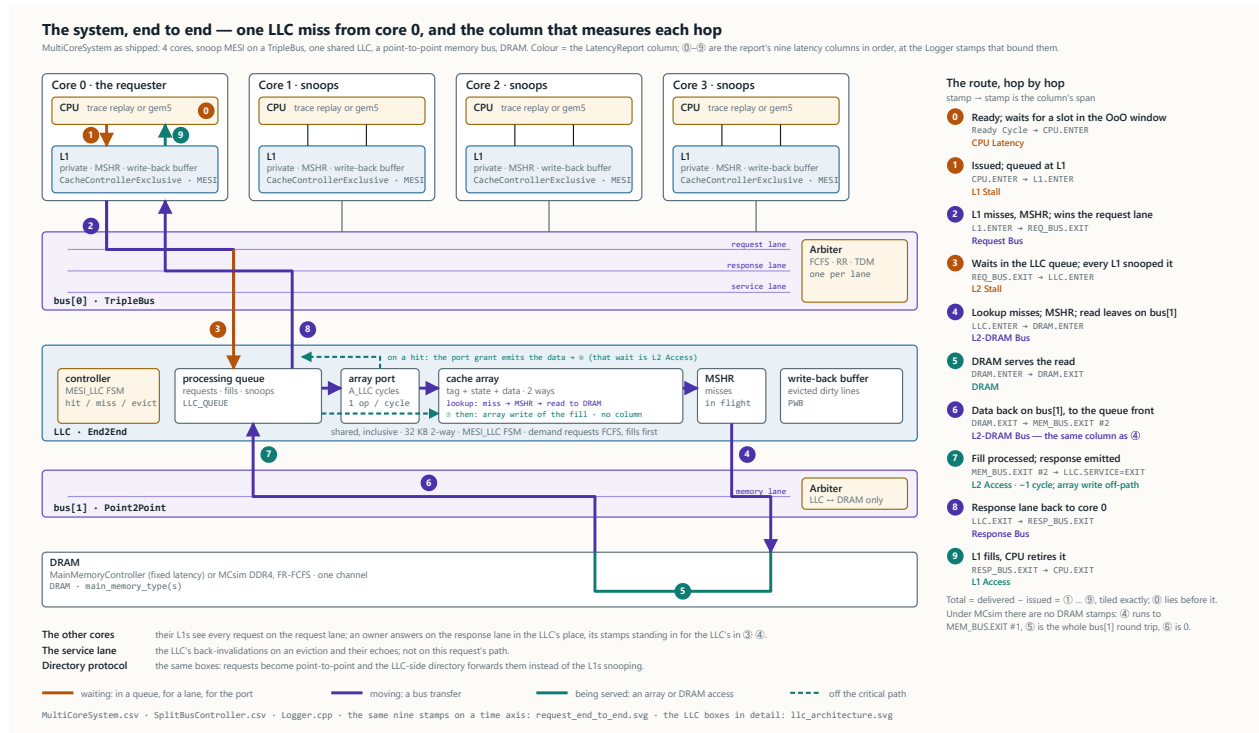


Figure 7.1. The system as shipped with one LLC *miss* from core 0 routed through it. Each hop carries the colour of the LatencyReport column that measures it (amber: waiting; purple: a bus transfer; teal: being served) and the number of the Logger stamp that bounds it, ①–⑩ being the report's nine latency columns in order; the side panel lists the stamp pair behind every column.

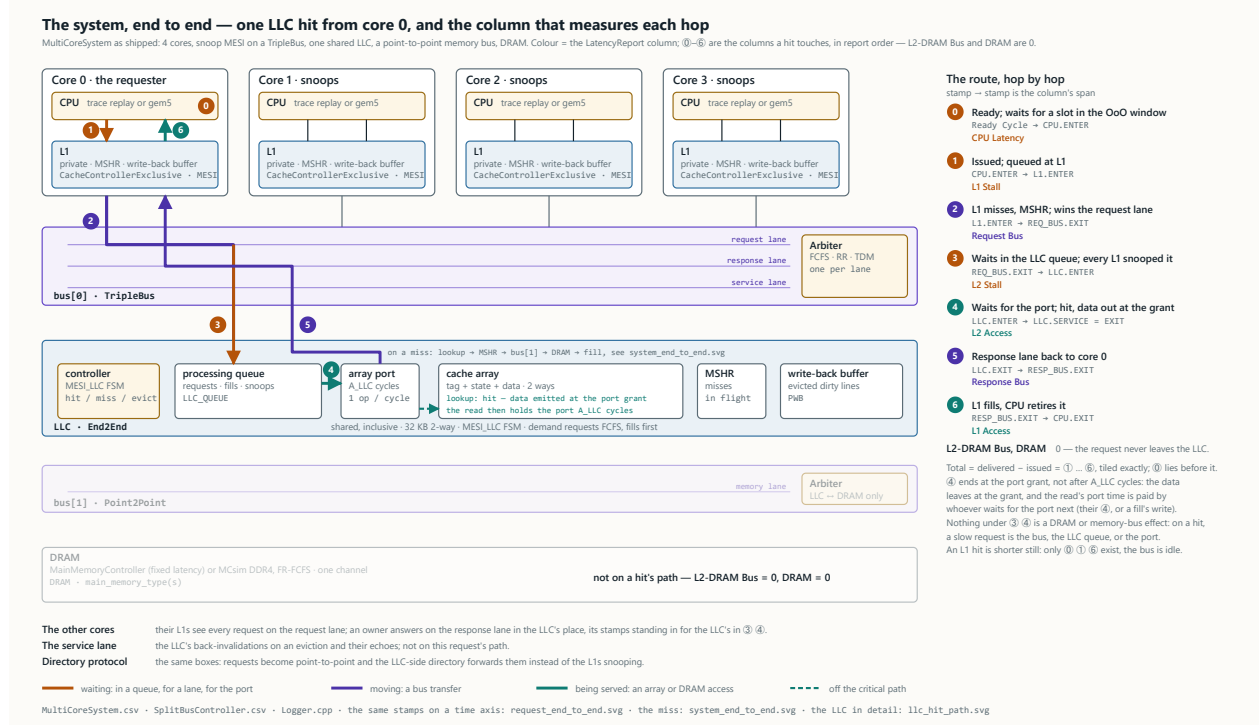


Figure 7.2. The same system with an LLC *hit* from core 0. The route stops at the LLC's array port, where the data is emitted at the grant; the memory bus and DRAM are not on the path, so L2-DRAM Bus and DRAM are 0.

Table 7.1. Where each Logger stamp is taken.

stamp	taken by
CPU.ENTER	CPU::addRequest: the request is issued (and bound to its core)
L1.ENTER	BaseController::processLogic when an L1 admits a request from below
REQ_BUS.EXIT	BusController::broadcast on bus[0]: the request is transmitted
LLC.ENTER	BaseController::processLogic when the LLC admits the bus request
MEM_BUS.EXIT	BusController on bus[1], once in each direction
DRAM.ENTER, DRAM.EXIT	MainMemoryController: request arrives, data leaves after m_memory_latency. MCsim stamps neither
LLC.SERVICE, LLC.EXIT	hitAction / removePendingAndRespond: port granted; response emitted
L1.SERVICE, L1.EXIT (before RESP_BUS.EXIT)	performWriteBack when an <i>owning</i> L1 supplies another core's request; the responder is then that L1, not the LLC
RESP_BUS.EXIT	BusController::send on bus[0]: the response is transmitted
CPU.EXIT	CPU::checkReceiveBuffer: delivered; triggers the report row

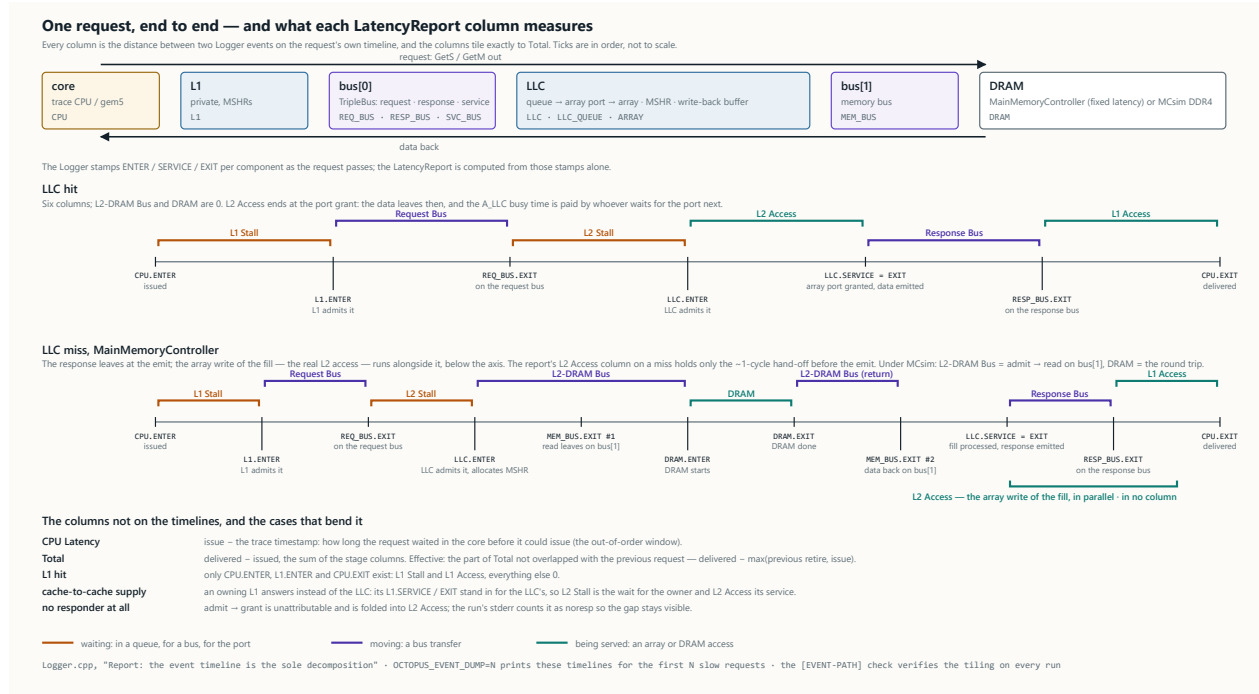
**Figure 7.3.** The same two routes on a time axis: an LLC hit and an LLC miss, with the nine LatencyReport columns bracketed between the Logger stamps that bound them. Ticks are in order, not to scale. Below the miss axis, the array write of the fill runs alongside the response and lands in no column.

Table 7.2. The LatencyReport columns, in report order, as differences of stamps. The nine latency columns tile exactly to Total; a stamp a request never reaches makes its column 0.

column	definition
CPU Latency	$\text{CPU}.\text{ENTER} - \text{Ready Cycle}$: the wait inside the core before the request could issue (the out-of-order window). Lies <i>before</i> Total
L1 Stall	$\text{L1}.\text{ENTER} - \text{CPU}.\text{ENTER}$
Request Bus	$\text{REQ_BUS}.\text{EXIT} - \text{L1}.\text{ENTER}$: the L1's miss handling and its wait for the request lane
L2 Stall	$\text{LLC}.\text{ENTER} - \text{REQ_BUS}.\text{EXIT}$: the wait in the LLC's queue
L2 Access	hit: $\text{LLC}.\text{SERVICE} - \text{LLC}.\text{ENTER}$, the wait for the array port; miss: $\text{LLC}.\text{SERVICE} - \text{MEM_BUS}.\text{EXIT}_2$, the fill's turn at the front of the queue, typically one cycle
Response Bus	$\text{RESP_BUS}.\text{EXIT} - \text{LLC}.\text{EXIT}$: the clean response-lane leg
L2-DRAM Bus	miss: $(\text{DRAM}.\text{ENTER} - \text{LLC}.\text{ENTER}) + (\text{MEM_BUS}.\text{EXIT}_2 - \text{DRAM}.\text{EXIT})$: one column over two hops, out and back
DRAM	miss: $\text{DRAM}.\text{EXIT} - \text{DRAM}.\text{ENTER}$
L1 Access	$\text{CPU}.\text{EXIT} - \max(\text{L1}.\text{ENTER}, \text{RESP_BUS}.\text{EXIT})$: an L1 hit's own service, or the fill and hand-off after the response-lane grant
Total	$\text{CPU}.\text{EXIT} - \text{CPU}.\text{ENTER}$ = the sum of the eight stage columns above (CPU Latency excluded)
Effective	$\text{CPU}.\text{EXIT} - \max(\text{previous request's finish}, \text{CPU}.\text{ENTER})$: the part of Total not overlapped with the previous request; its mean is the per-core average, its sum the true added execution time
Oldest	$\text{CPU}.\text{EXIT}$ – the cycle this request became the oldest outstanding request of its core; 0 if it never was. At OoO= 1 it equals Total

L2 Access means two different waits. On a hit it ends at the array-port grant, not after A_LLC cycles: the data is emitted at the grant, and the port's busy time is paid by whoever waits for the port next. On a miss it is the fill's turn at the LLC: each cycle one message moves from the memory-side RX into the *front* of the processing queue, a data message is always ready, and every ready message is handled that same cycle – so typically a single cycle, plus any backlog in that one FIFO, and never a wait behind queued demand requests. The response is emitted the moment the fill is processed, carrying the data from the fill message; the array write claims the port afterwards, in parallel. That write is the real “L2 access” of a miss, Figure 7.3 draws it below the axis, and it lands in no report column (the ARRAY lane of the visualizer shows it).

L2-DRAM Bus and DRAM depend on the memory model. With MainMemoryController the DRAM stamps exist and L2-DRAM Bus is the two memory-bus legs either side of the DRAM service. MCsim stamps no DRAM events, so there L2-DRAM Bus becomes “LLC admit → read leaves on bus[1]” and DRAM becomes the whole bus[1] round trip, return leg included. Nothing of the round trip is lost, but the split moves.

A cache-to-cache supply substitutes the responder. When the LLC never emits because the line's owner answered on the bus, the last $\text{L1}.\text{SERVICE}/\text{L1}.\text{EXIT}$ strictly before $\text{RESP_BUS}.\text{EXIT}$ become the responder milestones, and the responder's SERVICE replaces $\text{LLC}.\text{ENTER}$ as the admission anchor – the owner snoops the request off the bus and may answer before the LLC admits it, so the LLC's admission is not on the data path. Then L2 Stall is the wait for the owner, L2 Access its service, and Response Bus still $\text{RESP_BUS}.\text{EXIT} - \text{responder EXIT}$. If no responder stamp exists at all, the whole admit-to-grant interval is folded into L2 Access so the columns keep tiling, and the run's stderr counts the request as noresp.

7.4 Checking a column

The columns tile to Total on every row by construction, and the simulator says so: the [EVENT - PATH] line on stderr at the end of a run reports how many requests tiled, how many did not, and how many had no responder. When a column looks wrong, OCTOPUS_EVENT_DUMP=N prints the raw timeline — every stamp with its role and cycle — of the first N requests whose Total exceeds 100 cycles or that crossed the memory bus; OCTOPUS_EVENT_DUMP_MIN=<cycles> raises the threshold to isolate one outlier row seen in a report. The seven *mechanism tracker* columns that follow Oldest in the report (the line’s state when the request arrived at the LLC, whether the per-line gate held it, how many younger responses and array accesses went ahead of it) attribute a stall to its cause; Chapter 11 defines them.

Where in the code

- `src/Logger.cpp` — the stamps, `finalizeEvents` and the “Report: the event timeline is the sole decomposition” block that computes every column
- `header/Logger.h` — the Role and Phase enums
- `docs/Logger.md` — the same material as a reference
- `tutorial/00-setup-and-first-run/README.md` — reading a first report

Interconnects and arbitration

What you will find here. The three-part model every interconnect is built from; the bus family that the shipped systems use, with its lanes, latencies and back-pressure; the three arbiters and the one rule they share; the mesh and the single-switch NoC, with what they do and do not model today; and where each of these is a plug-in.

8.1 Topology, controller, interfaces

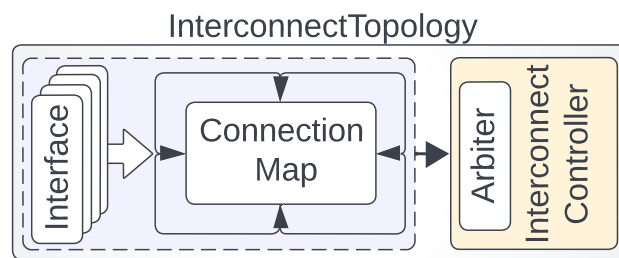


Figure 8.1. An interconnect is a topology that creates a connection map and instantiates one controller and the interfaces; the interfaces are the queues that hold messages at each node; the controller moves messages between interfaces according to the map and the configured latencies.

An interconnect is an `InterconnectTopology` — a set of message-holding `CommunicationInterfaces` plus a *connection map* describing who talks to whom — and an `InterconnectController` whose `Arbiter` resolves contention on shared links (Figure 8.1). Swapping the topology and controller pair yields a point-to-point link, a unified, split or triple bus, a mesh or a NoC; swapping the arbiter yields FCFS, round-robin or TDM. The controllers on either side see only interfaces, so nothing else in the system changes.

Interconnects have their own clock: `m_clk_period` is 50 for every bus and mesh against 100 for the cores, so an interconnect steps twice per core cycle. A transfer occupies a link for `m_request_latency` (2) interconnect ticks for a request and `m_response_latency` (5) for data — the 2- and 5-cycle slots the visualizer draws.

8.2 The bus family

Point-to-point (`Point2PointController`) joins exactly two components, one message per direction per tick, no arbitration. `bus[1]` between the LLC and memory is one.

Unified bus (`UnifiedBusController`) carries requests and data on one shared link under one arbiter.

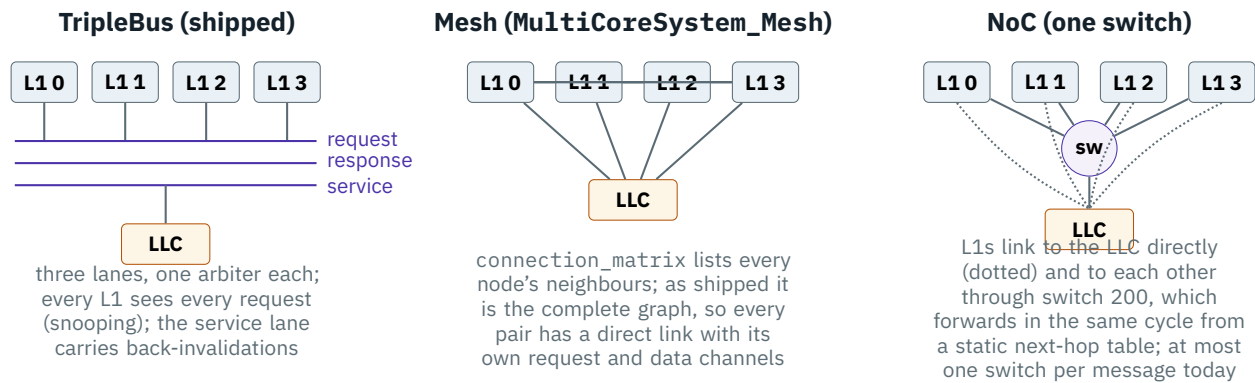


Figure 8.2. The three shapes that ship between the L1s and the LLC.

Split bus (SplitBusController) separates the request lane from the response lane, each with its own arbiter, so a long data transfer does not block the next request.

TripleBus (TripleBus with a Split controller; Figure 8.2 left) adds a third, *service* lane for the LLC's back-invalidations. It is what the shipped snooping systems use as bus[0]: every L1 sees every request on the request lane, data travels the response lane to its destinations only, and an INV on the service lane is broadcast to every interface including the LLC's own.

Back-pressure. Each interface holds `buffers_max_size` messages (256 as shipped). On the TripleBus `response_reserve` (128) keeps that many entries of the response buffers free for in-flight responses: the request lane stalls when the response buffer's free space falls to the reserve, so responses — which must always be able to drain — can never overflow. Delivery order at the receiving side is the subject of Chapter 4.

8.3 Arbiters

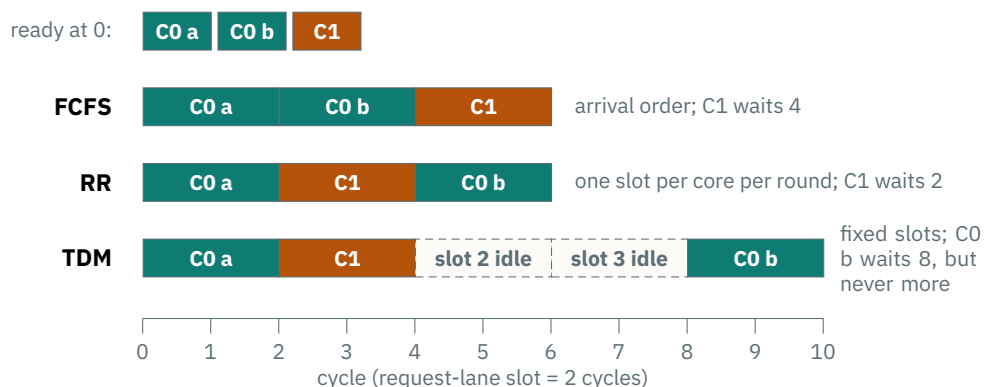


Figure 8.3. Three requests ready at the same cycle on one request lane — two from core 0, one from core 1 — under the three shipped arbiters.

An arbiter is a subclass of `Arbiter` implementing one method:

```
virtual bool elect(uint64_t cycle_number,
                  vector<vector<Message>*> &buffers, // one TX buffer per
                  interface Message *out_msg) = 0;    // the winner, if any
```

It is called once per interconnect tick for each lane that is free, with every interface's TX buffer, and names the message that gets the lane (Figure 8.3):

FCFS (FCFSArbiter) picks the oldest message across all buffers: minimal average wait, no bound on how long a core may be starved by a busier one.

Round-robin (RRArbiter) rotates a pointer over the cores and grants the pointed core's oldest message if it has one, otherwise the next core that does: every core gets at most one slot per round, so a core's wait is bounded by one slot per competitor.

TDM (TDMArbiter) gives each core a fixed slot in a fixed cycle, whether or not it has anything to send; an owner with nothing ready leaves its slot idle. The wait is bounded by the period regardless of what other cores do — the property real-time analyses want — at the cost of throughput. It is the shipped default on bus[0] (SplitBusController.csv).

An arbiter that grants by owner takes the owner's *oldest* message across all buffers (Arbiter::electOldest) not the first buffer in interface order that holds one: a core's cache-to-cache supplies sit in a different buffer from its own requests and would otherwise be passed over round after round. A new arbiter should use the same helper.

The arbiter of a bus is a configuration key, `arbiter_type`, in the controller's CSV or overridden per instance: `-p "bus[0].interconnect_controller.arbiter_type(s)=RRArbiter"`. The LLC's data port has its own arbiter under the same interface, `llc_controller.arbiter_type`. Adding an arbiter is a subclass, a line in `src/Arbiters/CMakeLists.txt` and a name in the controller's factory (Chapter 13).

8.4 Mesh and NoC

MultiCoreSystem_Mesh.csv replaces the bus with a Mesh: one node per entry of `component_ids`, and `connection_matrix[i]` listing node *i*'s neighbours. As shipped the matrix is the complete graph on the four L1s and the LLC (Figure 8.2 middle), so every pair has a direct link; each link carries requests and data on separate channels with their own latencies (2 and 5 ticks) and its own arbiter (FCFS by default), half-duplex, one message in flight per latency window. It is the natural home of the directory protocol, which sends point-to-point and never broadcasts; directory MSI runs the whole EEMBC suite on it.

The NoC is the mesh plus *switch* nodes (`switch_ids`, 200 as shipped). A switch is an interface that forwards whatever it receives in the same cycle, with no latency or arbitration of its own, from a static per-source next-hop table built at construction: every direct neighbour maps to itself, and everything a neighbouring switch lists maps to that switch. As shipped (Figure 8.2 right) the L1s link to the LLC directly and to each other through the switch. A message can cross at most one switch, because a switch's own table knows only its direct neighbours.

What the NoC does not model yet

Three things are worth knowing before measuring on it. A destination the routing table does not know is dropped silently and `pushMessage` returns true; the TX-side capacity check compares the *number of per-link queues*, a constant, against the limit, so there is no back-pressure on that side and congestion exits instead of stalling; and a hop is logged as one opaque bus event, so the report's bus columns and the visualizer cannot see the path. A multi-hop network with pluggable routing, per-hop credits and virtual networks per message class is designed in `docs/NoC_MultiHop_Plan.md`; the current single-switch NoC becomes its degenerate case.

Where in the code

- header/Interconnect/, src/Interconnect/ – Bus, TripleBus and their interfaces
- src/Interconnect/BusController.cpp, src/Interconnect/SplitBusController.cpp, src/Interconnect/TripleBusController.cpp, src/Interconnect/UnifiedBusController.cpp, src/Interconnect/Point2PointController.cpp – the bus controllers
- src/Interconnect/Mesh.cpp, src/Interconnect/MeshController.cpp, src/Interconnect/NoC.cpp, src/Interconnect/NoCController.cpp – the mesh and the NoC
- header/Arbiters/, src/Arbiters/ – Arbiter, FCFSArbiter, RRArbiter, TDMArbiter
- configuration/Interconnect/ – latencies, buffer depths, arbiter names, the mesh and NoC connection matrices (Chapter A)

Main memory

What you will find here. The two memory models behind `bus[1]`: a fixed-latency controller for fast exploration and MCsim's cycle-accurate DDR4 controllers for fidelity; how each is selected, clocked and stamped; the schedulers MCsim brings; and the two things that went wrong at this boundary and are worth knowing.

9.1 Two models, one interface

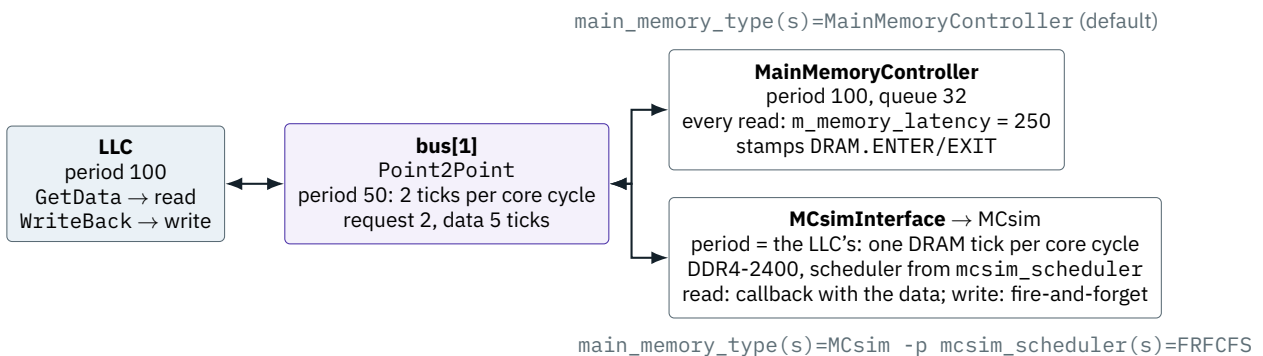


Figure 9.1. The LLC, the memory bus and the two memory models with their clocks.

The LLC talks to memory through `bus[1]`, a point-to-point interconnect: a `GetData` becomes a read, a `WriteBack` a write, and the fill comes back the same way (Figure 9.1). Whatever answers on the far side is a `CommunicationInterface` consumer, so the LLC does not know which model it is talking to; `main_memory_type` selects it.

MainMemoryController (the default) is a fixed-latency model: every read is served `m_memory_latency` cycles after it is accepted (250 as shipped), from a queue of `processing_queue_size` entries (32), at the cores' clock period. It stamps `DRAM.ENTER` on acceptance and `DRAM.EXIT` when the data leaves, so the report's DRAM column is exactly the service and L2-DRAM Bus the two bus legs (Chapter 7). It is the right model for design-space exploration where memory is not the variable, and it is what the sweeps use as the baseline.

MCsimInterface wraps MCsim, the extensible DRAM memory-controller simulator vendored under `src/MCsim/`: a DDR4-2400 device with configurable channels, ranks, bank groups and banks, per-requestor queues, and one scheduler chosen by `mcsim_scheduler` from the directories under `src/MCsim/system/`: FCFS, FR-FCFS (and its batched and capped

variants), BLISS, PAR-BS, MEDUSA, ORP, RTMem, ROC, RankReOrder, ReOrder, Round, PipeCAS, and the predictable controllers AMC, DCmc, MAG, MCMC, PMC and RROF. Each is a `system.ini` that names the request-queue structure, the command generator and the command scheduler; a new controller is on average about 130 lines. MCsim stamps no DRAM events: the DRAM column becomes the whole bus [1] round trip and L2-DRAM Bus the admit-to-leave hand-off. Writes are fire-and-forget. Selecting it:

```
-p "main_memory_type(s)=MCsim" -p "mcsim_scheduler(s)=FRFCFS"
```

MCsim is cycle-accurate and correspondingly slower — five to ten times on EEMBC, and prohibitive on the SPLASH giants, which is why the memory axis of the sweeps is EEMBC-only. Use FRFCFS unless the scheduler is the subject: the Round and ORP schedulers wedge on the shipped workloads.

9.2 Clocks at the boundary

The memory models are ClockedObjs like everything else. MainMemoryController ticks at the cores' period. MCsimInterface is constructed with the *LLC*'s period and must be: MCsim advances its DRAM clock once per tick of the interface, so one tick per core cycle is what makes a DRAM cycle a core cycle.

The simulator is deterministic and its reports are byte-identical on Linux and Windows, MCsim included; the environment check's reference run (Chapter 2) is an MCsim run for that reason. MCSIM_PROBE logs every request, command and completion with MCsim's own clock when the DRAM model itself has to be examined (Chapter 11).

CRLF in the MCsim configuration

MCsim's `.ini` parser rejects CRLF line endings with *Malformed Line N (missing equals)*. A tree checked out on Windows and copied to Linux has them; the environment check tests for it.

Where in the code

- `src/MainMemoryController.cpp`, `configuration/MainMemoryController.csv` — the fixed-latency model
- `header/MCsimInterface.h`, `src/MCsimInterface.cpp` — the bridge to MCsim
- `src/MCsim/` — MCsim itself
- `src/MCsim/README.md` — writing a memory controller
- `src/MCsim/system/<Name>/<Name>.ini` — one per scheduler
- `src/SystemConfigurations/MultiCoreSystem.cpp` — where `main_memory_type` is read

Configuration

What you will find here. How a parameter travels from the command line to the deepest component; the file hierarchy and its Extends rule; the typed keys that let -p set anything without a schema; what is data and what is code, and why -s selects both; the presets and the one axis that is not a single override; and the sweep harness that turns all of this into reproducible experiments.

10.1 One model for every component

Every component derives from Configurable: it reads its default parameters from its own CSV, a parent may override a child's defaults, and the command line overrides everything. A value can therefore be inherited, overridden or extended at any level, and the whole design space is reachable without recompiling (Figure 10.1). Two consequences follow. A designer sweeps the design space from the CLI without editing a file per experiment, which is what the sweep harness and the reproducibility scripts rely on. And defaults let a user focus only on the parameters they care about, trusting that the rest are configured properly.

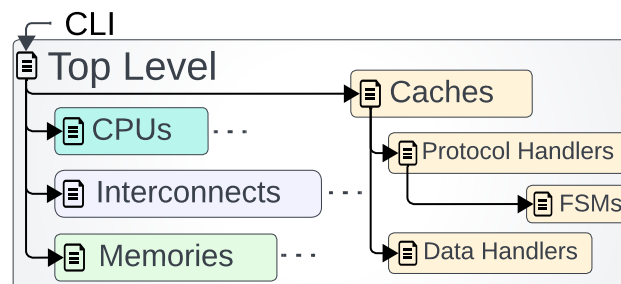


Figure 10.1. Hierarchical propagation of parameters: from the command line through the top-level system module down to each sub-component, each level able to override what it passes on.

10.2 Files and inheritance

Each simulation module is defined by three files: a C++ source, a header, and a CSV holding the default values of its parameters. A module derived from another inherits the parent's defaults through an Extends row and overrides or adds in its own file; a module made of sub-modules may override their defaults under a prefix. The chain for the shipped L1 is:

configuration/CacheControllers/BaseController.csv	defaults every
controller shares	
configuration/CacheControllers/CacheController.csv	Extends,
BaseController + overrides	

```
configuration/CacheControllers/CacheControllerExclusive.csv Extends,
    CacheController
configuration/SystemConfigurations/MultiCoreSystem.csv      the system:
    instances + per-instance overrides
```

A system CSV overrides per instance: `cache_controller[*].num_mshr(i),16` applies to every L1, `cache_controller[2].num_mshr(i),4` to one. Sub-components use dotted prefixes: `llc_controller.m_data_handler.cache_size(i)` reaches the LLC's data handler, `bus[0].interconnect_controller.arbiter_type(s)` the first bus's controller, and `cache_controller[*].dprint.enable(i)` every L1's debugger. The resolution order is the CSV hierarchy first, then the system file, then the command line.

10.3 Typed keys

The suffix of a key is the type of its value: (i) an integer, (s) a string, (vi) a list of integers, (vs) a list of strings. Values follow in the next cells; a cell beginning with # is a comment. The type travels with the key, which is what lets -p set *any* key without a schema:

```
-p "cache_controller[*].num_mshr(i)=4"
-p "bus_type(vs)=TripleBus,Bus"
-p "llc_controller.m_data_handler.way_partition(s)=0:0;1-3:1"
```

Chapter A lists every key that ships, file by file, with its type, default and comment.

10.4 Data and code

`MultiCoreSystem.csv` parameterises `src/SystemConfigurations/MultiCoreSystem.cpp`. That C++ decides what objects exist and how they are wired: a loop creating one L1 per core on `bus[0]`, one LLC between `bus[0]` and `bus[1]`, memory beyond. Every other choice — protocol, arbiter, geometry, queue depths, memory model, way partition, debugger scope — is data. So -s <name> selects *both* a C++ class and its CSV, by the same name; a `MyConfig.csv` cannot be selected with -s `MyConfig` unless a `MyConfig` class exists. That boundary — data below, wiring in C++ — is what the three-level-cache exercise crosses (Chapter 13).

The command line is parsed by `CLParser`: -s <system> names the system, -p "<key>=<value>" overrides one key and may be repeated; Chapter C has the full reference.

10.5 Presets, and the protocol axis

`configuration/SystemConfigurations/` also holds presets that are never selected directly: `MultiCoreSystem_Snoop.csv` (snoop MSI), `MultiCoreSystem_Directory.csv` (directory MSI, pinned to an FCFS bus), `MultiCoreSystem_Mesh.csv` (the mesh). `run_octopus.sh` -protocol *copies* the chosen one over `MultiCoreSystem.csv`, which is why `git checkout` - configuration/ is the reset for everything in this chapter.

The protocol is five coupled keys, not one

A protocol is `cache_controller_type`, `cache_controller[*].protocol_type`, `cache_controller[*].fsm_filename`, `llc_controller.protocol_type` and `llc_controller.fsm_filename` together. Every other axis in Table 10.1 is a single -p; this one is not. Overriding fewer than five — the two `fsm_filename` keys, say, which is the obvious thing to try — faults or crashes even for MESI. Switch protocol by selecting a preset.

Table 10.1. The configuration axes that ship, with the key that moves each.

axis	key	values
bus arbiter	<code>bus[0].interconnect_controller.arbiter_type</code>	FCFSArbiter RRArbiter TDMArbiter
LLC port arbiter	<code>llc_controller.arbiter_type</code>	the same
main memory	<code>main_memory_type</code>	MainMemoryController MCsim
DRAM scheduler	<code>mcsim_scheduler</code>	FRFCFS FCFS BLISS AMC MAG ...
LLC geometry	<code>llc_controller.m_data_handler.cache_size</code> , <code>m_ways_count</code>	ways, ways
LLC way partition	<code>llc_controller.m_data_handler.way_partition</code>	1-100; 1-3:1
queues	<code>*.num_mshr*.pwb_size</code> <code>*.processing_queue_size</code>	integers, -1 = unbounded
out-of-order window	<code>cpu[*].m_number_of_OoO_requests</code>	integer
coherence protocol	a whole preset	snoop MSI, snoop MESI, direc- tory MSI

10.6 The sweep harness

`sweeps/` holds one script per axis — `sweep_arbiter.sh`, `sweep_replacement.sh`, `sweep_cache.sh`, `sweep_memory.sh`, and the root `sweep_protocols.sh` for the coherence families — each varying one axis against a fixed baseline (snoop MESI, TripleBus, LRU, FCFS bus, fixed-latency memory) and recording, per (value, benchmark), whether the run completed plus the latency metrics parsed from `Summary.csv`. Results land in `results/<axis>/<suite>.csv` with the columns `value`, `benchmark`, `status`, `avg`, `wcTotal`, `wcEff`, `wcL1stall`, `wcReqBus`, `wcL2stall`, `wcL2access`, `wcRespBus`, `wcDramBus`, `wcDRAM`, `wcL1access`, `finish`, `wcOldest`; `sweeps/plot_results.py` draws them. Environment knobs select the suite (`SUITE=eembc|splash`), the fan-out (`JOBS`), a single benchmark (`BENCH`) and a wall-clock cap (`SAFETY`); `run_overnight.sh` runs the whole matrix across both suites.

Two facts shape how the harness runs. Octopus reads its entire configuration once, at process start-up, from a fixed path baked at compile time; so runs with *different* configurations cannot share the tree and axis values are serial, while benchmarks within one value run in parallel. And the arbiter lives in the interconnect’s Extends file, not the system CSV, so `sweep_arbiter.sh` edits and restores `SplitBusController.csv`. Every edit is verified before a run starts, because a simulator that is still running keeps the file open on Windows and a silent failed edit would run the wrong configuration without any error.

Where in the code

- `header/Configurable.h`, `src/Configurable.cpp` — the parameter model
- `src/CLParser.cpp` — the command line
- `configuration/` — every CSV (Chapter A)
- `run_octopus.sh` — preset selection
- `sweeps/sweep_common.sh` — the harness
- `REPRODUCIBILITY.md`, `sweeps/README.md` — the same material online

Monitoring: Logger, Debugger, trace and visualizer

What you will find here. The four ways to see inside a run. The Logger: automatic, global, fixed-format, one row per request with its latency split by stage and the per-core worst cases. The Debugger: opt-in, per-component, filterable, free-format, the tool for one address or one coherence bug. The raw event trace: every message of every kind, binary, for the resource lanes and the per-line transition table. And the visualizer that draws all three.

11.1 The Logger

The Logger performs *latency logging*: it silently tracks every request as it traverses the hierarchy and, when the request completes, decomposes its end-to-end latency into the stages it passed through. It is passive — it only reads clocks and records events — so it never changes any timing.

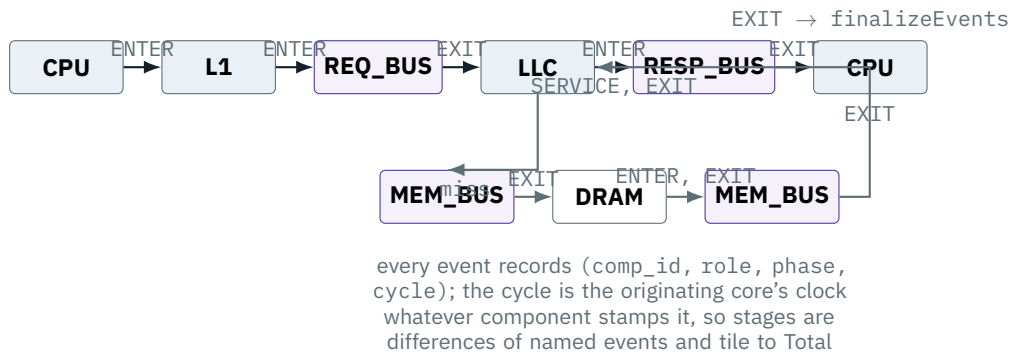


Figure 11.1. The Logger's view of one request: a chain of self-describing events.

Self-describing events. Each in-flight request, keyed by msg_id, owns a time-ordered list of events, and every event records *which component* stamped it and *which phase* (Figure 11.1):

```

enum class Role : uint8_t { CPU, L1, REQ_BUS, RESP_BUS, LLC, MEM_BUS, DRAM };
enum class Phase : uint8_t { ENTER, SERVICE, EXIT };
struct LogEvent { uint32_t comp_id; Role role; Phase phase; uint64_t cycle; };

```

The per-stage decomposition is a generic walk over that list, never an inference from a checkpoint's position. A previous design multiplexed several resources into two shared vectors and

recovered “which is which” by index; it mislabelled stages whenever a stamp was conditional or an extra event appeared. Named events remove the ambiguity, and adding a level or an interconnect is a matter of stamping its own ENTER/SERVICE/EXIT.

The clock-domain trick. Every cycle `CPU::cycleProcess` tells the Logger that core’s current cycle; `addRequest` binds a request to its core; every later event records the *originating core’s* clock, whichever component stamps it. All stages of a request are therefore measured in one clock and no per-component bookkeeping is needed. Chapter 7 lists where each stamp is taken and defines every column from them.

Reports. At the end of a run the Logger writes, per core, `LatencyReport_C<n>.csv` — one row per completed request, then a worst-case footer and an averages footer — and one cross-core `Summary.csv` with every stage’s worst case, the average, the finish cycle and the worst head-of-queue latency per core. A periodic task (Chapter 12) adds `JobReport_C<n>.csv`. Chapter D has the columns.

Effective latency. Total double-counts when requests overlap under an out-of-order core. Effective credits only the part of a request that does not overlap the previous request’s completion, so summing it recovers the true added execution time; the per-core *Average Latency* is its mean.

Mechanism trackers. Seven columns after Oldest attribute a stall to its cause (Table 11.1). They are what `sweeps/analyze_mechanisms.sh` aggregates, and what the visualizer’s class and “ahead” filters read.

Table 11.1. The mechanism tracker columns of a `LatencyReport` row.

column	meaning
LLC Arrival State	the line’s MESI_LLC state id when the request arrived at the LLC: <code>-2</code> never reached the LLC (an L1 hit), <code>-1</code> absent, stable (<code>I=3</code> , <code>S=4</code> , <code>EorM=5</code>) admitted at once, transient (<code>IorS_a=6</code> , <code>MN_d=7</code> , <code>S_d=8</code> , <code>I_d=9</code> , <code>N_a=10</code>) a coherence round trip, <code>NE_d=1</code> / <code>NM_d=2</code> a DRAM fetch already in flight: a <i>coalesced</i> hit if its own DRAM is 0
LLC Gate	1 if an older demand request to the same line was already queued at the LLC (the per-line FCFS gate held it)
Resp Ahead	younger own-core, bus-served responses granted on the response lane between this request’s emit and its grant
Resp Ahead Refills	how many of those were DRAM misses, refills that returned faster than this request’s LLC path
Array Ahead	accesses served at the LLC array port between this request’s park and its grant; 0 if it never parked
Array Ahead Writes	how many of those were data writes: L1 write-backs, snooped supplies, fills
LLC Stall State	the line’s state at the <i>first</i> cycle the LLC FSM judged the request not ready; <code>-2</code> = never stalled by the FSM, so an L2 Stall <code>> 1</code> with <code>-2</code> came from the per-line gate

Switches. `OCTOPUS_NO_LOG=1` skips per-request logging but still writes `Summary.csv`, for correctness sweeps where only termination matters. `OCTOPUS_EVENT_DEBUG=1` prints any tiling violation and the end-of-run tally. `OCTOPUS_EVENT_DUMP=N` and `OCTOPUS_EVENT_DUMP_MIN` print raw timelines (Chapter 7).

11.2 The Debugger

DebugPrint is the second facility: opt-in, per-component, filterable, free-format. It answers “what exactly happened to this message, or at this component?” – the tool for tracing a coherence bug, watching one address, or logging a rare event.

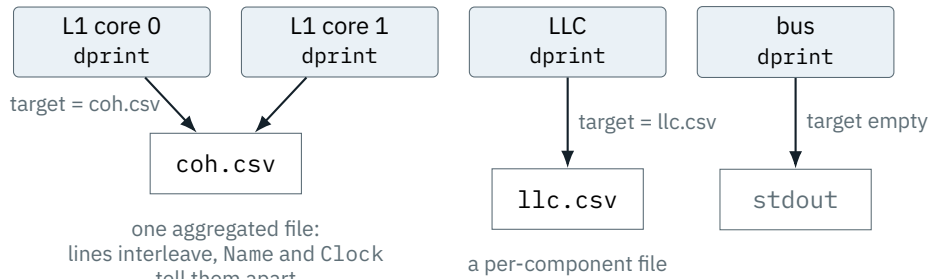


Figure 11.2. Every component owns a DebugPrint; instances that share a target interleave into one file, distinct targets give per-component files, an empty target prints to stdout.

Instances and output. Every cache controller builds one in BaseController under the dprint prefix, tagged with its source_name (CacheController2). A static map keys open files by name, so several components pointed at the same target produce one aggregated report whose Name and Clock columns tell the lines apart (Figure 11.2).

Three kinds of print. A *general event print*, `print(NULL, "queue FULL")`, fires on any event. A *message-associated print*, `print(&msg, "...")`, emits a configurable preamble of identity columns (`print_name`, `print_clk`, `print_msg_id`, `print_addr`, `print_from`, `print_to`, `print_owner`) and is subject to the filters. And the *transition hook*, `transition(&msg, id, old, event, next, stalled, fsm)`, which every protocol calls right after `getTransition`, prints the readable I -Store-> IM_ad (src, cv, data) with the FSM’s own names – the snoop L1, the snoop LLC and the directory protocols all read the same way, and a stall shows as old -event-> old [stall]. Keep new protocols on this hook: it is the single place transitions are reported, and it is also what feeds the raw trace.

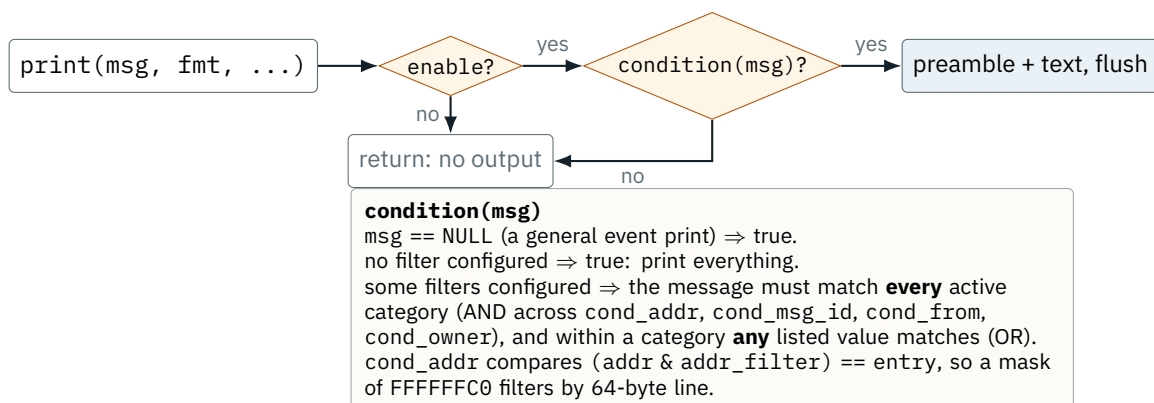


Figure 11.3. What print does, and the rule condition applies.

Filters. Four categories, each a list in the configuration: `cond_addr` (compared as (`addr & addr_filter`) == entry, so the hex mask FFFFFFFC0 selects a 64-byte line), `cond_msg_id`, `cond_from`, `cond_owner`. With no filter set everything prints; with any set, a message must

match every active category and any value within one (Figure 11.3). Note the units: `addr_filter` is hex, but the `cond_addr` entries and the `Addr` column are decimal.

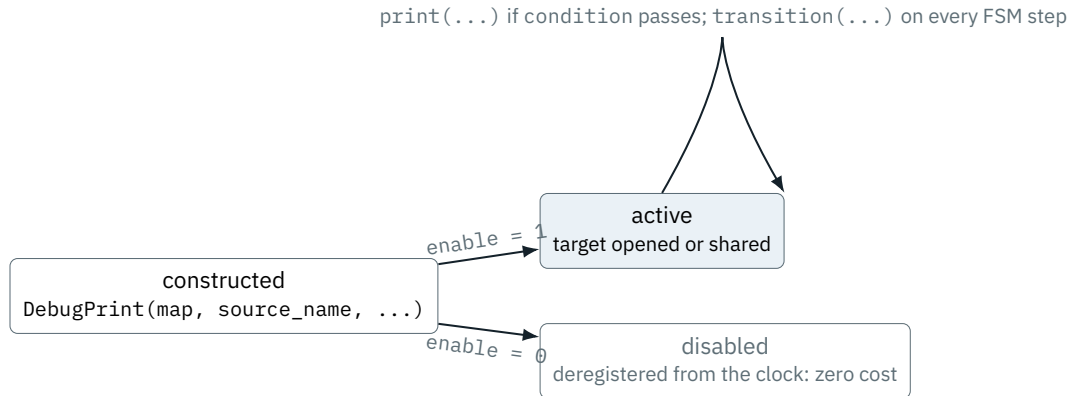


Figure 11.4. A `DebugPrint`'s life. A disabled instance deregisters from the clock in its constructor, so observability costs nothing when off.

Zero cost when off. A `DebugPrint` with `enable=0` removes itself from the clock loop in its constructor (Figure 11.4), so instances can be left in place throughout the code at no cost. Because every setting is an ordinary configuration key, enabling and scoping a trace needs no recompilation:

```
-p "cache_controller[*].dprint.enable(i)=1" -p "llc_controller.dprint.enable(i)=1" \
-p "cache_controller[*].dprint.target(s)=coh.csv" -p "llc_controller.dprint.target(s)=coh.csv" \
-p "cache_controller[*].dprint.addr_filter(s)=FFFFFFC0" -p "cache_controller[*].dprint.cond_addr(vs)=36749952"
```

11.3 The raw event trace

`OCTOPUS_TRACE=<file>` appends every resource event of *every* message — CPU requests and the non-request traffic the per-request report cannot see: write-backs, back-invalidations, cache-to-cache supplies, fills, LLC evictions — to a binary file at simulation speed, with no measurable slowdown. The `LatencyReport` stays the authoritative latency decomposition; the trace is what the visualizer's resource lanes and the per-line transition table are built from. It is zero-cost when the variable is unset.

A record is 32 bytes: the global core cycle, `msg_id`, address, the resource (the `Logger's Role`, extended with `SVC_BUS`, `ARRAY`, `LLC_QUEUE` and `FSM`), the phase, the message's kind, its owner core, the component id and a flags word. On an `ARRAY` record the flags carry the action that claimed the port and whether the claim *cut in* while the port was busy (a fill or a write-back of a line resident in the `MSHR` or write-back buffer may; Chapter 4); on an `FSM` record they carry the old and new state ids, and the phase is the event id, so the trace holds every coherence transition of every controller. Chapter D has the field table.

Giant runs. `OCTOPUS_TRACE_WINDOW=t0:t1` keeps only the events inside a cycle window, so the region around a worst case found in a report can be traced on a billion-cycle run. Conversion is streamed in 64 K-record chunks with bounded memory into Parquet parts the viewer reads as one table.

Time base. Bus events are stamped one cycle earlier in the LatencyReport than on the global clock the trace uses, because the buses tick before the CPUs within a timestamp; the occupancy builder shifts the bus lanes by -1 so they line up with the rows. Totals are unaffected.

11.4 Triage: when a run wedges

OCTOPUS_HANG_DUMP=<cycles> makes every controller that has processed nothing for that many cycles while its queue is non-empty dump, once, to stderr: each queued message with its readiness and its line's state, the pending-request lines, RX occupancy, and MSHR, write-back-buffer and data-access-buffer occupancy. It names the line and the transient state everyone is waiting on, and it finalizes the raw trace so the cycles into the stall are readable. The sequence that works: hang dump $\rightarrow$ address $\rightarrow$ an aggregated transition trace filtered to that address on the LLC and every L1 $\rightarrow$ the last few transitions before the stuck state. MCSIM_PROBE=<file> does the same for the DRAM model: every request, command and completion with MCsim's own clock, so two builds can be diffed decision by decision.

11.5 The visualizer

`./octoviz.sh view <workload_dir>` simulates a workload with the trace on, converts the run and opens *octoviz* in the browser; `run`, `convert` and `serve` are the same steps separately, and `serve <dir>` shows every converted run under a directory, such as a sweep. It is a local web app (Python 3 with duckdb and numpy; data never leaves the machine) that shows a run as a pipeline:

Pipeline one row per request in the window, sorted by issue, coloured segments per stage, a mark where the request became the oldest of its core; hover for the row, click to inspect.

Resource lanes request lane, service lane, LLC array port, memory bus (both directions), DRAM, response lane; each segment the occupying message, coloured by kind, core or class, with cut-ins outlined dashed. Bright is the filtered set, dim is everything else, so *who is ahead of me* is visible.

Inspector the row's fields, every request on the same line within ± 2000 cycles, and every trace interval of the selected id.

Coherence transition table every event a controller's FSM processed on one line, in cycle order, with the state of every agent after it: one row is the line's whole state vector.

Minimap and level of detail request density over the run; above a window size the server returns per-bin aggregates instead of rows, so a 98-million-request SPLASH run still answers in milliseconds.

Filters — cycle window, address with a mask, request id, cores, class, resources used, stall state, oldest-only, responses ahead — are AND-combined and kept in the URL hash, so a view is a shareable link; views can be popped out into their own windows, which stay synchronised.

Where in the code

- `header/Logger.h`, `src/Logger.cpp` — the Logger, the trace writer, the reports
- `header/DebugPrint.h`, `src/DebugPrint.cpp` — the Debugger
- `tools/octoviz/convert.py`, `tools/octoviz/occupancy.py`, `tools/octoviz/tracefile.py` — the converters
- `tools/octoviz/server.py`, `tools/octoviz/static/index.html` — the viewer
- `octoviz.sh` — the entry point
- `docs/Logger.md`, `docs/Debugger.md`, `docs/Trace.md`, `docs/Visualizer.md` — the same material online

Periodic tasks and the interference demo

What you will find here. How a core runs a periodic real-time task instead of replaying a trace, with the simulator reporting each job's release, finish and deadline; how a streaming neighbour evicts that task's lines through an inclusive shared cache; the one knob that stops it, LLC way reservation; and the demo that turns those measurements into a robot leaving its path.

12.1 Periodic tasks

A core can run a periodic task: a fixed job body of compute phases and memory accesses, re-run every period, with the simulator itself reporting each job's release, finish, deadline and miss. The memory system, protocols and arbiters are untouched; the task layer lives in `CPU::periodicLogic` and is opt-in per core. For core n the system looks in the workload directory for `task_C<n>.task.csv` first, then `trace_C<n>.trc.shared`, then nothing — an idle core that issues nothing and finishes at once, which is how “core 0 alone” is expressed.

Listing 12.1. A task program. `attr` rows set the task's parameters; the phase rows are the job body.

```
kind,a,b,c,d,comment
attr,period,50000,,,control period in CPU cycles (= implicit deadline)
attr,jobs,200,,,number of jobs, then the core finishes (-1 = free-running
    generator)
attr,ooo,1,,,outstanding-request window (1 = in-order core)
COMPUTE,300,,,advance 300 CPU cycles, no memory
READ,0x600040,,,one read
WRITE,0x600000,,,one write
LINEAR,0x600080,5,64,1.0,burst: base, count, stride, read ratio
RANDOM,0x2000000,0x400000,1024,0.85,base, range, count, read ratio (line-
    aligned)
```

With period T , relative deadline D (default T) and job index j , non-preemptive, one instance at a time: $\text{release}_j = \max(\text{finish}_{j-1}, jT)$, $\text{deadline}_j = jT + D$, finish_j is the cycle the last access of the body has returned plus trailing compute, and a job misses when $\text{finish}_j > \text{deadline}_j$. An overrun pushes the next release out; periods whose whole window ended before the overrunning job finished are *skipped* — no job is released for them, each is a missed deadline, and they are reported on the overrunning job's row. `attr, jobs, -1` makes a free-running generator with no period and no deadline: a memory aggressor, which stops when every finite core is done. `LINEAR` and `RANDOM` draw from a per-core deterministic generator, so runs are reproducible. The Logger writes `JobReport_C<n>.csv` next to the latency report, one row per job, flushed as the job finishes (Chapter D).

What is not modelled is as important: compute is a fixed cycle budget, so a task program is a

timing model, right for interference studies and not for data-dependent control flow; there is no preemption and no multi-task scheduling on a core; deadlines are observed and reported, never enforced.

12.2 Inclusion interference

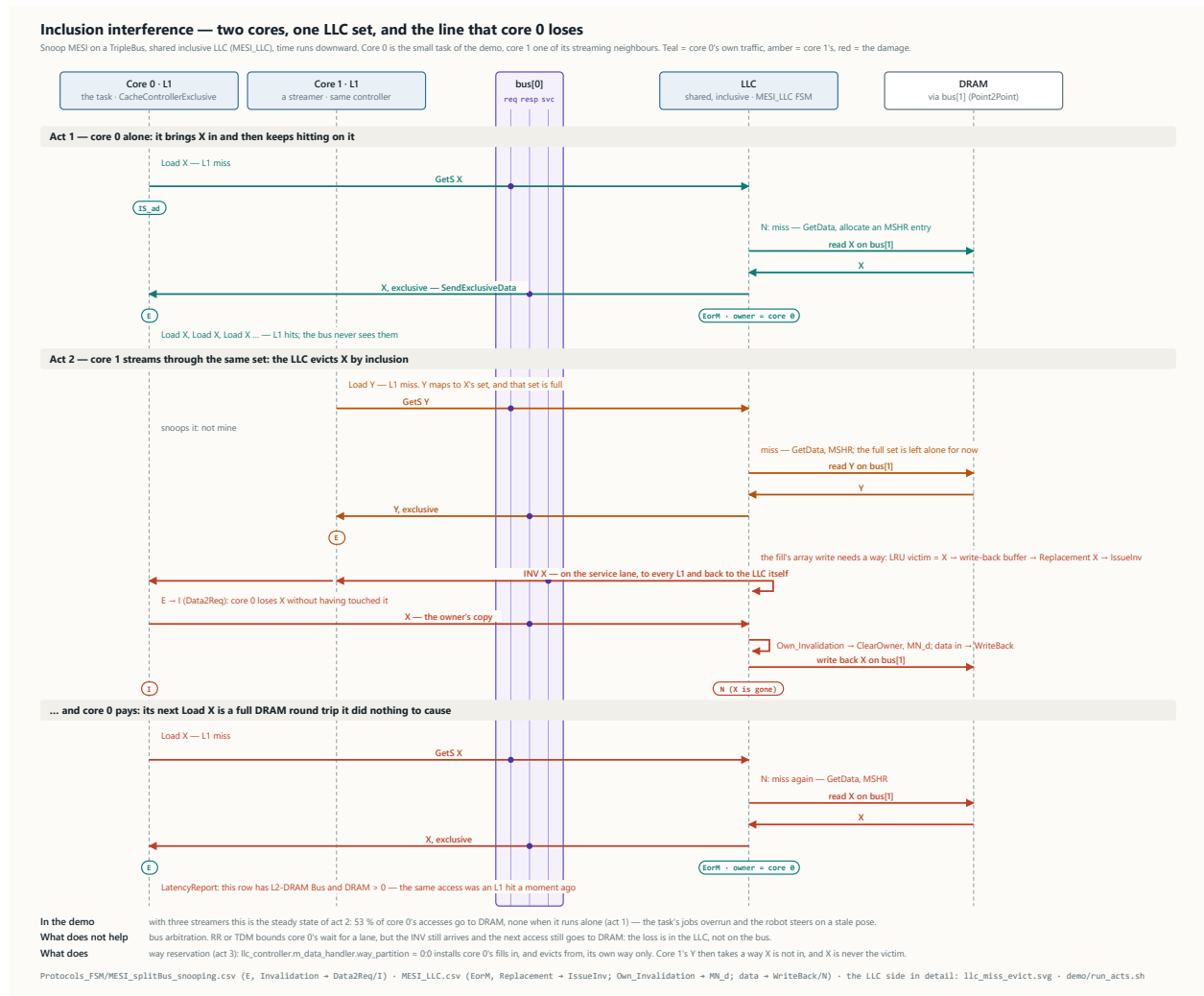


Figure 12.1. Two cores, one LLC set, and the line core 0 loses. Core 0 brings X in and hits on it in its L1; core 1 misses on Y in the same set; the fill evicts X; the invalidation on the service lane removes X from core 0's L1 too; core 0's next Load X is a full DRAM round trip it did nothing to cause.

Nine cache lines fit comfortably in a private L1, so a small task's working set should never leave it; yet under three streaming neighbours more than half of such a task's accesses go to DRAM. The mechanism is Figure 12.1, the same eviction Chapter 5 draws from inside the LLC, seen from the cores. The shared LLC is inclusive, so when it evicts a line it owns it must first invalidate the copy in the owner's private L1 (EorM → Own_Invalidation → MN_d on the LLC, E → Invalidation → I in the L1). The LLC is 32 KB and 2-way, hence 256 sets; the task's nine lines land in a handful of sets, and streamers sweeping 256 KB and 4 MB regions cover every set repeatedly. In the demo's contention act the LLC evicted 45 728 lines, 195 of them one of the task's nine — and 195 is precisely the number of task accesses that had to leave the L1. The second half of the damage is the memory controller: FR-FCFS prefers the streams' row hits over an isolated read, so the task's few DRAM reads waited a mean of 4 524 cycles and up to 30 904, against 739 and 25 024 for a streamer. The low-intensity core is penalised

precisely because it is well behaved.

Bus arbitration cannot prevent any of this. Round-robin or TDM bounds the task's wait for a lane; the invalidation still arrives and the next access still goes to DRAM, because the loss is in the cache, not on the bus.

12.3 Way reservation

```
llc_controller.m_data_handler.way_partition(s),0:0;1-3:1
```

The format is <cores>:<ways>; . . . with a-b ranges. A listed core's fills are installed in, and evict from, its own ways only; cores not listed share the ways nobody claimed, so 0:0 alone reserves way 0 for core 0 and leaves the rest to everyone else. It is implemented in the data handler — the empty-way search and the replacement candidate are restricted to the requester's mask, and the controller names the requester while a fill is installed — with masked candidates in the LRU and random policies. Empty means no partitioning, byte-identical to before. With 0:0 on the 2-way LLC the demo's task under full contention runs exactly as it does alone.

12.4 The demo

A mobile robot follows a planned path. Its localization task — an EKF pose update of about 4 150 compute cycles and twenty accesses over nine lines, period 5 000 — runs on core 0 of a four-core machine with the shared LLC and one DDR4 controller. When three other cores stream memory, the localization job finishes late, the robot steers on a stale waypoint and leaves the path; reserving one cache way puts it back exactly where it was alone. Every millisecond of that story is a cycle-accurate Octopus run.

Table 12.1. The three acts. All use the sweeps' pinned configuration — snoop MESI, 32 KB 2-way inclusive LLC, Msim DDR4 with FR-FCFS — and differ only in per-run -p overrides.

act	what differs	jobs	missed	job time mean / worst	path error rmse / max
1 alone	cores 1–3 idle, FCFS bus	200	0	4 193 / 4 747	0.13 m / 0.18 m
2 contention	three streaming cores, FCFS bus	57	198	19 717 / 181 266	4.13 m / 6.63 m
3 protected	same load, RR bus + way_partition=0:0	200	0	4 193 / 4 747	0.13 m / 0.18 m

The chain is short (Table 12.1): `demo/run_acts.sh` runs the three acts, each a workload directory with the task on core 0 and streamers or nothing on the others; the Logger writes the job and latency reports; `demo/extract_jobs.py` joins them into one record per job; `demo/plot_trajectory.py` draws Figure 12.2 and `build_page.py` the self-contained page. The robot's motion is a function of two numbers per job, the release and the finish: job j senses the pose at its release and plans the waypoint for one period later, the waypoint is applied when the job finishes, and the robot drives toward it at a fixed speed cap. The path's shape, its scale, the speed cap and the playback rate are chosen for presentation; everything about timing is measured, and acts 1 and 3 produce byte-identical job reports. In act 2 one job ran 181 266 cycles inside a 5 000-cycle period, its slowest access 27 848 cycles of which 27 832 were DRAM; its sum of access latencies plus the compute budget equals its execution time exactly. Nothing is added on top of the simulation.

Robot trajectory vs. the localization task's timing under shared-memory contention (Octopus, period 5000 cycles, 200 periods)

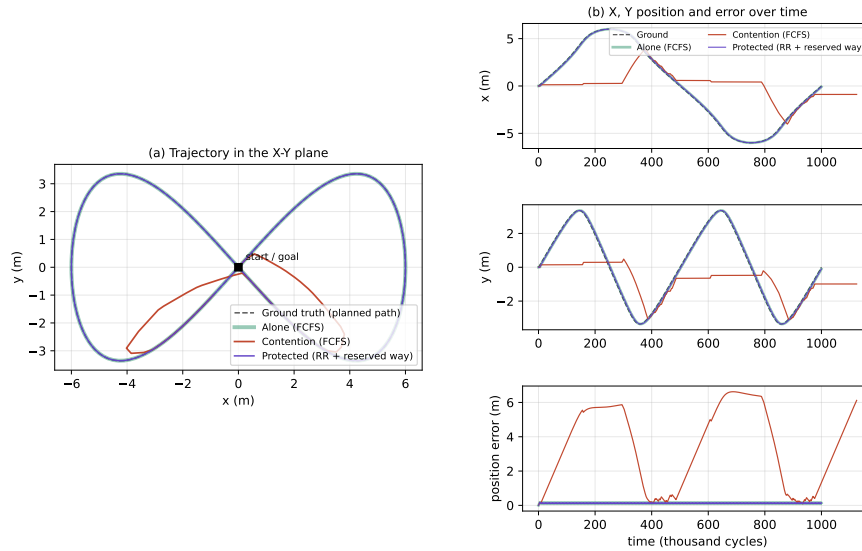


Figure 12.2. The robot's path in the three acts: alone and protected coincide exactly; under contention one 181 266-cycle job left the robot steering for 35 periods on a stale waypoint, the long chord across the figure.

Where in the code

- `src/CPU.cpp` — `periodicLogic`
- `src/CacheDataHandler.cpp` — way partitioning
- `docs/Tasks.md` — the task layer as a reference
- `demo/run_acts.sh` — the three acts
- `demo/tasks/` — the task programs
- `demo/extract_jobs.py`, `demo/plot_trajectory.py`, `demo/build_page.py` — the post-processing
- `demo/README.md` — what is measured and what is drawn
- `docs/imgs/inclusion_interference.svg` — the sequence figure

Extending Octopus

What you will find here. Where to edit, for each kind of extension; what each costs in code and in rebuilds; the three worked extensions the tutorial uses — an arbiter, a protocol change, a third cache level — with the trap in each; and the extension points that are planned but not yet there.

13.1 The map

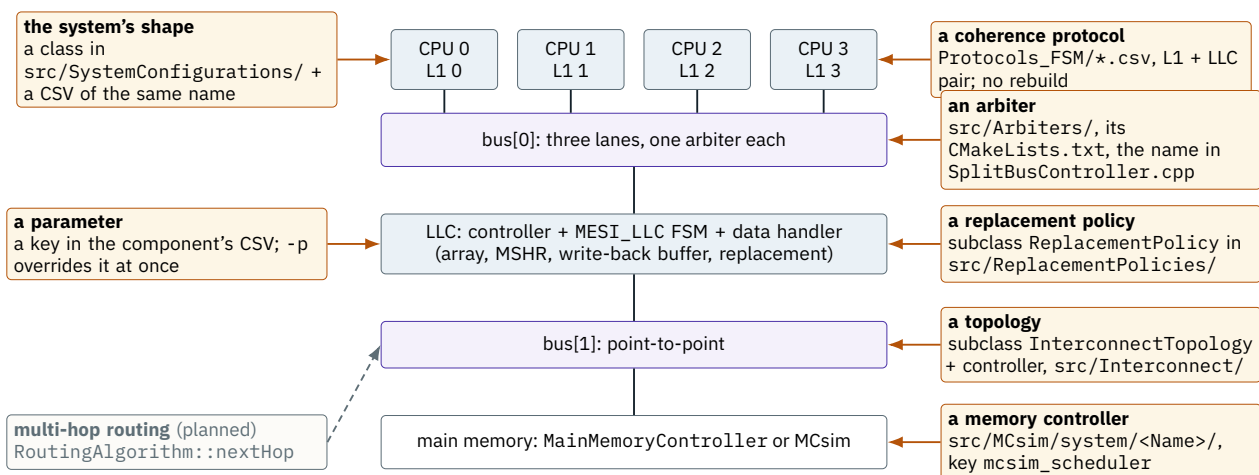


Figure 13.1. The shipped system with a pin on every extension point. Amber pins exist; the grey one is planned.

The split of coherence into a handler plus a CSV table is why a protocol variant costs a spreadsheet edit and no rebuild, and why MESI and MOESI extend MSI at a fraction of the code of monolithic simulators (Table 6.1). Everything else is a subclass with a handful of virtual methods and a name in a factory. A rebuild after touching one file is about fifty seconds on a four-core codespace: `cmake -build build -j$(nproc)`.

13.2 An arbiter

An arbiter is a subclass of `Arbiter` implementing one pure virtual, `elect` (Section 8.3). The base class offers `findMessage()` and `electOldestOwned(buffers, owner, out)`, which picks the oldest message owned by a given core across all buffers; use it rather than re-deriving it, because a core's cache-to-cache supplies sit in a different buffer from its own requests (Section 8.3). A weighted round-robin where core 0 gets two slots per round is about twenty

Table 13.1. What each extension touches.

to add ...	do this	rebuild
a coherence protocol or variant	write or edit a CSV FSM, L1 and LLC tables as a pair (Chapter 6); a new family with new actions is a protocol-handler subclass in <code>src/Protocols/</code>	no (yes for a new family)
an arbiter	subclass <code>Arbiter</code> , implement <code>elect()</code> , add the file to <code>src/Arbiters/CMakeLists.txt</code> , register the name in the controller's factory	yes
a replacement policy	subclass <code>ReplacementPolicy</code> (<code>update</code> , <code>getReplacementCandidate</code>); honour the way mask	yes
an interconnect topology	subclass <code>InterconnectTopology</code> and <code>InterconnectController</code> ; the controllers on either side see only interfaces	yes
a memory controller	a scheduler directory under <code>src/MCsim/system/</code> , selected by <code>mcsim_scheduler</code> ; about 130 lines on average	yes (MCsim)
a system shape	a class beside <code>MultiCoreSystem</code> in <code>src/SystemConfigurations/</code> plus a CSV of the same name	yes
a parameter	a key in the component's CSV, read from parameters in its constructor; overridable with <code>-p</code> at once	yes
a Logger stage	stamp <code>ENTER/SERVICE/EXIT</code> with a new Role; the decomposition walks named events (Chapter 11)	yes

lines against RRArbiter, and its effect shows at once in core 0's worst-case request-bus latency:

```
cp header/Arbiters/RRArbiter.h header/Arbiters/WRRArbiter.h
cp src/Arbiters/RRArbiter.cpp src/Arbiters/WRRArbiter.cpp
sed -i 's/RRArbiter/WRRArbiter/g' header/Arbiters/WRRArbiter.h src/Arbiters/
WRRArbiter.cpp
# add WRRArbiter.cpp to src/Arbiters/CMakeLists.txt; register the name in
# src/Interconnect/SplitBusController.cpp; rebuild; then:
./build/Octopus_Simulator -s MultiCoreSystem -p "workload_path(s)=$W/" \
-p "bus[0].interconnect_controller.arbiter_type(s)=WRRArbiter"
```

With an unmodified copy of RR you must get exactly RR's numbers — that is the check that the registration works — and only then make it weighted. `cacheb01`, where four cores fight over one block, is the benchmark on which the arbiter matters most.

Register it on the bus you are measuring

The shipped `bus[0]` is a `TripleBus` whose controller type is `Split`, so `src/Interconnect/SplitBusController.cpp` is the factory that decides which arbiters exist on it. `MeshController.cpp` and `NoCController.cpp` have the same chain for the mesh; registering there instead compiles cleanly and does nothing.

13.3 A protocol change

The protocol is `Protocols_FSM/*.csv`, one table per agent, read at start-up; no compiler is involved (Chapter 6). Two exercises of different depth: diff `MESI_splitBus_snooping.csv` against `M0ESI_splitBus_snooping.csv`, find the O state, list every transition into and out of it, say in one sentence what it buys and what it costs, then run both on `cacheb01` and check the sentence against the response-bus column; or pick a race the table handles with a `Stall/` and handle it with an explicit transient state instead, run `a2time01`, and confirm the result is identical — only *how* the case is handled changed, not *what* happens. Keep a copy of `Summary.csv` from before the edit and diff it after. The one real trap is the row order (Section 6.2): a new state's row goes last.

13.4 A third cache level

`src/SystemConfigurations/MultiCoreSystem.cpp` hardwires the shape — a loop creating one L1 per core on `bus[0]`, one LLC between `bus[0]` and `bus[1]`, memory beyond. There is no parameter for a third level; the wiring is in C++, which is the boundary between data and code (Chapter 10). So this is a new class beside `MultiCoreSystem` and `MultiCoreSystem_Mesh`, selected with `-s ThreeLevelSystem`, which needs `ThreeLevelSystem.{h,c}` and configuration `/SystemConfigurations/ThreeLevelSystem.csv` — the same name is how `-s` finds both. Every building block exists and is generic: `createController(type, map, lower, upper, name)` makes a controller of any type from a parameter sub-map, `Bus` and `TripleBus` build from `getSubMap("bus", i)`, `bus_type` already sizes the bus array from the CSV so a third bus is a CSV line, and `getSubMap("cache_controller", i)` gives per-instance parameters. The new class is `MultiCoreSystem.cpp` with one more loop: L1s on `bus[0]`, private L2s between `bus[0]` and `bus[1]`, the LLC between `bus[1]` and `bus[2]`, memory on `bus[2]`; register it where `MultiCoreSystem_Mesh` is registered and add the source to the `SystemConfigurations` CMake list.

Two things to look at when it runs. The report’s L2 Stall and L2 Access now mean the private L2, and the LLC’s stages fold into the bus and DRAM columns — the Logger attributes stages per controller through `setLogRole`, and a new level deserves its own role. And coherence: a private L2 between a snooping L1 and the LLC is not a transparent box; something has to snoop on behalf of the L1. The cheapest correct answer is an inclusive L2 that forwards every bus transaction up, which is what the reference solution does; what a real design would do instead is the discussion the exercise exists for.

13.5 Planned

Multi-hop routing on the NoC is designed, not built: a `Topology` that knows coordinates and endpoint attachment, a `RoutingAlgorithm` plug-in with one method, `nextHop(router, dest, msg)`, selected by name like an arbiter, input-buffered routers with credit-based back-pressure, and three virtual networks so that a directory protocol’s request-forward-response dependency cannot deadlock. The current single-switch NoC becomes its degenerate case, and “add a routing algorithm” becomes the mesh’s analogue of “add an arbiter”. `docs/NoC_MultiHop_Plan.md` has the design and its phasing.

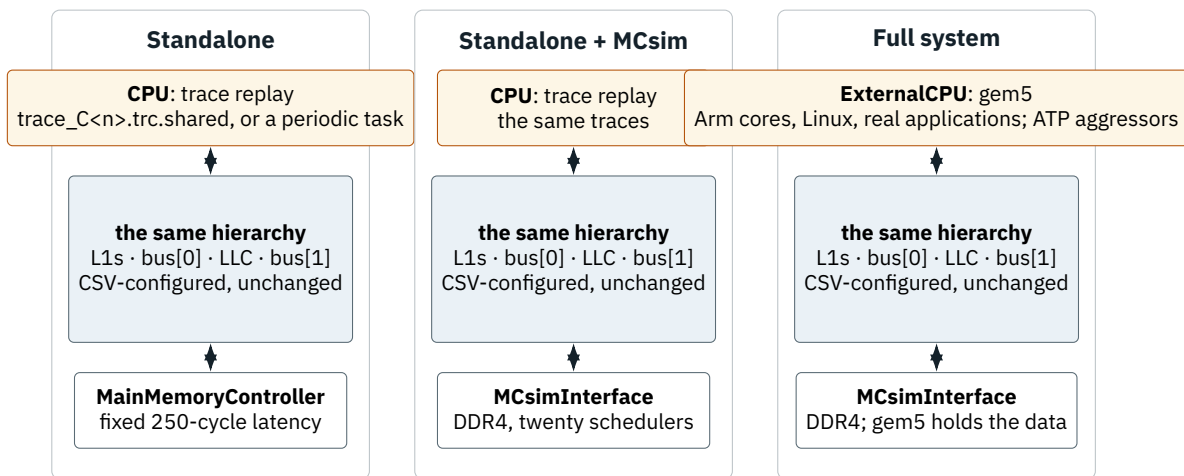
Where in the code

- `header/Arbiters/Arbiter.h` — the arbiter interface
- `src/Interconnect/SplitBusController.cpp` — the arbiter factory for the bus
- `header/ReplacementPolicies/ReplacementPolicy.h`,
`src/ReplacementPolicies/` — replacement (LRU, random)
- `src/SystemConfigurations/` — the system classes
- `tutorial/03-extending-octopus/` — the three exercises with their checks
- `tutorial/solutions/` — the reference patches

Integration modes: gem5 and the full-system stack

What you will find here. The three ways to run the same hierarchy — trace replay, trace replay over MCsim, and a real CPU model under gem5 with Arm cores booting Linux — and why the middle never changes; what the gem5 bridge does, what the Arm ISA support cost, how the stack is built and run, and the one constraint that shapes any session using it.

14.1 Only the endpoints change



the endpoints are CommunicationInterface producers and consumers; swapping them changes the request source and the memory sink, nothing in between

Figure 14.1. Three operating modes, one hierarchy.

The request source and the memory sink are CommunicationInterface producers and consumers like every other component (Figure 14.1). In standalone mode the source is the trace-driven CPU and the sink the fixed-latency MainMemoryController; swapping the sink for MCsimInterface gives cycle-accurate DRAM (Chapter 9); swapping the source for ExternalCPU puts a real CPU model in front. ExternalCPU offers an API through which an external simulator pushes memory requests and registers a callback fired when each is fulfilled; nothing downstream can tell the difference, and the same CSV-configured hierarchy runs unchanged in every mode. A user chooses speed by omitting interfaces, single-side accuracy, or both for the highest fidelity.

14.2 Full system: gem5, Arm, Linux

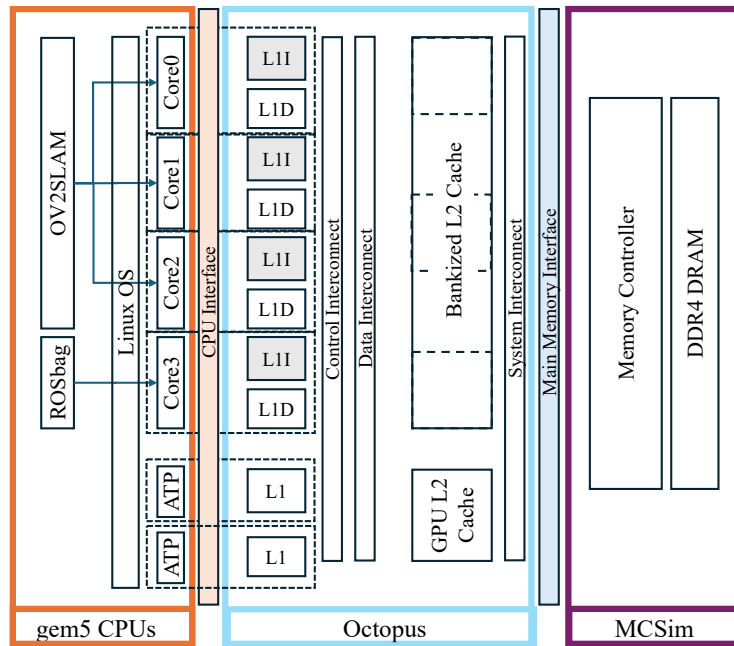


Figure 14.2. Full-system integration: gem5 executes Arm cores under Linux and holds the functional memory values; Octopus supplies the cycle-accurate timing and coherence for every request; MCSim models the DDR4 controller. The ATP engine injects aggressor traffic.

Standalone trace mode is fast, but embedded validation eventually needs a real OS and real applications. Octopus therefore also runs as a first-class memory model inside gem5 (Figure 14.2): gem5 executes out-of-order Arm cores booting Linux and holds the functional memory values, while Octopus supplies the timing and coherence for every request and MCSim optionally models the DDR controller. Octopus is compiled *into* gem5 as a scons EXTRAS module, together with Arm’s Adaptive Traffic Profile engine for GPU- and DPU-like aggressor traffic:

```
cd $root/OctopusSimulator/MCsim/src && make libmcsim.so           # MCsim as
a shared library
cd $root/OctopusSimulator && cmake -S . -B build && cmake --build build
cd $root/gem5 && scons EXTRAS=../ATP-Engine:../OctopusSimulator -j $(nproc)
build/ARM/gem5.fast
export LD_LIBRARY_PATH=$root/OctopusSimulator/build:$root/OctopusSimulator/
MCsim/src:$LD_LIBRARY_PATH
./build/ARM/gem5.opt -re -d <out> $root/OctopusSimulator/gem5/configs/fs_arm.
py
./build/ARM/gem5.opt -re -d <out> $root/OctopusSimulator/gem5/configs/fs_arm.
py --classic           # gem5's own caches, same preset
./build/ARM/gem5.opt -re -d <out> $root/OctopusSimulator/gem5/configs/fs_arm.
py \
--octopus-param 'bus[0].interconnect_controller.m_response_latency(i)=1'
```

The bridge, its gem5 configurations and the preset that describes the hierarchy behind gem5 (MultiCoreSystem_gem5.csv) live under gem5/ on the gem5_ARM_Challenge branch and in the ESWEEK container image; the tutorial branch carries the simulator side (ExternalCPU) and the build instructions. `--octopus-param` takes any name (type)=value line of the preset and may be repeated, so the five keys of a protocol switch it to MESI without rebuilding anything. After a change to the library, `cmake -build build` and the scons line relink only the bridge.

The Arm ISA is the hard part. Booting Linux requires faithful atomics over Octopus’s asynchronous, out-of-order message model. The bridge implements Arm LL/SC (LDXR/STXR) with per-core exclusivity maps, revoked on any conflicting write, and the LSE atomic memory operations (SWP, CAS/CASP, the LD/ST arithmetic and logical variants), with ordering constraints that keep exclusivity correct even when responses return out of order. The bridge is about a thousand lines; integration and Linux bring-up took about six months, a one-time cost every later full-system study reuses. On the Arm Industrial Challenge AR-HUD workload — an OV²SLAM stereo pipeline plus a CNN pose estimator on a Jetson-class SoC model — coordinating the shared arbiters cut the worst observed memory latency by more than twenty times at near-baseline throughput.

Harnesses. On the same branch, `gem5/harness/build.sh` builds three single binaries against `build/libOctopus.so` with no `gem5: l1` (hit and miss latencies, same-line bursts), `reorder` (with an override, e.g. a 10-cycle L1 port) and `stress` (multi-core random contention with a hang detector). Each takes a log directory and name (`type`)=`value` overrides. Two debug aids apply to any run: `OCTOPUS_TRACE_ADDR=<addr>` (or 1) prints each controller’s events on a line, `OCTOPUS_DUMP_AT=<cycle>` dumps every controller’s queues at that cycle. The standalone presets are the regression check for library changes: run `MultiCoreSystem_Directory` and `MultiCoreSystem_Snoop` and `diff BMs/TestBM/newLogger/` against a build of the commit you started from.

14.3 Running a session on it

The constraint that shapes any use of the stack: an Octopus C++ edit means relinking `gem5.opt` — minutes, not the fifty-second rebuild of standalone work. A session on `gem5` therefore varies configuration, never Octopus source, and an arbiter written for the standalone build appears inside `gem5` only if the image’s `gem5.opt` was built with it. Two shapes of exercise follow. In syscall-emulation mode, one short program under `gem5` makes the interface concrete: the request leaves the `gem5` CPU, crosses `ExternalCPU`, becomes an ordinary `Message` through L1, bus, LLC and `MCsimInterface`, and its `LatencyReport` row decomposes exactly that path — the same code that ran standalone, with only the source changed. In full system, a checkpoint taken past Linux boot is restored and a workload runs on a real kernel with Octopus as its memory system; booting is separated from measuring, and the pre-warm is launched in the background long before it is looked at. The tutorial’s two `gem5` exercises (Chapter 15) are these; the ESWEEK container image carries the prebuilt `gem5.opt`.

Where in the code

- `header/ExternalCPU.h`, `src/ExternalCPU.cpp` — the CPU-side interface
- `header/MCsimInterface.h`, `src/MCsimInterface.cpp` — the memory side
- `README.md` — “Integrating Octopus with `gem5`”: the layout, the build and the run commands
- `gem5/` on the `gem5_ARM_Challenge` branch — the bridge, its configurations (`configs/fs_arm.py`, `configs/octopus_cache_hierarchy.py`), the harnesses and `cmds.md`; the ESWEEK container image carries them prebuilt
- `tutorial/04-gem5-and-fullsystem-stack/` — the two exercises

Chapter 15

Hands-on tutorial

What you will find here. The five exercises of the hands-on tutorial as they ship under `tutorial/`, in order, each with its goal, its commands, what to look at and the trap it contains.

15.1 How the exercises are built

Each exercise folder has the same shape: a `README.md` with the exact commands and what you should see, a `check.sh` that runs the exercise and says in one line whether it worked, an `expected/` directory holding the reference result it compares against, and where needed a `start.sh` that puts the starting files in place. The simulator is deterministic and platform-independent — the same inputs give the same cycle counts on Linux, Windows and in a codespace, to the last digit — so `check.sh` is an equality test, not a smoke test: if your numbers differ from the reference, something real changed, usually a file under `configuration/`. Two commands to remember throughout:

```
bash scripts/check_environment.sh      # is my environment healthy?
git checkout -- configuration/         # put every configuration file back as
    shipped
```

Fell behind? `bash tutorial/solutions/apply.sh <exercise>` applies the reference solution for that exercise so you can continue with the next one.

The exercises come in two blocks. The first runs Octopus standalone: setup and first run (about 15 minutes), exploration (25), the configuration model (10) and extending Octopus (45, one exercise of three). The second puts the same hierarchy behind `gem5`: syscall-emulation mode (15–20) and full-system Arm from a checkpoint (25–30, plus a take-home), whose pre-warm is launched at the start of the block.

15.2 00 — Setup and first run

Goal: a working environment, one completed simulation, and a first read of its output. Chapter 2 is this exercise in full: run `scripts/check_environment.sh` and expect eight `[PASS]` lines ending in *15364 requests, worst DRAM 359 cycles*; run `a2time01` on the shipped configuration; read `Summary.csv` and the first rows of `LatencyReport_C0.csv` with Figures 7.1 to 7.3 open; run `check.sh`, which compares `Summary.csv` byte for byte against the reference; then open the visualizer and find the pipeline, the lanes and the transition table. One thing to know before the next exercise: `run_octopus.sh -protocol` copies a preset over `MultiCoreSystem.csv`, so hand edits to that file vanish.

15.3 01 — Exploration, one axis at a time

Goal: convince yourself the axes really are independent. Change one thing, re-run in seconds, watch one number move. The helper runs `a2time01` with any overrides and prints the three numbers worth comparing:

```
W=$PWD/BMs/eembc-traces/a2time01-trace
run(){ ./build/Octopus_Simulator -s MultiCoreSystem -p "workload_path(s)=$W/"
"$@" >/dev/null 2>&1 \
    && awk -F, 'NR==2{print "  worst total="$10"  worst DRAM="$8"  finish="
$13}' $W/newLogger/Summary.csv; }
echo FCFS; run -p "bus[0].interconnect_controller.arbiter_type(s)=FCFSArbiter"
echo RR; run -p "bus[0].interconnect_controller.arbiter_type(s)=RRArbiter"
echo TDM; run -p "bus[0].interconnect_controller.arbiter_type(s)=TDMArbiter"
echo MCsim; run -p "main_memory_type(s)=MCsim" -p "mcsim_scheduler(s)=FRFCFS"
echo MSHR4; run -p "cache_controller[*].num_mshr(i)=4"
echo part; run -p "llc_controller.m_data_handler.way_partition(s)=0:0;1-3:1"
```

Fill in a table as you go; the point is not the numbers but that each row differs from the last in exactly one parameter. The part row is the one with a story behind it (Chapter 12). The protocol is the one axis that is not a `-p`: switch it by preset with `run_octopus.sh -protocol snoop|directory` (Chapter 10). If you finish early, run the same six on `cache01-trace`, where all four cores fight over one block and the arbiter rows spread out. Two things the numbers will show: the fixed-latency model reports a worst DRAM of exactly the configured latency where MCsim reports 359 on the same trace — the fixed model is optimistic exactly where you would expect — and a fault after switching the protocol by hand is almost always the five-key coupling, not a protocol bug.

15.4 02 — The configuration model

Goal: understand why configuration looks the way it does, so the extension exercises make sense. Ten minutes of reading, nothing to build: open `BaseController.csv`, `CacheController.csv`, `CacheControllerExclusive.csv` and `MultiCoreSystem.csv` in that order and follow one key through Extends and the per-instance overrides; read the typed suffixes; find where the L1 MSHR depth is set, override it to 1 with `-p` and watch the L1 stall column grow; see that `-s` selects a C++ class and a CSV by the same name, which is the boundary the third exercise of 03 crosses; and look at `git diff` after a `run_octopus.sh -protocol` to see exactly which five keys a protocol is. Chapter 10 is the chapter.

15.5 03 — Extending Octopus

Three exercises of different natures; pick one, because 45 minutes is enough for one done properly. All three edit the real source tree — `src/Arbiters/`, `src/Protocols_FSM/`, `src/SystemConfiguration` — not a copy; the exercise folders hold only the instructions and the answer key. Chapter 13 has each in full.

01 — a bus arbiter (C++, small, well-fenced; a fifty-second rebuild). Weighted round-robin from a copy of `RRArbiter`; the trap is registering the name in `SplitBusController.cpp`, the bus you are measuring, not in the mesh's controller.

02 — a coherence protocol (CSV only; no rebuild). Read the O state out of the MESI/MOESI diff, or replace a `Stall/` with an explicit transient state; the trap is the row order.

03 – a third cache level (a new system class; driven from the front). The boundary between data and code, crossed; the open question is who snoops for an L1 behind a private L2.

15.6 04 – gem5 and the full-system stack

Everything so far ran Octopus standalone; here the request source is gem5 and the same hierarchy, unchanged, sits behind it (Chapter 14). The one constraint to know first: Octopus is compiled into gem5 as an EXTRAS module, so an Octopus C++ edit means relinking `gem5.opt`; both exercises vary configuration only.

01 – gem5 + Octopus in SE mode (15–20 min). One short program under gem5; trace the journey of a request through `ExternalCPU`, the hierarchy and `MCsimInterface`; compare its `LatencyReport` with the same configuration standalone; vary the bus arbiter in the configuration and re-run.

02 – full-system Arm Linux (25–30 min plus a take-home). The pre-warm, launched at the start of the block, restores a checkpoint taken past boot and starts the workload; the walk-through covers what the checkpoint contains, why booting is separated from measuring and how LL/SC and the LSE atomics reach the protocol; then everyone tails their own log and opens the `LatencyReport` it is writing. The take-home is the full journey – boot, reach the region of interest, write the checkpoint – with its real durations.

Traps, collected

The preset copy over `MultiCoreSystem.csv`; the protocol as five coupled keys; the FSM row order; the arbiter registered in the wrong controller; directory under TDM starves one core, so the directory preset pins FCFS and only a hand override reaches it; a run that wedges leaves a readable trace only because `OCTOPUS_HANG_DUMP` finalizes it; and on Windows in Git Bash the workload path wants `cygpath -m`.

Where in the code

- `tutorial/README.md` – the index
- `tutorial/00-setup-and-first-run/`, `tutorial/01-exploration/`, `tutorial/02-configuration/`, `tutorial/03-extending-octopus/`, `tutorial/04-gem5-and-fullsystem-stack/` – the exercises, each with its `README.md`, `check.sh` and `expected/`
- `tutorial/solutions/` – the patches and `apply.sh`

Evaluation

What you will find here. What the sweep harness measures and the figures this chapter will carry once the sweeps have been re-run on the current code. The numbers themselves are deliberately absent: the results and figures currently under `results/` were produced before the inclusive-LLC eviction and bus-ordering fixes and are stale.

16.1 What is measured

Every sweep script varies one axis against the fixed baseline — snoop MESI, TripleBus, LRU, FCFS bus, fixed-latency memory, default geometry — over the EEMBC suite (and, for the non-memory axes, SPLASH-2), and records per (value, benchmark) whether the run completed and the worst case of every stage, the average effective latency, the finish cycle and the worst head-of-queue latency (Chapter 10, Chapter D). The per-axis component shows *where* a knob acts: the arbiter moves the bus columns, the LLC capacity the DRAM column, the memory model the DRAM column again by a different amount, replacement almost nothing.

16.2 The figures to come

- **Per-request worst-case latency by stage, across axes** — the stacked bars that show which shared resource a knob moves (`fig1_stage_breakdown`).
- **Mean versus worst-case latency by bus arbiter** — FCFS, RR and TDM on a log scale: the predictability-for-throughput trade (`fig4_wcet_vs_avg`).
- **LLC capacity and the DRAM column** (`fig2_cache_capacity`); **fixed latency against MCsim** (`fig3_memory_dram`); **normalised latency across axes** (`fig5_normalized`); **the axis-by-metric heatmap** (`fig6_config_heatmap`); **completion status per configuration and benchmark** (`fig7_status_matrix`).
- **The three acts of the demo** in numbers (Table 12.1 already carries them).
- **Analytical against measured:** the per-resource bound summed, against the measured worst case of the oldest request, which is the additive-latency argument of Chapter 1 made on this simulator.

16.3 Reproducing

```
bash sweeps/sweep_all.sh           # every single-axis sweep, EEMBC
bash sweeps/run_overnight.sh       # both suites, giants once under
baseline
```

```
python sweeps/plot_results.py           # results/figures/*.{png,pdf}
```

REPRODUCIBILITY.md gives the runtime expectations per axis and suite and the artifact map from figure to script; results/README.md explains the layout and the columns. When the sweeps have been re-run, this chapter embeds the figures from results/figures/ and replaces this note.

Chapter A

Configuration key reference

Every key in `configuration/**/*.csv` as shipped, one table per file, generated by `docs/manual/tools/gen_config_tables.py` at build time. The type is the suffix the key carries in the CSV ((i) integer, (s) string, (vi)/(vs) lists); the value column is the shipped default. Chapter 10 explains how files extend each other and how a `-p` override reaches a key.

configuration/SystemConfigurations/MultiCoreSystem.csv

key	type	value	meaning
<code>workload_path</code>	string	<code>BMs/TestBM/</code>	Absolute path is recommended
<code>num_cores</code>	int	<code>4</code>	
<code>bus_type</code>	string list	<code>TripleBus, Bus</code>	Bus[0] (L1<->LLC) is TripleBus: carries the LLC back-invalidation on its service channel. Bus[1] is LLC<->memory snoop broadcast bus
<code>bus[0].controller_type</code>	string	<code>Split</code>	
<code>bus[0].m_lower_level_ids</code>	int list	<code>0, 1, 2, 3</code>	LLC ID
<code>bus[0].upper_level_cache_ids</code>	int list	<code>10</code>	
<code>cache_controller_type</code>	string	<code>Cache Controller Exclusive</code>	L1 private caches: basic snoop controller
<code>llc_controller_type</code>	string	<code>Cache Controller End2End</code>	LLC: End2End controller drives TripleBus service channel for back-invalidation on eviction
<code>bus[*].buffers_max_size</code>	int	<code>256</code>	Response/request buffer depth: usable + in-flight reserve (sized to MSHR bound)
<code>bus[0].response_reserve</code>	int	<code>128</code>	Snoop bus in-flight response reserve: request bus stalls when response buffer free space falls to this (bounds response-bus back-pressure without overflow or controller gating)
<code>bus[1].m_lower_level_ids</code>	int list	<code>10</code>	LLC ID
<code>bus[1].upper_level_cache_ids</code>	int list	<code>100</code>	Main memory ID
<code>bus[1].controller_type</code>	string	<code>Point2Point</code>	
<code>bus[1].interconnect_controller.candidates_id</code>	int list	<code>0, 1, 2, 3, 10</code>	Candidates are the l1 ids and llc id
<code>cache_controller[*].m_shared_memory_id</code>	int	<code>10</code>	

key	type	value	meaning
cache_controller[*].protocol_type	string	SNOOP_MESI	
cache_controller[*].fsm_filename	string	MESI_split Bus_snooping	
cache_controller[*].num_mshr	int	16	L1 MSHR depth (max concurrent outstanding misses). -1 = unbounded
cache_controller[*].m_data_handler.pwb_size	int	8	L1 write-back buffer depth. -1 = unbounded
cache_controller[*].processing_queue_size	int	16	L1 processing-queue depth (bounds demand admission). -1 = unbounded
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	SNOOP_LLC_MESI	
llc_controller.fsm_filename	string	MESI_LLC	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.num_mshr	int	64	LLC MSHR depth: larger than L1 (fans in all cores). -1 = unbounded
llc_controller.m_data_handler.pwb_size	int	32	LLC write-back buffer depth: larger than L1. -1 = unbounded
llc_controller.processing_queue_size	int	64	LLC processing-queue depth: larger than L1 (holds cache-to-cache forwarding concurrency). -1 = unbounded
llc_controller.arbiter_type	string	RRArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[*].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string	BMs/test DPrint/dprint.csv	
llc_controller.dprint.target	string	BMs/test DPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string		filter in hexadecimal
llc_controller.dprint.addr_filter	string		filter in hexadecimal
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses

key	type	value	meaning
main_memory_type	string	MCsim	
mcsim_scheduler	string	FRFCFS	

configuration/SystemConfigurations/MultiCoreSystem_Directory.csv

key	type	value	meaning
workload_path	string	BMs/TestBM/	Absolute path is recommended
num_cores	int	4	
bus_type	string list	TripleBus, Bus	Bus[0] is the interconnect between private caches and LLC and Bus[1] is between LLC and main memory
cache_controller_type	string	Cache Controller Directory	Type should match the name of the class
bus[*].buffers_max_size	int	32	
bus[1].m_lower_level_ids	int list	10	LLC ID
bus[1].upper_level_cache_ids	int list	100	Main memory ID
bus[1].controller_type	string	Point2Point	
bus[1].interconnect_controller.candidates_id	int list	0, 1, 2, 3, 10	Candidates are the l1 ids and llc id
bus[0].interconnect_controller.arbiter_type	string	FCFSArbiter	Pinned: every directory validation was done under FCFS. The shared SplitBusController default is TDM (for snoop) and directory+TDM starves one core on aifirf01/cacheb01 – an open bug rather than a config choice
cache_controller[*].m_shared_memory_id	int	10	
cache_controller[*].protocol_type	string	DIRECTORY_MSI	
cache_controller[*].fsm_filename	string	MSI_directory	
cache_controller[*].num_mshr	int	16	L1 MSHR depth (max concurrent outstanding misses). -1 = unbounded
cache_controller[*].m_data_handler.pwb_size	int	8	L1 write-back buffer depth. -1 = unbounded
cache_controller[*].processing_queue_size	int	16	L1 processing-queue depth (bounds demand admission). -1 = unbounded
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	DIRECTORY_LLC_MSI	
llc_controller.fsm_filename	string	MSI_LLC_directory	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	

key	type	value	meaning
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.num_mshr	int	64	LLC MSHR depth: larger than L1 (fans in all cores). -1 = unbounded
llc_controller.m_data_handler.pwb_size	int	32	LLC write-back buffer depth: larger than L1. -1 = unbounded
llc_controller.processing_queue_size	int	32	LLC processing-queue depth: larger than L1. -1 = unbounded
llc_controller.arbiter_type	string	FCFSArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[*].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string	BMs/test DPrint/dprint.csv	
llc_controller.dprint.target	string	BMs/test DPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string		filter in hexadecimal
llc_controller.dprint.addr_filter	string		filter in hexadecimal
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses

configuration/SystemConfigurations/MultiCoreSystem_Mesh.csv

key	type	value	meaning
workload_path	string	BMs/TestBM/	Absolute path is recommended
num_cores	int	4	
cache_controller_type	string	Cache Controller Directory	Type should match the name of the class
interconnect_type	string	NoC	
interconnect.buffer_max_size	int	32	
bus.buffer_max_size	int	32	
bus.m_lower_level_ids	int list	10	LLC ID
bus.upper_level_cache_ids	int list	100	Main memory ID
bus.controller_type	string	Point2Point	
bus.interconnect_controller.candidates_id	int list	0, 1, 2, 3, 10	Candidates are the l1 ids and llc id
cache_controller[*].m_shared_memory_id	int	10	
cache_controller[*].protocol_type	string	DIRECTORY_ MSI	

key	type	value	meaning
cache_controller[*].fsm_filename	string	MSI_directory	
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	DIRECTORY_LLCSMSI	
llc_controller.fsm_filename	string	MSI_LLCSdirectory	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.arbiter_type	string	RRArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[2].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string		
llc_controller.dprint.target	string	BMS/testDPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string	0x filter in hexadecimalFFFFFFFFFFFFFFFFC0	
llc_controller.dprint.addr_filter	string	0x filter in hexadecimalFFFFFFFFFFFFFFFFC0	
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses
cache_controller[2].dprint.cond_msg_id	int list	245788	

configuration/SystemConfigurations/MultiCoreSystem_Snoop.csv

key	type	value	meaning
workload_path	string	BMS/TestBM/	Absolute path is recommended
num_cores	int	4	
bus_type	string list	TripleBus, Bus	Bus[0] (L1<->LLC) is TripleBus: carries the LLC back-invalidation on its service channel. Bus[1] is LLC<->memory

key	type	value	meaning
bus[0].controller_type	string	Split	snoop broadcast bus
bus[0].m_lower_level_ids	int list	0, 1, 2, 3	LLC ID
bus[0].upper_level_cache_ids	int list	10	
cache_controller_type	string	Cache Controller	L1 private caches: basic snoop controller
llc_controller_type	string	Cache Controller_End2End	LLC: End2End controller drives TripleBus service channel for back-invalidation on eviction
bus[*].buffers_max_size	int	256	Response/request buffer depth: usable + in-flight reserve (sized to MSHR bound)
bus[0].response_reserve	int	128	Snoop bus in-flight response reserve: request bus stalls when response buffer free space falls to this (bounds response-bus back-pressure without overflow or controller gating)
bus[1].m_lower_level_ids	int list	10	LLC ID
bus[1].upper_level_cache_ids	int list	100	Main memory ID
bus[1].controller_type	string	Point2Point	Candidates are the l1 ids and llc id
bus[1].interconnect_controller.candidates_id	int list	0, 1, 2, 3, 10	
cache_controller[*].m_shared_memory_id	int	10	
cache_controller[*].protocol_type	string	SNOOP_MSI	
cache_controller[*].fsm_filename	string	MSI_split Bus_snooping	
cache_controller[*].num_mshr	int	16	L1 MSHR depth (max concurrent outstanding misses). -1 = unbounded
cache_controller[*].m_data_handler.pwb_size	int	8	L1 write-back buffer depth. -1 = unbounded
cache_controller[*].processing_queue_size	int	16	L1 processing-queue depth (bounds demand admission). -1 = unbounded
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	SNOOP_LLC_MSI	
llc_controller.fsm_filename	string	MSI_LLC	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.num_mshr	int	64	LLC MSHR depth: larger than L1 (fans in all cores). -1 = unbounded
llc_controller.m_data_handler.pwb_size	int	32	LLC write-back buffer depth: larger than L1. -1 = unbounded

key	type	value	meaning
llc_controller.processing_queue_size	int	64	LLC processing-queue depth: larger than L1 (holds cache-to-cache forwarding concurrency). -1 = unbounded
llc_controller.arbiter_type	string	FCFSArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[*].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string	BMs/test DPrint/dprint.csv	
llc_controller.dprint.target	string	BMs/test DPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string		filter in hexadecimal
llc_controller.dprint.addr_filter	string		filter in hexadecimal
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses

configuration/CacheControllers/BaseController.csv

key	type	value	meaning
m_id	int	-1	
m_shared_memory_id	int	-1	
m_clk_period	int	100	Clock period in nano seconds
processing_queue_size	int	64	If size is -1 the queue will grow indefinitely
protocol_type	string	SN00P_MSI	Type should be written similar to the names in ProtocolType enum in Protocols/Protocols.h
fsm_filename	string	MSI_split Bus_snooping	Name should match the name of the file in Protocols_FSM directory
perfect_llc	int	0	1 = this controller never issues to memory above it (every access hits

configuration/CacheControllers/CacheController.csv

Extends BaseController; every key below overrides or adds to it.

key	type	value	meaning
arbiter_type	string	NULL	Arbiters are Null, RR, FCFS, and TDM
arbiter_candidates_ids	int list	0, 1, 2, 3	

key	type	value	meaning
num_mshr	int	16	Max concurrent outstanding misses (MSHR depth). -1 = unbounded

configuration/CacheControllers/CacheControllerDirectory.csv

Extends CacheController; every key below overrides or adds to it.

key	type	value	meaning
processing_queue_size	int	32	Directory based protocols require larger sizes than snooping. A good practice is to set it to num_cores*max_mem_req

configuration/CacheControllers/CacheControllerExclusive.csv

Extends CacheController; every key below overrides or adds to it.

key	type	value	meaning

configuration/CacheControllers/CacheController_End2End.csv

Extends CacheController; every key below overrides or adds to it.

key	type	value	meaning

configuration/Interconnect/Bus.csv

key	type	value	meaning
m_lower_level_ids	int list	0, 1, 2, 3	
upper_level_cache_ids	int list	10	
controller_type	string	Unified	
buffers_max_size	int	16	
m_clk_period	int	50	Clk period in nano seconds

configuration/Interconnect/BusController.csv

key	type	value	meaning
m_request_latency	int	2	
m_response_latency	int	5	
arbiter_type	string	RRArbiter	Type should match the name of the arbiter class

configuration/Interconnect/DirectInterconnect.csv

key	type	value	meaning
buffers_max_size	int	16	Clk period in nano seconds
m_clk_period	int	50	

configuration/Interconnect/Mesh.csv

key	type	value	meaning
component_ids	int list	0, 1, 2, 3, 10	Clk period in nano seconds
buffers_max_size	int	16	
m_clk_period	int	50	first entry is the node id and the other entries are what the node is connected to
connection_matrix[0]	int list	0, 1, 2, 3, 10	
connection_matrix[1]	int list	1, 0, 2, 3, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[2]	int list	2, 0, 1, 3, 10	
connection_matrix[3]	int list	3, 0, 1, 2, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[4]	int list	10, 0, 1, 2, 3	

configuration/Interconnect/MeshController.csv

key	type	value	meaning
m_request_latency	int	2	Type should match the name of the arbiter class
m_data_latency	int	5	
arbiter_type	string	FCFSArbiter	

configuration/Interconnect/NoC.csv

Extends Mesh; every key below overrides or adds to it.

key	type	value	meaning
switch_ids	int list	200	Ids of the switches in the network first entry is the node id and the other entries are what the node is connected to
connection_matrix[0]	int list	0, 200, 10	

key	type	value	meaning
connection_matrix[1]	int list	1, 200, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[2]	int list	2, 200, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[3]	int list	3, 200, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[4]	int list	10, 0, 1, 2, 3	first entry is the node id and the other entries are what the node is connected to
connection_matrix[5]	int list	200, 0, 1, 2, 3	first entry is the node id and the other entries are what the node is connected to

configuration/Interconnect/NoCController.csv

Extends MeshController; every key below overrides or adds to it.

key	type	value	meaning
-----	------	-------	---------

configuration/Interconnect/Point2PointController.csv

Extends BusController; every key below overrides or adds to it.

key	type	value	meaning
candidates_id	int list		

configuration/Interconnect/SplitBusController.csv

Extends BusController; every key below overrides or adds to it.

key	type	value	meaning
arbiter_type	string	FCFSArbiter	Type should match the name of the arbiter class
response_reserve	int	0	Response-buffer slots reserved for in-flight responses; request bus stalls when free space falls to this. 0 = stall at full.

configuration/Interconnect/TripleBus.csv

Extends Bus; every key below overrides or adds to it.

key	type	value	meaning
controller_type	string	Triple	

configuration/Interconnect/TripleBusController.csv

Extends SplitBusController; every key below overrides or adds to it.

key	type	value	meaning
-----	------	-------	---------

configuration/Interconnect/UnifiedBusController.csv

Extends BusController; every key below overrides or adds to it.

key	type	value	meaning
-----	------	-------	---------

configuration/CPU.csv

key	type	value	meaning
m_clk_period	int	100	
m_number_of_0o0_requests	int	8	

configuration/CacheDataHandler.csv

key	type	value	meaning
cache_size	int	16384	Cache size in bytes
m_block_size	int	64	Block size in bytes
m_ways_count	int	1	Number of ways. If the number is 1 this will make the cache a direct-mapped cache
m_data_access_latency	int	0	The latency of reading or writing data. This latency does not apply on reading or writing cache lines bits
replacement_policy_name	string	LRU	Choose out of LRU or RANDOM
way_partition	string		Per-core way reservation "cores:ways;..." with a-b ranges (e.g. 0:0;1-3:1). Empty = no partitioning. Unlisted cores share the unclaimed ways

configuration/CacheDataHandler_COTS.csv

Extends CacheDataHandler; every key below overrides or adds to it.

key	type	value	meaning
pwb_size	int	8	Write-back buffer depth (pending write-backs). -1 = unbounded

configuration/DebugPrint.csv

key	type	value	meaning
m_clk_period	int	100	
target	string		Write file path or leave it empty to print on the console
enable	int	0	0 or 1
print_preamble	int	1	0 or 1
print_name	int	1	0 or 1
print_clk	int	0	0 or 1
print_msg_id	int	1	0 or 1
print_addr	int	0	0 or 1
print_from	int	1	0 or 1
print_to	int	1	0 or 1
print_owner	int	1	0 or 1
addr_filter	string		filter in hexadecimal
cond_addr	string list		Acceptable Addresses
cond_msg_id	int list		Acceptable Ids
cond_from	int list		Acceptable Senders
cond_owner	int list		Acceptable Owners

configuration/MainMemoryController.csv

key	type	value	meaning
m_clk_period	int	100	
m_id	int	100	
m_llc_id	int	10	
m_memory_latency	int	250	Latency in cycles
processing_queue_size	int	32	Request buffer depth. -1 = unbounded (legacy behavior)

The shipped coherence FSMs

The transition tables of the protocol families that ship as complete presets (snoop MSI and MESI, directory MSI) and the snoop MOESI pair, generated from `Protocols_FSM/*.csv` by `docs/manual/tools/gen_fsm_tables.py`. Rows are states (**bold** = stable, *italic* = transient), columns are events, a cell is actions/next_state. Chapter 6 explains how to read and edit them.

B.1 MSI_splitBus_snooping

Table B.1. MSI_splitBus_snooping: the numbered lists the table below is written in.

events	actions	states
	0 Stall	0 I
	1 Hit	1 IS_ad
	2 GetS	2 IS_d
	3 GetM	3 IS_a
	4 PutM	4 IS_dI
	5 Data2Req	5 IM_ad
	6 Data2Both	6 IM_d
	7 SaveReq	7 IM_a
	8 Fault	8 IM_dI
		9 IM_dS
		10 IM_dSI
		11 S
		12 SM_ad
		13 SM_d
		14 SM_a
		15 SM_dI
		16 SM_dS
		17 SM_dSI
		18 M
		19 MI_a
		20 MII_a
		21 II_a

Table B.2. MSI_splitBus_snooping: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Own_GetS	Own_GetM	Own_PutM	Other_GetS	Other_GetM	Other_PutM	OwnData	Invalidation
I	GetS/IS_ad	GetM/IM_ad		Fault	Fault	Fault				Fault	
<i>IS_ad</i>	Stall/	Stall/	Stall/	IS_d	Fault	Fault				IS_a	
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IS_dI		Hit/S	IS_dI
<i>IS_a</i>	Stall/	Stall/	Stall/	Hit/S	Fault	Fault			Fault	Fault	IS_ad
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/I	
<i>IM_ad</i>	Stall/	Stall/	Stall/	Fault	IM_d	Fault				IM_a	
<i>IM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IM_dS	SaveReq/IM_dI		Hit/M	SaveReq/IM_dI
<i>IM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault				Fault	IM_ad
<i>IM_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I	
<i>IM_dS</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IM_dSI		Hit/Data2Both/S	IM_dSI
<i>IM_dSI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I	
S	Hit/	GetM/SM_ad	I	Fault	Fault	Fault		I		Fault	I
<i>SM_ad</i>	Hit/	Stall/	Stall/	Fault	SM_d	Fault		IM_ad		SM_a	IM_ad
<i>SM_d</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/SM_dS	SaveReq/SM_dI		Hit/M	SaveReq/IM_dI
<i>SM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault		IM_a	Fault	Fault	IM_a
<i>SM_dI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I	
<i>SM_dS</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault		SM_dSI		Hit/Data2Both/S	SM_dSI
<i>SM_dSI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I	
M	Hit/	Hit/	PutM/MI_a	Fault	Fault	Fault	Data2Both/S	Data2Req/I		Fault	Data2Req/I
<i>MI_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I	Data2Both/II_a	Data2Req/II_a		Fault	MII_a
<i>MII_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I				Fault	
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	I				Fault	

B.2 MSI_LLC

Table B.3. MSI_LLC: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NIorS_d
2 Replacement	2 SendData	2 NM_d
3 PutM_fromOwner	3 SaveData	3 IorS
4 PutM_fromNonOwner	4 SetOwner	4 M
5 Data_fromLowerInterface	5 ClearOwner	5 IorS_d
6 Data_fromUpperInterface	6 IssueInv	6 IorS_a
7 Own_Invalidation	7 WriteBack	7 N_d
	8 Fault	8 N_a

Table B.4. MSI_LLC: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	<i>GetS</i>	<i>GetM</i>	<i>Replacement</i>	<i>PutM_fromOwner</i>	<i>PutM_fromNonOwner</i>	<i>Data_fromLowerInterface</i>	<i>Data_fromUpperInterface</i>	<i>Own_Invalidation</i>
N	GetData/NIorS_d	GetData/SetOwner/NM_d		Fault		Fault	Fault	Fault
<i>NIorS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/IorS	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/M	Fault
IorS	SendData/	SendData/SetOwner/M	IssueInv/	Fault		Fault	Fault	WriteBack/N
M	ClearOwner/IorS_d	SetOwner/	IssueInv/	ClearOwner/IorS_d		SaveData/IorS_a	Fault	ClearOwner/N_d
<i>IorS_d</i>	Stall/	Stall/	Stall/	Stall/		SaveData/IorS	Fault	N_d
<i>IorS_a</i>	ClearOwner/IorS	Stall/	Stall/	ClearOwner/IorS		Fault	Fault	WriteBack/N_a
<i>N_d</i>	Stall/	Stall/	Stall/	Fault		WriteBack/N	Fault	Fault
<i>N_a</i>	ClearOwner/N	Stall/	Stall/	ClearOwner/N		Fault	Fault	Fault

B.3 MESI_splitBus_snooping

Table B.5. MESI_splitBus_snooping: the numbered lists the table below is written in.

events	actions	states
0 Load	0 Stall	0 I
1 Store	1 Hit	1 IS_ad
2 Replacement	2 GetS	2 IS_d
3 Own_GetS	3 GetM	3 IS_a
4 Own_GetM	4 PutM	4 IS_dI
5 Own_PutM	5 Data2Req	5 IM_ad
6 Other_GetS	6 Data2Both	6 IM_d
7 Other_GetM	7 SaveReq	7 IM_a
8 Other_PutM	8 Fault	8 IM_dI
9 OwnData	9 removeSavedReq	9 IM_dS
10 Invalidation		10 IM_dSI
11 OwnData_Exclusive		11 S
		12 SM_ad
		13 SM_d
		14 SM_a
		15 SM_dI
		16 SM_dS
		17 SM_dSI
		18 M
		19 MI_a
		20 II_a
		21 E
		22 IES_d
		23 IES_dI

Table B.6. MESI_splitBus_snooping: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Own_GetS	Own_GetM	Own_PutM	Other_GetS	Other_GetM	Other_PutM	OwnData	Invalidation	OwnData_Exclusive
I	GetS/IS_ad	GetM/IM_ad		Fault	Fault	Fault				Fault		Fault
<i>IS_ad</i>	Stall/	Stall/	Stall/	IS_d	Fault	Fault				IS_a		Fault
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IES_d	SaveReq/IS_dI		Hit/S	IS_dI	Hit/E
<i>IS_a</i>	Stall/	Stall/	Stall/	Hit/S	Fault	Fault			Fault	Fault	IS_ad	Fault
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/I		Hit/Data2Req/I
<i>IM_ad</i>	Stall/	Stall/	Stall/	Fault	IM_d	Fault				IM_a		Fault
<i>IM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IM_dS	SaveReq/IM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>IM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault				Fault	IM_ad	Fault
<i>IM_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>IM_dS</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IM_dSI		Hit/Data2Both/S	IM_dSI	Fault
<i>IM_dSI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I		Fault
S	Hit/	GetM/SM_ad	I	Fault	Fault	Fault		I		Fault	I	Fault
<i>SM_ad</i>	Hit/	Stall/	Stall/	Fault	SM_d	Fault		IM_ad		SM_a	IM_ad	Fault
<i>SM_d</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/SM_dS	SaveReq/SM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>SM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault		IM_a	Fault	Fault	IM_a	Fault
<i>SM_dI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>SM_dS</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault		SM_dSI		Hit/Data2Both/S	SM_dSI	Fault
<i>SM_dSI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I		Fault
M	Hit/	Hit/	PutM/MI_a	Fault	Fault	Fault	Data2Both/S	Data2Req/I		Fault	Data2Req/I	Fault
<i>MI_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I	Data2Both/II_a	Data2Req/II_a		Fault	Data2Req/II_a	Fault
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	I				Fault		Fault
E	Hit/	Hit/M	PutM/MI_a	Fault	Fault	Fault	Data2Both/S	Data2Req/I		Fault	Data2Req/I	Fault
<i>IES_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IES_dI		Hit/removeSavedReq/S	IS_dI	Hit/Data2Both/S
<i>IES_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/S	IS_dI	Hit/Data2Both/I

B.4 MESI_LLC

Table B.7. MESI_LLC: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NE_d
2 Replacement	2 SendData	2 NM_d
3 PutM_fromOwner	3 SaveData	3 I
4 PutM_fromNonOwner	4 SetOwner	4 S
5 Data_fromLowerInterface	5 ClearOwner	5 EorM
6 Data_fromUpperInterface	6 IssueInv	6 IorS_a
7 Own_Invalidation	7 WriteBack	7 MN_d
	8 Fault	8 S_d
	9 SendExclusiveData	9 I_d
		10 N_a

Table B.8. MESI_LLC: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	Replacement	PutM_fromOwner	PutM_fromNonOwner	Data_fromLowerInterface	Data_fromUpperInterface	Own_Invalidation
N	GetData/SetOwner/NE_d	GetData/SetOwner/NM_d		Fault		Fault	Fault	SaveData/N
<i>NE_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendExclusiveData/EorM	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/EorM	Fault
I	SendExclusiveData/SetOwner/EorM	SendData/SetOwner/EorM	IssueInv/	Fault		Fault	Fault	WriteBack/N
S	SendData/	SendData/SetOwner/EorM	IssueInv/	Fault		Fault	Fault	WriteBack/N
EorM	ClearOwner/S_d	SetOwner/	IssueInv/	ClearOwner/I_d		SaveData/IorS_a	Fault	ClearOwner/MN_d
<i>IorS_a</i>	ClearOwner/S	Stall/	Stall/	ClearOwner/I		Fault	Fault	WriteBack/N_a
<i>MN_d</i>	Stall/	Stall/	Stall/	Fault		WriteBack/N	Fault	SaveData/MN_d
<i>S_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/S	Fault	MN_d
<i>I_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/I	Fault	MN_d
<i>N_a</i>	ClearOwner/N	Stall/	Stall/	ClearOwner/N		Fault	Fault	Fault

B.5 MOESI_splitBus_snooping

Table B.9. MOESI_splitBus_snooping: the numbered lists the table below is written in.

events	actions	states
0 Load	0 Stall	0 I
1 Store	1 Hit	1 IS_ad
2 Replacement	2 GetS	2 IS_d
3 Own_GetS	3 GetM	3 IS_a
4 Own_GetM	4 PutM	4 IS_dI
5 Own_PutM	5 Data2Req	5 IM_ad
6 Other_GetS	6 Data2Both	6 IM_d
7 Other_GetM	7 SaveReq	7 IM_a
8 Other_PutM	8 Fault	8 IM_dI
9 OwnData	9 removeSavedReq	9 IM_d0
10 Invalidation		10 S
11 OwnData_Exclusive		11 SM_ad
		12 SM_d
		13 SM_a
		14 SM_dI
		15 SM_d0
		16 M
		17 MI_a
		18 II_a
		19 E
		20 O
		21 OM_a
		22 OI_a
		23 EI_a
		24 IES_d
		25 IES_dI
		26 EM_a
		27 IM_dOI
		28 SM_dOI

Table B.10. M0ESI_splitBus_snooping: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Own_GetS	Own_GetM	Own_PutM	Other_GetS	Other_GetM	Other_PutM	OwnData	Invalidation	OwnData_Exclusive
I	GetS/IS_ad	GetM/IM_ad		Fault	Fault	Fault				Fault		Fault
<i>IS_ad</i>	Stall/	Stall/	Stall/	IS_d	Fault	Fault				IS_a		Fault
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IES_d	SaveReq/IS_dI		Hit/S	IS_dI	Hit/E
<i>IS_a</i>	Stall/	Stall/	Stall/	Hit/S	Fault	Fault			Fault	Fault	IS_ad	Fault
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/I		Hit/Data2Req/I
<i>IM_ad</i>	Stall/	Stall/	Stall/	Fault	IM_d	Fault				IM_a		Fault
<i>IM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IM_d0	SaveReq/IM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>IM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault				Fault	IM_ad	Fault
<i>IM_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>IM_dO</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/	SaveReq/IM_dOI		Hit/Data2Req/0	SaveReq/IM_dOI	Fault
S	Hit/	GetM/SM_ad	I	Fault	Fault	Fault		I		Fault	I	Fault
<i>SM_ad</i>	Hit/	Stall/	Stall/	Fault	SM_d	Fault		IM_ad		SM_a	IM_ad	Fault
<i>SM_d</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/SM_d0	SaveReq/SM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>SM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault		IM_a	Fault	Fault	IM_a	Fault
<i>SM_dI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>SM_dO</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/	SaveReq/SM_dOI		Hit/Data2Req/0	SaveReq/SM_dOI	Fault
M	Hit/	Hit/	PutM/MI_a	Fault	Fault	Fault	Data2Req/0	Data2Req/I		Fault	Data2Req/I	Fault
<i>MI_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I	Data2Req/OI_a	Data2Req/II_a		Fault	Data2Req/II_a	Fault
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	I				Fault		Fault
E	Hit/	GetM/EM_a	PutM/EI_a	Fault	Fault	Fault	Data2Req/S	Data2Req/I		Fault	I	Fault
O	Hit/	GetM/OM_a	PutM/OI_a	Fault	Fault	Fault	Data2Req/	Data2Req/I		Fault	Data2Req/I	Fault
<i>OM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault	Data2Req/	Data2Req/IM_ad		Fault	Data2Req/IM_ad	Fault
<i>OI_a</i>	Hit/	Stall/	Stall/	Fault	Fault	Data2Req/I	Data2Req/	Data2Req/II_a		Fault	Data2Req/II_a	Fault
<i>EI_a</i>	Hit/	Stall/	Stall/	Fault	Fault	I	Data2Req/	Data2Req/II_a		Fault	II_a	Fault
<i>IES_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IES_dI		Hit/removeSavedReq/S	IS_dI	Hit/Data2Req/S
<i>IES_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/S	IS_dI	Hit/Data2Req/I
<i>EM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault	Data2Req/SM_ad	Data2Req/IM_ad	Stall/	Fault	IM_ad	Fault
<i>IM_dOI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>SM_dOI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault

B.6 MOESI_LLC

Table B.11. MOESI_LLC: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NE_d
2 Replacement	2 SendData	2 NM_d
3 PutM_fromOwner	3 SaveData	3 I
4 PutM_fromNonOwner	4 SetOwner	4 S
5 Data_fromLowerInterface	5 ClearOwner	5 E
6 Data_fromUpperInterface	6 IssueInv	6 I_a
7 Own_Invalidation	7 WriteBack	7 MN_d
	8 Fault	8 S_d
	9 SendExclusiveData	9 I_d
		10 O
		11 S_a
		12 O_d
		13 M

Table B.12. M0ESI_LLC: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	Replacement	PutM_fromOwner	PutM_fromNonOwner	Data_fromLowerInterface	Data_fromUpperInterface	Own_Invalidation
N	GetData/SetOwner/NE_d	GetData/SetOwner/NM_d		Fault		Fault	Fault	SaveData/N
<i>NE_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendExclusiveData/E	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/M	Fault
I	SendExclusiveData/SetOwner/E	SendData/SetOwner/M	IssueInv/	Fault		Fault	Fault	WriteBack/N
S	SendData/	SendData/SetOwner/M	IssueInv/	Fault		Fault	Fault	WriteBack/N
E	ClearOwner/S	SetOwner/M	IssueInv/	ClearOwner/I		Fault	Fault	ClearOwner/WriteBack/N
<i>I_a</i>	Stall/	Stall/	Stall/	ClearOwner/I		Fault	Fault	Fault
<i>MN_d</i>	Stall/	Stall/	Stall/	Fault		WriteBack/N	Fault	SaveData/MN_d
<i>S_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/S	Fault	MN_d
<i>I_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/I	Fault	MN_d
O		SetOwner/M	IssueInv/	<i>S_d</i>		SaveData/S	Fault	ClearOwner/MN_d
<i>S_a</i>	Stall/	Stall/	Stall/	ClearOwner/S		Fault	Fault	Fault
<i>O_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/O	Fault	MN_d
M	0	SetOwner/M	IssueInv/	ClearOwner/I_d		SaveData/I_a	Fault	ClearOwner/MN_d

B.7 MSI_directory

Table B.13. MSI_directory: the numbered lists the table below is written in.

events	actions	states
0 Load	0 Stall	0 I
1 Store	1 Hit	1 IS_d
2 Replacement	2 GetS	2 IS_dI
3 Fwd_GetS	3 GetM	3 IM_ad
4 Fwd_GetM	4 PutS	4 IM_a
5 Inv	5 PutM_Data	5 IM_aS
6 Put_Ack	6 Data2Both	6 IM_aSI
7 Data_zeroAck	7 Data2Req	7 IM_aI
8 Data_nonZeroAck	8 InvAck2Req	8 S
9 DataFromOwner	9 InvAck_Data	9 SM_ad
10 InvAck	10 Ack_dec	10 SM_a
11 lastInvAck	11 AckNum_set	11 M
	12 SaveReq	12 MI_a
	13 Fault	13 SI_a
		14 II_a

Table B.14. MSI_directory: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Fwd_GetS	Fwd_GetM	Inv	Put_Ack	Data_zeroAck	Data_nonZeroAck	DataFromOwner	InvAck	LastInvAck
I	GetS/IS_d	GetM/IM_ad		Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	InvAck2Req/IS_dI	Fault	Hit/S	Fault	Hit/S	Fault	Fault
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Hit/I	Fault	Hit/I	Fault	Fault
<i>IM_ad</i>	Stall/	Stall/	Stall/	Stall/	Stall/	Stall/	Fault	Hit/M	AckNum_set/IM_a	Hit/M	Ack_dec/	Fault
<i>IM_a</i>	Stall/	Stall/	Stall/	SaveReq/IM_aS	SaveReq/IM_aI	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Ack_dec/M
<i>IM_aS</i>	Stall/	Stall/	Stall/	Fault	Fault	InvAck2Req/IM_aSI	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Ack_dec/Data2Both/S
<i>IM_aSI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Data2Both/Ack_dec/I
<i>IM_aI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Data2Req/Ack_dec/I
S	Hit/	GetM/SM_ad	PutS/SI_a	Fault	Fault	InvAck2Req/I	Fault	Fault	Fault	Fault	Fault	Fault
<i>SM_ad</i>	Hit/	Stall/	Stall/	Stall/	Stall/	InvAck2Req/IM_ad	Fault	Hit/M	AckNum_set/SM_a	Fault	Ack_dec/	Fault
<i>SM_a</i>	Hit/	Stall/	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Ack_dec/M
M	Hit/	Hit/	PutM_Data/MI_a	Data2Both/S	Data2Req/I	InvAck_Data/I	Fault	Fault	Fault	Fault	Fault	Fault
<i>MI_a</i>	Stall/	Stall/	Stall/	Data2Both/SI_a	Data2Req/II_a		I	Fault	Fault	Fault	Fault	Fault
<i>SI_a</i>	Stall/	Stall/	Stall/	Fault	Fault	InvAck2Req/II_a	I	Fault	Fault	Fault	Fault	Fault
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	I	Fault	Fault	Fault	Fault	Fault

B.8 MSI_LLC_directory

Table B.15. MSI_LLC_directory: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NS_d
2 Replacement	2 SendData	2 NM_d
3 PutS	3 SaveData	3 I
4 last_PutS	4 SetOwner	4 S
5 InvAck	5 ClearOwner	5 M
6 last_InvAck	6 SendInv	6 S_d
7 PutM_Data_fromOwner	7 WriteBack	7 N_d
8 PutM_Data_fromNonOwner	8 FwdGetS	8 N_a
9 Data_fromLowerInterface	9 FwdGetM	
10 Data_fromUpperInterface	10 PutAck	
	11 IncSharers	
	12 DecSharers	
	13 Fault	

Table B.16. MSI_LLC_directory: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	RepPlacement	PutS	Last_PutS	InvAck	Last_InvAck	PutM_Data_FromOwner	PutM_Data_FromNonOwner	Data_FromLowerInterface	Data_FromUpperInterface
N	GetData/IncSharers/NS_d	SetOwner/GetData/NM_d	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault
<i>NS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault
I	SendData/IncSharers/S	SetOwner/SendData/M	WriteBack/N	PutAck/	PutAck/	Fault	PutAck/	Fault	Fault	Fault	Fault
S	SendData/IncSharers/	SetOwner/SendData/SendInv/M	SendInv/WriteBack/N_a	PutAck/DecSharers/	PutAck/DecSharers/I	Fault	Fault	Fault	Fault	Fault	Fault
M	ClearOwner/FwdGetS/IncSharers/S_d	SetOwner/FwdGetM/	SendInv/N_d	PutAck/	PutAck/	DecSharers/	DecSharers/	ClearOwner/DecSharers/PutAck/SaveData/I	PutAck/DecSharers/	Fault	Fault
<i>S_d</i>	Stall/	Stall/	Stall/	Stall/	Stall/	Fault	Fault	Fault	PutAck/DecSharers/	SaveData/S	Fault
<i>N_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	ClearOwner/DecSharers/WriteBack/N	ClearOwner/DecSharers/PutAck/WriteBack/N	PutAck/DecSharers/	DecSharers/WriteBack/N	Fault
<i>N_a</i>	Stall/	Stall/	Stall/	PutAck/	PutAck/	DecSharers/	DecSharers/N	Fault	PutAck/	Fault	Fault

Command line, scripts and environment

C.1 The simulator

```
./build/Octopus_Simulator -s <SystemClass> [-p "<key>=<value>"]...
```

- s <name>** the system to build: a C++ class in `src/SystemConfigurations/` and its CSV of the same name in `configuration/SystemConfigurations/`. `MultiCoreSystem(bus)` and `MultiCoreSystem_Mesh` ship.
- p "<key>=<value>"** override one configuration key; repeat for several. The key carries its type suffix and, for an instance, its index or `[*]` (Chapter 10). Values with spaces or semicolons need the quotes.

The parser (`CLParser`) pairs each `-x` with the argument that follows it, so `-p key=value` and `-p "key=value"` are the same; it prints every argument it saw at start-up. The one key every run needs is `workload_path(s)`: a directory holding `trace_C<n>.trc`. shared per core (or `task_C<n>.task.csv` for a periodic task); the reports are written into `<workload>/newLogger/`.

C.2 Scripts

C.3 Environment variables

Table C.1. The driver scripts at the repository root and under scripts/, sweeps/, demo/.

script	what it does
get_benchmarks.sh	clones the benchmark repository (FanosResearch/OctopusBMs) into BMs/ if absent and inflates the compressed SPLASH-2 traces; idempotent, called by every driver. BMS_SPARSE="eembc-traces" clones only that suite
scripts/check_environment.sh	the PASS/FAIL line per component (compiler, cmake, xz, Python packages, binary, MCSim ini line endings, benchmarks), then a reference run whose request count and worst DRAM latency must match the committed numbers
run_octopus.sh	-protocol snoop directory -suite eembc splash [-bench <name>] [-jobs N] [-safety <s>] [-out <dir>]: copies the preset over MultiCoreSystem.csv, runs the suite in parallel and judges completion per core (own trace covered and footer written)
run_splash.sh, sweep_protocols.sh	the SPLASH driver and the coherence-family sweep
octoviz.sh	run convert serve view <dir> [- <simulator args>]: the visualizer entry point; WINDOW=t0:t1, TRACE=0, PORT, TIMEOUT, SYSTEM
sweeps/sweep_*.sh	one axis each (arbiter, replacement, cache, memory, pcc_ab, pcc_par); SUITE, JOBS, BENCH, SAFETY, EXCLUDE; sweep_all.sh runs them all, run_overnight.sh both suites
sweeps/plot_results.py, analyze_*.sh	the figures and the mechanism / trace analyses over sweep output
demo/run_acts.sh	the three acts of the interference demo (act1 act2 act3); demo/extract_jobs.py, build_page.py, live.sh
demo_protocol.sh	the MI-versus-MESI example of Chapter 6
docs/manual/build.sh	this manual

Table C.2. Environment switches read by the simulator. All are zero-cost when unset.

variable	effect
OCTOPUS_TRACE=<file>	write the raw event trace (Chapter 11)
OCTOPUS_TRACE_WINDOW=t0:t1	keep only trace events inside this core-cycle window
OCTOPUS_EVENT_DUMP=N	print the raw Logger timelines of the first <i>N</i> slow requests (Total above the threshold, or any memory-bus crossing)
OCTOPUS_EVENT_DUMP_MIN=<cycles>	the Total threshold for the dump (default 100)
OCTOPUS_EVENT_DEBUG=1	report any request whose stages do not tile to Total, and the end-of-run ok/fail/noresp tally
OCTOPUS_NO_LOG=1	skip per-request logging; still write Summary.csv
OCTOPUS_HANG_DUMP=<cycles>	dump every controller that has been idle with a non-empty queue for that long, and finalize the trace
MCSIM_PROBE=<file>	log every MCSim request, command and completion with the DRAM clock

File formats

D.1 LatencyReport_C<n>.csv

One file per core in <workload>/newLogger/: a header, one row per completed request in completion order, then two footer rows (per-column worst case, then averages). Columns, 1-indexed as the scripts address them:

#	column	meaning
1	RequestID	the global msg_id; joins with Debugger traces and the raw trace
2	Request Address	
3	Ready Cycle	the simulation cycle at which the request could first issue; Ready + CPU is the issue cycle, so rows are time-resolvable
4	CPU Latency	issue – ready
5–12	L1 Stall, Requist Bus, L2 Stall, L2 Access, Response Bus, L2-DRAM Bus, DRAM, L1 Access Latency	the stage columns, Chapter 7
13	Total Latency	the sum of 5–12
14	Effective Latency	the non-overlapped part of Total
15	Oldest Latency	head-of-queue latency; 0 if the request was never the oldest
16–22	LLC Arrival State, LLC Gate, Resp Ahead, Resp Ahead Refills, Array Ahead, Array Ahead Writes, LLC Stall State	the mechanism trackers, Chapter 11

D.2 Summary.csv

One row per core: Worst-case L1 Stall Latency, Worst-case Request Bus Latency, Worst-case L2 Stall Latency, Worst-case L2 Access Latency, Worst-case Response Bus Latency, Worst-case L2-DRAM Bus Latency, Worst-case DRAM Latency, Worst-case L1 Access Latency, Worst-case Total Latency, Worst-case Effective Latency, Average Latency (the mean Effective), Finish Cycle (the core's last completion, its total execution time) and Worst-case Oldest Latency, appended last so the earlier indices stay stable. This is the machine-readable hook every sweep reads.

D.3 JobReport_C<n>.csv

Written for a core running a periodic task (Chapter 12), one row per job, flushed as each job finishes so the file can be tailed during a run:

```
job,release_cycle,finish_cycle,deadline_cycle,deadline_miss,n_accesses,
  skipped_periods,mean_access_lat,max_access_lat
```

The last two are the job's mean and worst access latency as the core sees it, issue to response.

D.4 The task program, task_C<n>.task.csv

Column 1 is the row kind. attr rows set period, jobs (−1 = free-running), ooo, deadline, start; phase rows are the job body, replayed once per job: COMPUTE, cycles; READ, addr; WRITE, addr; LINEAR, base, count, stride, rd_ratio; RANDOM, base, range, count, rd_ratio. Numbers accept hex or decimal; # lines and blank lines are ignored.

D.5 The sweep result, results/<axis>/<suite>.csv

```
value,benchmark,status,avg,wcTotal,wcEff,wcL1stall,wcReqBus,wcL2stall,
  wcL2access,wcRespBus,wcDramBus,wcDRAM,wcL1access,finish,wcOldest
```

status is OK, INCOMPLETE or TIMEOUT; the metrics are the Summary.csv worst cases over all cores.

D.6 The raw event trace

A file written under OCTOPUS_TRACE: a magic, records in 64 K-record chunks, a chunk index (offset, count, cycle and id ranges) and a trailer, so a windowed read decodes only the chunks it needs and a file without a trailer (a killed run) is still readable. Each record is 32 bytes, little-endian:

field	type	meaning
cycle	u64	global core cycle
msg_id	u64	the message id
addr	u64	the message's address
resource	u8	0 CPU, 1 L1, 2 REQ_BUS, 3 RESP_BUS, 4 LLC, 5 MEM_BUS, 6 DRAM, 8 SVC_BUS, 9 ARRAY, 10 LLC_QUEUE, 11 FSM
phase	u8	0 ENTER, 1 SERVICE, 2 EXIT; for FSM records the event id
kind	u8	1 DEMAND, 2 GETS, 3 GETM, 4 PUTM, 5 INV, 6 MEM_READ, 7 EVICT, 8 WB_DATA, 9 WB_INV, 10 SUPPLY, 11 SUPPLY_DEFERRED, 12 RESP, 13 FILL, 14 FILL_ROLLBACK, 15 MEM_WRITE; 0 untagged
core	u8	the message's owner
comp	u16	component id (L1 0–3, LLC 10, DRAM 100; bus 0 = L1–LLC, 1 = memory bus)
flags	u16	ARRAY: the action that claimed the port (1 read for a response, 4 read for a write-back or supply, 6 write of a fill, write-back or supply), bit 8 cut-in, bit 9 MSHR-resident, bit 10 write-back-buffer-resident. FSM: old_state « 8 new_state

At close the Logger writes <file>.names with each controller's state and event names, in FSM CSV order, which the converter uses to resolve FSM records. tools/octoviz/convert.py <newLogger> −trace <file> produces octoviz.parquet (the rows), occupancy.parquet (one busy interval per resource per message) and fsm.parquet (one row per transition).

Index of configuration keys

addr_filter, 47, 76
arbiter_candidates_ids, 71
arbiter_type, 36, 71–74

buffers_max_size, 35, 72, 73
bus.buffers_max_size, 68
bus.controller_type, 68
bus.interconnect_controller.candidates_id, 68
bus.m_lower_level_ids, 68
bus.upper_level_cache_ids, 68
bus[*].buffers_max_size, 65, 67, 70
bus[0].controller_type, 65, 70
bus[0].interconnect_controller.arbiter_type, 67
bus[0].m_lower_level_ids, 65, 70
bus[0].response_reserve, 65, 70
bus[0].upper_level_cache_ids, 65, 70
bus[1].controller_type, 65, 67, 70
bus[1].interconnect_controller.candidates_id, 65, 67, 70
bus[1].m_lower_level_ids, 65, 67, 70
bus[1].upper_level_cache_ids, 65, 67, 70
bus_type, 55, 65, 67, 69

cache_controller[*].dprint.addr_filter, 66, 68, 69, 71
cache_controller[*].dprint.cond_addr, 66, 68, 69, 71
cache_controller[*].dprint.enable, 66, 68, 71
cache_controller[*].dprint.target, 66, 68, 69, 71
cache_controller[*].fsm_filename, 66, 67, 69, 70
cache_controller[*].m_data_handler.pwb_size, 66, 67, 70
cache_controller[*].m_shared_memory_id, 65, 67, 68, 70
cache_controller[*].num_mshr, 66, 67, 70
cache_controller[*].processing_queue_size, 66, 67, 70
cache_controller[*].protocol_type, 66–68, 70
cache_controller[0].m_id, 66, 67, 69, 70
cache_controller[1].m_id, 66, 67, 69, 70
cache_controller[2].dprint.cond_msg_id, 69
cache_controller[2].dprint.enable, 69
cache_controller[2].m_id, 66, 67, 69, 70
cache_controller[3].m_id, 66, 67, 69, 70
cache_controller_type, 65, 67, 68, 70
cache_size, 75
candidates_id, 74

component_ids, 36, 73
cond_addr, 46, 47, 76
cond_from, 46, 76
cond_msg_id, 46, 76
cond_owner, 46, 76
connection_matrix[0], 73
connection_matrix[1], 73, 74
connection_matrix[2], 73, 74
connection_matrix[3], 73, 74
connection_matrix[4], 73, 74
connection_matrix[5], 74
connection_matrix[i], 36
controller_type, 72, 74

enable, 47, 76

fsm_filename, 71

interconnect.buffers_max_size, 68
interconnect_type, 68

llc_controller.arbiter_candidates_ids, 66, 68, 69, 71
llc_controller.arbiter_type, 36, 66, 68, 69, 71
llc_controller.dprint.addr_filter, 66, 68, 69, 71
llc_controller.dprint.cond_addr, 66, 68, 69, 71
llc_controller.dprint.enable, 66, 68, 69, 71
llc_controller.dprint.target, 66, 68, 69, 71
llc_controller.fsm_filename, 66, 67, 69, 70
llc_controller.m_data_handler.cache_size, 66, 67, 69, 70
llc_controller.m_data_handler.m_data_access_latency, 66, 68–70
llc_controller.m_data_handler.m_ways_count, 66, 67, 69, 70
llc_controller.m_data_handler.pwb_size, 66, 68, 70
llc_controller.m_id, 66, 67, 69, 70
llc_controller.m_shared_memory_id, 66, 67, 69, 70
llc_controller.num_mshr, 66, 68, 70
llc_controller.processing_queue_size, 66, 68, 71
llc_controller.protocol_type, 66, 67, 69, 70
llc_controller_type, 65, 70

m_block_size, 75
m_clk_period, 34, 71–73, 75, 76

m_data_access_latency, 75
 m_data_handler.m_data_access_latency, 14
 m_data_handler.pwb_size, 14
 m_data_latency, 73
 m_id, 71, 76
 m_llc_id, 76
 m_lower_level_ids, 72
 m_main_memory.m_id, 66, 68, 69, 71
 m_main_memory.m_llc_id, 66, 68, 69, 71
 m_memory_latency, 31, 38, 76
 m_number_of_OoO_requests, 9, 75
 m_request_latency, 34, 72, 73
 m_response_latency, 34, 72
 m_shared_memory_id, 71
 m_ways_count, 75
 main_memory_type, 38, 39, 67
 mcsim_scheduler, 38, 67

 num_cores, 8, 65, 67–69
 num_mshr, 14, 72

 perfect_llc, 71

 print_addr, 46, 76
 print_clk, 46, 76
 print_from, 46, 76
 print_msg_id, 46, 76
 print_name, 46, 76
 print_owner, 46, 76
 print_preamble, 76
 print_to, 46, 76
 processing_queue_size, 13, 38, 71, 72, 76
 protocol_type, 71
 pwb_size, 75

 replacement_policy_name, 75
 response_reserve, 35, 74

 switch_ids, 36, 73

 target, 46, 76

 upper_level_cache_ids, 72

 way_partition, 75
 workload_path, 65, 67–69

Index

ClearOwner (FSM action), 15, 25

Data2Both (FSM action), 14, 17, 25, 26

Data2Req (FSM action), 25, 26

Fault (FSM action), 25

GetData (FSM action), 20, 25, 38

GetM (FSM action), 25

GetS (FSM action), 14, 25, 26

Hit (FSM action), 25

IssueInv (FSM action), 14, 15, 17, 20, 22, 25

MCSIM_PROBE (environment), 39, 48

OCTOPUS_DUMP_AT (environment), 59

OCTOPUS_EVENT_DEBUG (environment), 45

OCTOPUS_EVENT_DUMP (environment), 33, 45

OCTOPUS_EVENT_DUMP_MIN (environment), 33, 45

OCTOPUS_HANG_DUMP (environment), 48, 62

OCTOPUS_NO_LOG (environment), 45

OCTOPUS_TRACE (environment), 47, 96

OCTOPUS_TRACE_ADDR (environment), 59

OCTOPUS_TRACE_WINDOW (environment), 47

PutM (FSM action), 25

removeSavedReq (FSM action), 25

SaveData (FSM action), 17, 25

SaveReq (FSM action), 25, 26

SendData (FSM action), 25

SendExclusiveData (FSM action), 25

SetOwner (FSM action), 15, 25

Stall (FSM action), 25

WriteBack (FSM action), 22, 25, 38